

CAUSAL MAP GARDEN

Working Papers

Contents

Working Papers

Minimalist coding for causal mapping

A formalisation of causal mapping

Combining opposites, sentiment

Despite-claims

Coding packages of links

Causal mapping as causal QDA

A simple measure of the goodness of fit of a causal theory to a text corpus

Magnetisation

Animated social map around Donald Trump

Animated social map of US news

Lonely in London

Lonely in London response

Causal mapping of loneliness interviews

Assessing change in (cognitive models of) systems over time



WORKING PAPERS

CHAPTER CONTENTS.

📅 23 Dec 2025

This chapter is a new set of working papers about causal mapping.

PAGES IN THIS CHAPTER

📄 **Minimalist coding for causal mapping**

📄 **A formalisation of causal mapping**

📄 **Combining opposites, sentiment**

📄 **Despite-claims**

📄 **Causal mapping as causal QDA**

📄 **A simple measure of the goodness of fit of a causal theory to a text corpus**

📄 **Magnetisation**

📄 **Animated social map around Donald Trump**

📄 **Animated social map of US news**

📄 **Lonely in London**

📄 **Lonely in London response**

Causal mapping of loneliness interviews

Assessing change in (cognitive models of) systems over time

Minimalist coding for causal mapping

Abstract

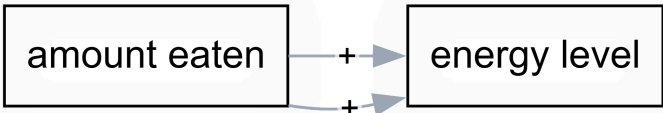
This working paper explains our **Minimalist / Barefoot** approach to coding causal claims in text as simple directed links (“X influenced Y”), developed through extensive experience with large-scale practical coding. We write it now because, although our previous work motivates causal mapping in evaluation (Powell et al. 2025), shows how QuIP-style “stories of change” elicit natively causal narrative evidence (Copestake et al. n.d.), demonstrates ToC validation by comparing empirical maps with programme theory (Powell & Larquemin n.d.), and shows that generative AI can extract links exhaustively with quotes as a low-level assistant (n.d.; Powell et al. 2025), none of these papers is a standalone, reader-facing account of **the coding stance itself**. Our approach is notable in particular because it rejects variable-based approaches used by most causal mapping traditions, which share many assumptions with rules used to build Systems Diagrams, Fuzzy Cognitive Maps, Causal Loop Diagrams, etc. We give an alternative, more primitive account of what exactly counts as a coded causal claim. We explain what we deliberately do *not* encode such as strength and polarity, and we mention the limits of this approach. **Intended audience:** evaluators, academics and qualitative researchers who want a citeable and teachable causal coding protocol, and AI/NLP readers who want an auditable way to identify and process causal content in text.

Introduction: Why minimalist coding?

The variable-based approach

1. I was eating less and felt quite lethargic
 2. I started to eat adequately and was feeling more lively

To the best of our knowledge, all major approaches to causal mapping (Axelrod n.d.; Eden et al. 2023; Laukkanen & Wang n.d.; Maule et al. 2023) would most likely code (1) as something like `amount eaten —> energy level`. And they would treat (2) pretty much the same.



Variable-based coding

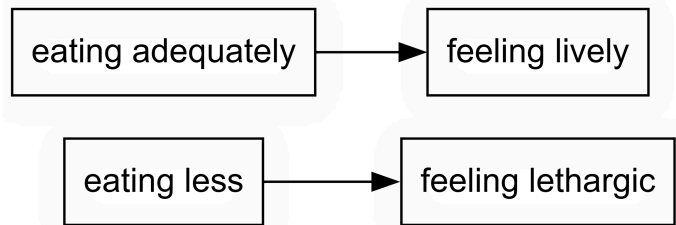
(Nomenclature: we will call all cause and/or effect labels "factor labels", whether in the variable-based or minimalist approaches.)

To make this work, we have to believe there are at least two things called variables which can take different values (higher/lower?) and that causal effects are powered by at some kind of (monotonic?) relationship between them: something like, *the more* I eat, *the more* energy I have (in addition to which we have to code, somewhere, four more actual, factual claims: I did eat X amount, and I did have Y energy levels).

Bizarrely, this also means that (1) and (2) are treated as essentially *the same*. They both affirm a "positive" influence of the first variable on the second: more of X means more of Y.

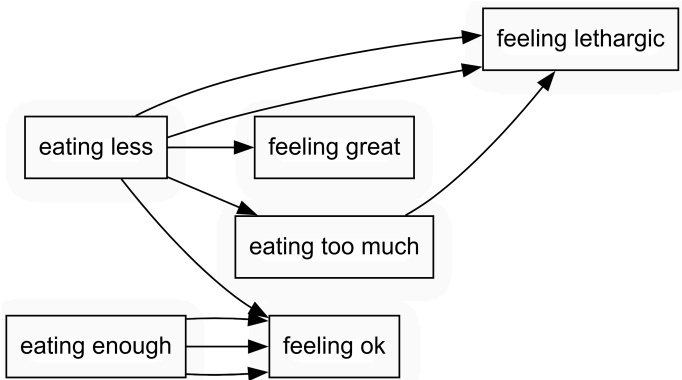
But is that what the speakers *meant*? How do we know if the speakers have say a continuous or Boolean (yes/no) model of "eating"? If Boolean, what is the opposite of eating a lot? Eating only a little? Eating less? Not eating? If continuous, how do we know what kind of function they use in their own internal model?

The variable-based model has to really batter reality into a brutally inadequate shape, and at the same time makes us use formal machinery which we maybe don't need or want.



A propositional, minimalist coding of the same information

Reading multiple texts on the same topic, or different diary entries from the same person, we will probably find much richer narratives which do not fit into those kinds of shapes at all. People eat too little, and then eat too much, and express different ideas of adequacy and normalcy at different times in different ways. Is "feeling lethargic" just low value on the same variable as "feeling great"? Who knows?



An example of propositional, minimalist coding of more texts on the same theme

In other words, to use the variable-based approach, we mostly have to go way beyond what the speaker actually meant; so it is not really a form of qualitative data analysis with the aim of modelling sources' beliefs. It's not just coding, it's modelling, with an ambiguity about whether we are trying to model the world or individuals' putative internal models of it.

We have found that it is often over-specified (and often psychologically implausible) to treat ordinary-language causal claims as if they asserted an explicit functional relationship between well-defined variables. Trying to force that kind of structure on everything turns the “easy 90%” of coding into a harder and more arbitrary task. Of course, one can decide to use a particular non-minimalist, perhaps variable-based representation for a particular modelling purpose; our claim is only that this is usually not a faithful representation of what most speakers actually say or imply, most of the time. For example, if we code “I got really tired because I have Long Covid”, we could perhaps treat both endpoints as Boolean variables, but what about “I got really tired because it was really hot”, and “I got really tired because it was really cold” — how are we going to represent “temperature” as a single variable while preserving the speaker's intended meaning? If what we want to do is model a system, we can pick a solution. But if we want to model *cognition/beliefs/claims as expressed in text*, many “variable semantics” choices are over-committed.

Charles Ragin (n.d.) and others have done admirable thinking on Boolean values and opposites and have constructed sophisticated tools with somewhat simpler ontological baggage (set membership rather than values-of-variables) which perhaps model sentences like (1) and (2) somewhat better. But for many purposes we don't need that baggage either to code ordinary-language claims about causation. Let's see how far we can get with the minimal possible formal baggage.

Most causal mappers have always been quite conscious that modelling people's causal beliefs is not the same as modelling the real world. Sometimes the approach (Axelrod n.d.) is explicitly designed to model someone's thinking. But often it is hard to keep the two worlds separate, what we have called the Janus problem (Powell et al. 2025, p. x). Modelling the real world is exciting, and useful. Traditions like Systems Diagrams, Fuzzy Cognitive Maps, Causal Loop Diagrams and Directed Acyclic Graphs all, to varying degrees, treat the diagram as a model of how the world works rather than a record of what people said. Variables, weights, and (sometimes) functional forms slip in, because the diagram itself is doing the modelling. Our claim is weaker. We are recording what people said about causes, with quotes, and leaving the question of how the world really works to a later step.

Using variable-based approaches allows one to code linear or even non-linear causal influences of single or even multiple causes on their effects. One can do the “coding” by simply writing down (using appropriate special syntax) the connections, because one is an expert, and/or one can verify such statements statistically on the basis of observational data. Thus armed, one can make predictions or have sophisticated arguments about counterfactuals.

The minimalist approach

But using minimalist coding we cannot do that, because our claims are formally weaker and therefore our inference rules are weaker. What we *can* do is still really interesting. We can ask and answer many different kinds of useful questions like:

- what are the main influences on (or effects of) a particular factor, according to the sources?
- what are the upstream, indirect influences on (or effects of) a particular factor, bearing in mind [The transitivity trap](#)?

- how well is a given programme theory validated by the respondents' narratives? (We can do this basically by using embeddings to get measures of semantic similarity between labels and aggregate these as a goodness of fit of theory to data.)

That is all exciting and useful. It's a surprisingly simple way to make a lot of sense out of a lot of texts which is, with caveats, almost completely automatable.

We start by describing the “minimalist” approach to coding causal statements used for QuIP, developed originally by James, Fiona and colleagues at Bath SDR and further formalised at Causal Map Ltd in collaboration with Bath SDR. This formalisation lives inside the [Causal Map app](#). We then set out the coding principles, position the approach within qualitative research, show how it can be extended with useful transformations, and finally ask how it copes with harder constructions like enabling, blocking and causal packages.

The minimalist approach is notable because it is based in our **joint experience of coding thousands and thousands of stakeholder interviews and other data such as project reports**, mostly from international development and related sectors, as well as coding hundreds of thousands of pages with AI-assisted coding. These have nearly always involved **multiple sources talking about at least partially overlapping subject matter**. So this coding produces individual causal maps for each source, which can then be combined in various ways.

Contrast: multiple sources

This is in contrast to methods of constructing single-source maps of expert thinking (Axelrod n.d.) or the collective construction of a consensus map ({Barbrook-Johnson n.d.}).

Contrast: deferred judgement

Our approach is also different in emphasis from the kind of approach adopted by Jewlya Lynn (2026) and other evaluators (it is perhaps implied in Aston (2023) and in Contribution Analysis more broadly). That approach usually **first gathers all the evidence for any influence of X and Y prior to actual mapping**. So before a link from X to Y is actually formally coded it has already been subject to important evaluative judgement: are these testimonies, from these different sources in this context, enough for us to draw a link, and if so with what other attributes (e.g. this link may optionally also be judged to have strength 3 or relevance 2 or whatever).

Whereas our default approach is to code all causal claims willy-nilly: this often means that we code multiple, not necessarily compatible, claims between X and Y, creating sets of *co-terminal links* (sets with X at one end and Y at the other); we call such a set a *bundle*. Our maps show only a single arrow from X to Y but in the database there are actually multiple parallel claims. We defer judgement until later about the quality, compatibility and relevance of these claims and what they actually contribute to answering research or evaluation questions. We often report raw statistics e.g. about the number of claims before making any such judgement. See [Quality assurance at each step of the causal coding workflow](#).

Tasks

We treat causal mapping as three tasks (Powell et al. 2025).

- Task 1, data gathering, not covered here;
- Task 2, coding: creating **evidence-with-provenance** (a links table)
- Task 3, analysis: a sequence of transforms of the original links table: the final interpretation of the analysis can be

understood as a concatenation of the interpretation rules for each transform.

Project context (how this paper fits)

A companion formalisation paper [A formalisation of causal mapping](#) makes key parts of the method more precise (data structures, constraints, and conservative inference rules), but the focus here is the narrative rationale and practical coding guidance.

Unique contribution (what this paper adds):

- A definition of the **minimalist/barefoot** coding stance (“X influenced Y, with provenance”) and what it deliberately does *not* encode.
- An account of why this stance is useful at scale (including AI-assisted extraction) and where it fails (enablers/blockers; conjunctions/packages).
- Examples of additional extensions / transforms, called “filters” in the Causal Map app. Both the initial definition of a links table and each additional filter is described in terms of syntax and of semantics (interpretation rules).

This paper is part of a small set of new working papers

- Companion formal spec: [A formalisation of causal mapping](#)
- QDA positioning: [Causal mapping as causal QDA](#)
- Practical transforms/diagnostics: [Magnetisation](#); [A simple measure of the goodness of fit of a causal theory to a text corpus](#); [Combining opposites, sentiment](#); other transforms to come later

- A worked “AI clerk vs human architect” example: [Assessing change in \(cognitive models of\) systems over time](#)

From the four prior papers already cited in the abstract, we adopt one stance in particular (Powell et al. 2025): causal mapping is primarily a way to assemble and organise **evidence-with-provenance**, keeping the subsequent evaluative judgement about “what is really happening” distinct. A coded link is first and foremost “there is evidence that a source claims X influenced Y”, not a system model with weights or effect sizes. The present paper extracts and clarifies the core “minimalist” coding commitments that make that workflow, including its AI-assisted form (n.d.; Powell et al. 2025), workable and checkable.

Our experience has been that the vast majority of causal claims in these kinds of texts are easily and satisfactorily coded in the simplest possible form “X causally influenced Y”. Explicit invocation of concepts like enabling/blocking, or necessary and/or sufficient conditions, or linear or even non-linear functions, or packages of causes, or even the strength of a link, are relatively rare. The causes and effects are not conceived of as variables, the causal link is undifferentiated, without even polarity, and if any counterfactual is implied it remains very unclear.

This approach is what we call **“Minimalist” or “Barefoot” Coding**.

Minimalist coding principles

The 90% rule

We have found that it is pretty easy to agree how to apply minimalist coding to say 90% of explicit causal claims in texts, without missing out essential causal information, whereas it is very difficult to find appropriate frameworks to cope with the remaining 10%.

Fewest assumptions

Minimalist coding is perhaps the most primitive possible form of causal coding which makes no assumptions about the ontological form of the causal factors involved (the “causes” and “effects”) or about *how* causes influence effects. In particular we do not have to decide if cause and/or effect is perhaps Boolean or ordinal, or if perhaps multiple causes belong in some kind of package or if there is some kind of specific functional relationship between causes and effects.

An act of causal coding is simply adding a link to a database or list of links: a link consists of **one new or reused cause label and one new or reused effect label**, together with the highlighted quote and the ID of the source.

Take the second diary entry from the start of this paper:

I started to eat adequately and was feeling more lively

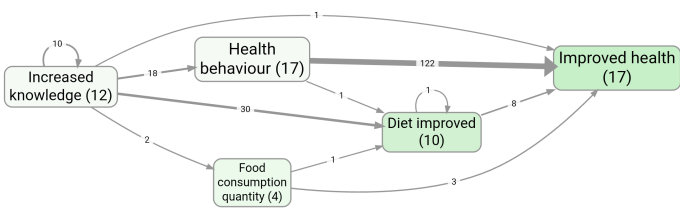
It can be trivially coded minimalist-style as

I started to eat adequately → feeling more lively (Source ID: diary entry 2; Quote: I started to eat adequately and was feeling more lively)

That's it.

Causal maps

Crucially, we can then display the coded claims for individuals as a graphical causal map, and we can also display the entire map for all individuals and/or maps filtered in different ways to answer different questions. There is a handful of other applications (Ackermann et al. n.d.; Laukkanen 2024) for causal mapping which also do this; but as far as we know, only Causal Map also allows direct QDA-style causal coding of texts.



Bookmark #266 — a typical first-pass map: top five factors by frequency, with no further encoding of polarity, weight, or function. The numbers are evidence counts (citations on links, source counts on factors), not effect sizes.

Data structure

Although we have the option of creating additional tags associated with each link (where many approaches would for example code the polarity of a link) this is not central to our approach.

We don't use a separate native table for factor labels: they are simply derived on the fly from whatever labels happen to be used in the current table of links. This makes data processing simpler and also suggests an ontological stance: causal factors only exist in virtue of being part of causal claims.

We do however have an additional table for source metadata including the IDs of sources, which can be joined to the links table in order, for example, to be able to say "show me all the claims made by women".

Causal powers

When a source uses causal language, we treat it as a claim that one thing made a difference to another (rather than a mere temporal sequence) *in virtue of its causal power to do so*.

Causal influence, not determination

We believe that it's rare for people to make claims about causal determination: someone can say that eating properly again made them feel more lively, and then also agree that some good sleep had a lot to do with it too, without this feeling like a contradiction.

Not even polarity

We differ even from most other approaches which are explicitly called "causal mapping" in that we do not even "out of the box" record polarity of links (to do so would involve making assumptions about the nature of the "variables" at each end of the link as well the function from one to the other).

The Focus on Cognition

In the minimalist approach, **we are quite clear that what we are trying to code is the speaker's surface cognitions and causal thinking**, while the actual reality of the things themselves is simply bracketed off at this stage, either to be revisited later (because we are indeed interested in the facts beyond the claims) or not (because we are anyway interested in the cognitions).

Staying on the surface

At Causal Map, we rarely make any effort to get beneath the surface, to try to infer hidden or implicit meanings. This is particularly well-suited to coding at scale and/or with AI. Our colleagues at Bath SDR do this a bit differently, spending more effort to read across an entire source to work out what the source *really* meant to say.

Closer to the cognitive truth

People rarely talk about causes as variables. In our experience they say "X made Y happen", or "Y was because of X", and that is most of what they say. They almost never commit to whether X is continuous or Boolean, or to whether the relationship is linear, threshold, or anything else. If we are trying to record what they said, we are better off not adding any machinery they did not specify. That is what we mean by staying close to the cognitive surface: the labels carry whatever the speaker was carrying, and we add nothing else.

Unclear counterfactuals

A causal claim in ordinary speech rarely carries a clean counterfactual. "I started to eat adequately and was feeling more lively" doesn't tell us whether the speaker thinks they would have felt only slightly more lively without eating properly, or would have stayed lethargic, or whether the comparison wouldn't apply because their circumstances would have been different anyway. Variable-based coding has to commit to one of these answers, even if only by default. Minimalist coding doesn't. A coded link says "X influenced Y", and the counterfactual stays where the speaker left it.

General versus specific

Minimalist coding focuses primarily on **factual causal claims** which also warrant the inference that both X and Y actually happened / were the case.

Most causal claims in the kinds of texts we have dealt with (interviews and published or internal reports in international development and some other sectors) are factual, about the present or past. Sometimes we see general claims, and we often just code these willy-nilly. In any case, the distinction between general claims and claims about specific events that actually happen is often fractal (a general claim can seem specific in a broader context) and difficult to maintain completely when modelling ordinary language.

We don't code absences

Minimalist coding may be reasonably also called **Qualitative Causal Coding** or **Causal QDA coding**. It shares characteristics with some forms of coding within Qualitative Data Analysis (QDA), in particular demonstrating an asymmetry between presence and absence (a specific tag not being applied is not the same as the application of an "oppositely-poled" tag).

We do not code absences unless they are specified within the text (e.g. perhaps "because of the barking dog, the owner did not come out of the house").

While codes may be counted, the concept of a *proportion* of codes is challenging because the denominator is often unclear.

If families are talking about reasons for family disputes, and family F mentions social media use, and family G mentions homework, we do not usually interpret this as meaning that family F does *not* think that homework can also be a cause of family disputes. (This should derive formally from the interpretation of multiple causal claims).

The labels do all the work

At Causal Map Ltd, our canonical methodology initially involves in vivo coding, using the actual words in the text as factor labels. This initial process generates hundreds of overlapping factor labels. This part is really easy (and is easy to automate with AI). Obviously, hundreds (or hundreds of thousands) of overlapping factor labels are not very useful, so we need to somehow consolidate them. Arguably, minimalist coding makes the initial coding easy but it just defers some of the challenges to the recoding phase.

One way to code additional meaning within factor labels is to add **tags as literal text inside the label**.

Tags can be used to group factors into **themes** which are not themselves causal factors (e.g. health, environment), without making them part of a hierarchy. They can add **lightweight metadata** for later filtering (e.g. (Outcome), #nutrition, [Work]), while keeping the underlying causal claim as just $X \rightarrow Y$.

Suitable filters can then be provided to:

- **Filter by theme/flag:** because tags are just text, any “match start/anywhere/exact” label filter can include or exclude factors containing #health, (Outcome), etc.
- **Hide tags for display:** to strip tags out for a cleaner map/table view, while retaining them in the underlying data.

Evidence strength is not causal effect size

Counts (how many times a link is coded; how many sources mention it) measure *evidence volume/breadth in the corpus*, not causal magnitude in the world. This matters because the outputs can look like a quantitative system model even when they are not one.

In the app we routinely distinguish:

The transitivity trap and “thread tracing”

It is usually invalid to infer a long causal chain by stitching together links from different sources. A conservative rule is: only treat an indirect pathway $A \rightarrow B \rightarrow C$ as supported when the *same source* provides each step (“thread tracing”), unless contexts are explicitly aligned.

The canonical failure mode is: source 1 says $A \rightarrow B$, source 2 says $B \rightarrow C$, and we mistakenly conclude that anyone told a coherent story $A \rightarrow B \rightarrow C$. In practice, for a given query we loop through sources one at a time, construct the valid paths *within each source*, and then combine only the edges that appear in valid within-source paths.

Tagging factor labels is different from adding link metadata because it literally creates different causal factors. So **Improved health** and **Improved health [outcome]** are formally two different causal factors. However if we apply a filter to strip the [outcome] tag, they then become the same causal factor.

For a fuller discussion (including examples like #nutrition and (Outcome) and why uniqueness matters), see: [Factor label tags — coding factor metadata within its label](#).

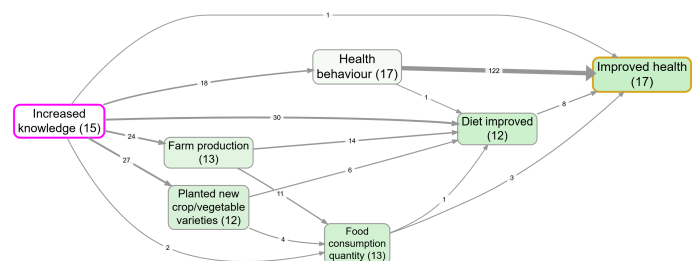
Coping with many labels

We can:

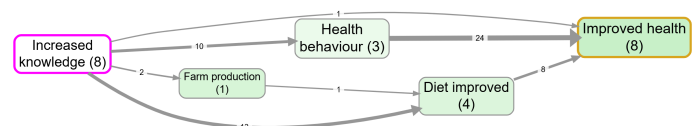
- Use human- or AI-powered clustering techniques to consolidate the codes according to some theory or iteratively according to some developing theory
- Use AI-powered clustering techniques to consolidate the codes according to automated, numerical encoding of their meanings
- “Hard-recode” the entire dataset using a newly agreed codebook (see above)
- “Soft-recode” the dataset on the fly using embeddings to recode raw labels into those codebook labels to which they are most similar

- **Citation count:** how many coded mentions support a link (volume).
- **Source count:** how many distinct sources mention it (breadth / rough consensus).

These are useful for prioritising what to look at, or for filtering (e.g. keeping only links with `min_source_count = 2`), but they do not tell you whether a causal influence is “stronger” in any effect-size sense.



Bookmark #1129 — path tracing from Increased knowledge to Improved health: every link on a route between them, across all sources. Easy to misread as a story anyone told.



Bookmark #981 — same trace with source tracing on. Only routes told within a single source survive: a much smaller, more defensible set of edges.

The filter pipeline as a mental model for analysis

Most analyses are built by applying a sequence of simple operations (for example: subset links to retain only specific sources; trace paths; rewrite labels; bundle duplicates; then show a map/table). Thinking in terms of a pipeline helps make the meaning of an output explicit: it is always “the result of these filters, in this order”.

Concretely, you can think of an analysis as “a links table passed through transforms”, where `|>` is a “pipe” symbol which just passes a result from the left to a new transform on the right.

e.g.:

```
Links |> filter_sources(...) |> trace_paths(...) |>
transform_labels(zoom) |> combine_opposites |> bundle_links |>
map_view
```

This is also why “the same dataset” can yield very different-looking maps without any contradiction: you are simply looking at different derived views defined by different pipelines.

So one can think of analysis as a sequence of transforms of the original links table. The final interpretation of the analysis can be understood as a concatenation of the interpretation rules for each transform.

This pipeline also exists outside the app as a standalone open-source package, ([n.d., /filter-engine](https://www.npmjs.com/package/(n.d.)/filter-engine)) ([https://www.npmjs.com/package/\(n.d.\)/filter-engine](https://www.npmjs.com/package/(n.d.)/filter-engine)), which takes a links table in, applies an ordered sequence of these filters (label and tag filters, path tracing, frequency thresholds, combine-opposites, zoom and more), and returns a links table out.

Positioning within qualitative research (and likely critiques)

This paper is about a **coding stance** and a corresponding **intermediate representation** (a links table with provenance), not a claim that “coding = analysis”. In standard QDA terms, minimalist causal coding is best understood as a disciplined way to build an auditable evidence base that can later be interpreted, queried, and written up (Miles et al. n.d.; Saldaña n.d.). It is, of course, not the only kind of way to do QDA.

To reduce avoidable points of attack from mainstream qualitative social science readers, we make four explicit commitments up front:

- 1. We code claims, not causal truth.** A coded link is evidence that a source *claimed* an influence relation. It is not (by itself) a causal inference claim about the world. This keeps the method compatible with both realist and constructivist sensibilities (n.d.; Charmaz n.d.).
- 2. We do not treat “counts” as effect sizes.** Counting supports prioritisation and transparency, but it is not a substitute for interpretation; frequency/breadth in a corpus is not magnitude in the world. (See “Evidence strength is not causal effect size” above.)
- 3. We trade interpretive depth for audibility and scale, on purpose.** We stay close to surface causal language and preserve provenance (quotes + source ids) so that readers can check what is being claimed. This is a pragmatic stance when working with many sources and/or AI assistance; it does not deny that richer interpretive work can be valuable.
- 4. Minimalist coding is not a full QDA workflow.** It is one layer that can sit alongside thematic analysis or qualitative content analysis when those are needed (Braun & Clarke 2026; Mayring 2026).

How a qualitative-methods reviewer might criticise this (and our response)

Below are common criticisms (often reasonable) and the corresponding guardrails/defences built into the minimalist stance.

Critique 1: “This is reductionist / strips context / decontextualises quotes”

Response: We keep provenance as first-class data (source id + quote) and treat every map/table view as a derived view of that evidence. Context is handled explicitly by joining source metadata (e.g. subgroup, time) and filtering the links table; it is not “assumed away”. When deeper within-case interpretation matters, the same links table can be re-read within full-text context, and the write-up remains responsible for integrating that context (Miles et al. n.d.).

Critique 2: “You are smuggling in a positivist epistemology (or a realist metaphysics)”

Response: The method’s core object is **reported causal thinking** (surface causal claims), not an ontic causal model. We therefore keep the epistemic claim modest: “this is what sources said, with quotes”, and we separate that from subsequent judgement about what is happening. That separation is consistent with standard qualitative criteria emphasising transparency, audit trails, and reflexive interpretation (n.d.; Charmaz n.d.).

Critique 3: “Coding is not analysis; links don’t ‘explain’ anything”

Response: Agreed. Minimalist causal coding produces a structured evidence base that supports multiple analyses (filters, comparisons, pathway queries) and supports more disciplined writing, but it does not replace interpretation. In practice, it can reduce time spent on low-level organisation of text so that more time can go into interpretation and argument (Saldaña n.d.; Glaser 2026).

Critique 4: “You are privileging causal language and missing other kinds of meaning”

Response: Yes: by design. Minimalist coding is for projects where the core questions are themselves causal (mechanisms, pathways, theories of change). When non-causal meaning is central (identity work, norms, metaphors), other QDA approaches could lead; minimalist coding can still be a useful *additional* layer rather than the whole analysis ((Braun & Clarke 2026)).

Critique 5: “Automatable coding invites false precision and overclaiming”

Response: We explicitly constrain what automation is allowed to do: extract candidate links with quotes; keep everything auditable; and avoid treating the resulting network as a system model with polarity/weights “by default”. The transparency of provenance is the main defence against black-box “AI did the analysis” claims.

Extensions

There are many extensions one can add on top of minimalist coding. Small extensions can just add convenient functionality; other extensions can upgrade the system to other more fully-fledged kinds of coding like Fuzzy Cognitive Mapping.

An extension in general consists of syntax rules for how to construct and carry out a transformation, and semantic or interpretation rules for what this transformation means.

Some useful extensions

Hierarchical coding and “zooming” (FIL-ZOOM)

Motivation

In any qualitative coding process there is a tension between **detail** (keeping what respondents actually said) and **summarisation** (making patterns visible). Hierarchical factor labels give us a simple way to keep both:

- We can keep a detailed factor such as **Healthy behaviour; hand washing**.
- We can also treat it as an instance of a higher-level causal factor **Healthy behaviour** when we want a higher-level view.

This is particularly useful in causal mapping because it lets us simplify a complex map *without changing the underlying link evidence*: we are simply rewriting labels into more general ones.

Coding conventions

We use the separator **;** to create nested factor labels, using a template like:

General concept; specific concept

For example:

New intervention; midwife training -> Healthy behaviour; hand washing

This is an example of what we called earlier “letting the labels do the work”. As we do not in any case have a native factors table where we might record actual relationships between say lower-level and higher-level factors, we can do the same thing implicitly simply with the text of the label.

In AI-assisted coding, a practical way to encourage hierarchical labels is to explicitly instruct the coder (human or AI) to label each factor using the template **“general concept; specific concept”**, and to always provide a verbatim quote for each coded claim so every link remains checkable.

For example, an AI might produce a single coded link such as:

- **Improved agricultural practices; diversified crops -> Increased income generation; more sales**

This paper treats in full only one extension: **hierarchical coding**, because it is simple, widely useful in practice, and it directly supports a transparent family of “zoom” transforms. The other extensions appear only as concise, standalone stubs; fuller treatments are published as separate work-in-progress papers.

- Quote: “with a lot of good product, we are now able to sell more.”

You can read the separator as:

- “A, and in particular B” (forward), or
- “B, which is an example of / part of A” (reverse).

This can be extended to multiple levels:

Improved hospital skills training; new midwife training; hand washing instructions improved

Two practical conveniences follow immediately:

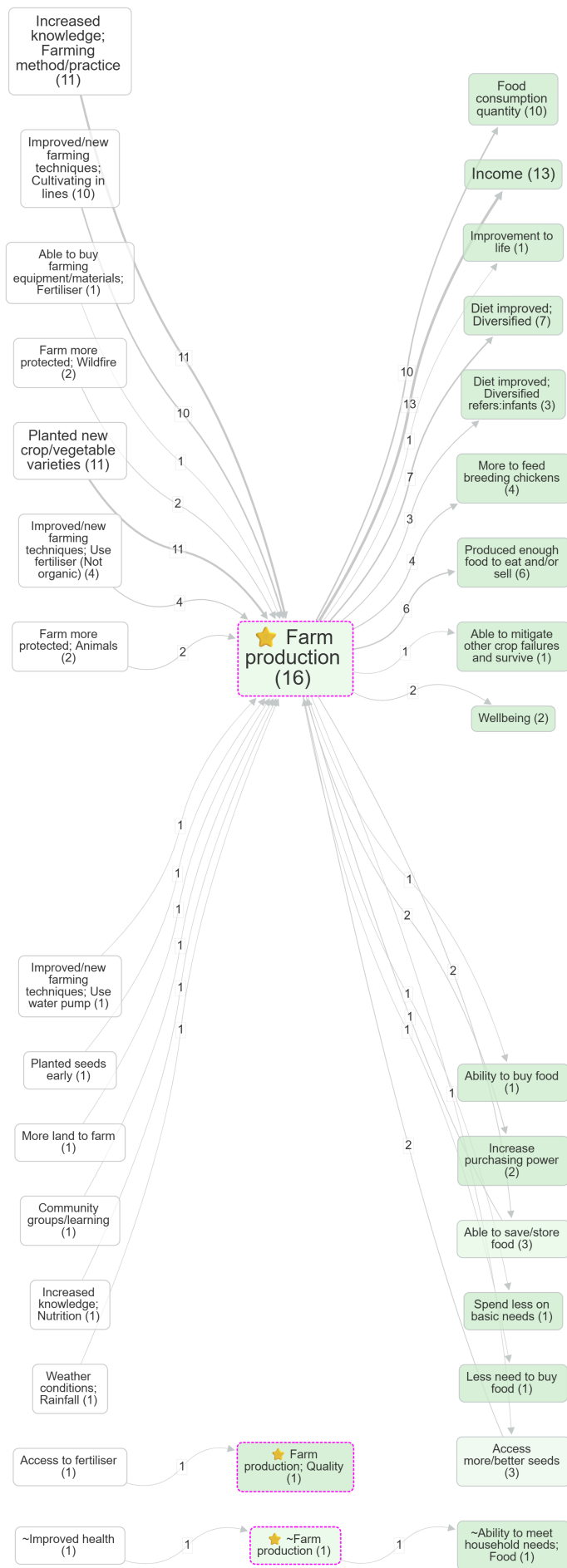
- Searching for the higher-level label (e.g. “Healthy behaviour”) will also find its nested sublabels.
- Higher-level labels can be created and changed on the fly (they are just the visible prefixes before **;**), without maintaining a separate “parent code” structure.

Zooming and visualisation

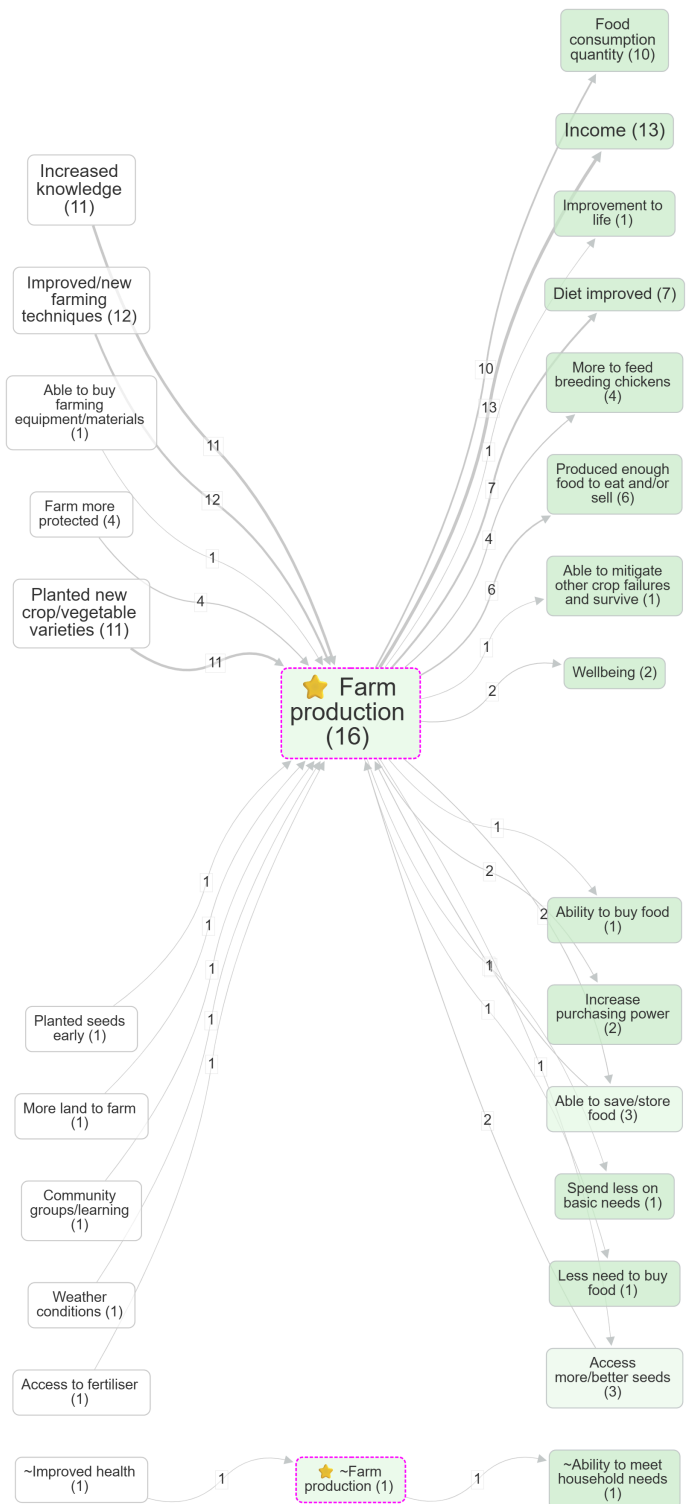
The basic idea of a zoom view is: **rewrite nested labels to a chosen level of abstraction**, then draw the map using the rewritten labels.

At “zoom level 1” we simply truncate at the first **;**:

- **Healthy behaviour; hand washing** becomes **Healthy behaviour**.



Bookmark #805 — Focus filter on *farm production* with no zooming: detailed labels remain.



Bookmark #806 — same focus, zoomed to level 1: nested labels collapse to their top-level parents, the map gets more readable, and no underlying coding has changed.

So if we have (at the detailed level):

Improved hospital skills training; improved midwife training; hand washing instructions improved

then we can show higher-level summary links such as:

- Improved hospital skills training; improved midwife training
- and (at the coarsest level) Improved hospital skills training

Caveats (don't use the ; separator mechanically)

Intuitively: by writing **A; B** you are explicitly licensing the interpretation “for high-level summary purposes, treat this as A”.

You can also see this as a family of conservative “deductions” licensed by the ; separator. We are explicitly allowing ourselves (for summary purposes) to treat the more detailed version also as evidence for the two less detailed versions: **Improved hospital skills training; improved midwife training; hand washing instructions improved** is also evidence for **Improved hospital skills training**.

The ; separator encodes a commitment: that the more specific factor can be “rolled up” into the more general one without essentially distorting the causal story.

Other tips

Try not to mix desirability within one hierarchy (e.g. avoid **Stakeholder capacity; lack of skills**, because zooming out would treat it as evidence for “Stakeholder capacity”). Use opposites coding (~...) for this kind of case.

A factor cannot belong to **two different** hierarchies at once (because there is no single parent to roll up into).

When not to use a hierarchy (use a tag instead)

Don't use ; to express a non-causal *topic theme*. **A; B** explicitly licenses: “for high-level summary purposes, treat this as A”, so **A** must be a real (semi-quantitative) causal factor, not just a theme label like “Health”.

If something is merely health-related (with no natural roll-up into a single causal factor), use a tag instead:

- **Vaccinations law is passed** [Health]
- **Mortality rate** [Health]

Only use a hierarchy when the right-to-left label is genuinely a refinement of the left-to-right one (so roll-up makes sense), e.g.:

- **Improved health training; Improved hospital skills training**

Other extensions (stubs; treated in subsequent articles)

The formal treatment of these is in [A formalisation of causal mapping](#). Each stub below points to its own work-in-progress paper where one exists.

Analysis filters (FIL-CTX, FIL-FREQUENCY, FIL-BUNDLE)

Three extensions are ordinary analysis filters in the pipeline sense above, and need no new coding conventions: restricting the evidence base by source metadata, for example only women or only a time period (FIL-CTX); retaining only links meeting an evidence threshold such as **min_source_count=2** (FIL-FREQUENCY); and computing evidence-strength columns such as **Citation_Count** and **Source_Count** for each **Cause -> Effect** pair, for use in map and table views (FIL-BUNDLE).

Opposites coding (FIL-OPP) (stub)

Rather than encoding signed/weighted edges, we can get similar benefits by treating explicit opposites in labels (e.g. **~Employment** vs **Employment**) as a label-level device with simple inference and visualisation rules.

See [Combining opposites, sentiment](#)

Despite coding (FIL-DESPITE) (stub)

Narratives often name an influence that was present and relevant but failed: “the river still rose, despite the flood-prevention work”. Coding an ordinary link asserts the opposite of what the speaker meant; coding nothing throws the information away. So we mark such links with a special type, **despite**. A despite link **Z -despite-> E** is read as: the source claims Z happened, E happened (or, in the helping flavour, failed to happen), and Z had some causal power to work against (or for) E, but did not prevail. A despite link is never evidence for an ordinary **Z -> E**. Despite links can be visualised distinctly, filtered, and counted separately or together with ordinary links.

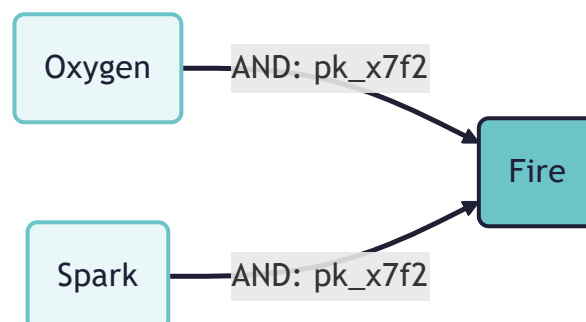


See [Despite-claims](#)

Coding packages (FIL-PACKAGE) (stub)

Sometimes a source presents several influences as working *together*: “you need both oxygen and a spark to make fire”. Two separate links drop the conjunction; one combined label (**oxygen and a spark**) is no longer parsable. The package extension keeps the individual links and ties them together with two extra pieces of link metadata: a shared arbitrary **package** code, unique to that combination, and a **package_type** naming the connective, usually **AND** or **OR**, with **?** where the speaker grouped the causes in one breath without saying which. (**OR** is looser: alternative sufficient causes arguably need no package at all, so an **OR** package records only that the speaker offered the influences together as alternatives rather than asserting each outright; **?** is the honest default when even that much is unclear.) Despite claims join in naturally: when a narrative names both the failed influence and the influence that overcame it (“the fan failed to cool him down because the sun was too strong”), the despite link and the ordinary link share a package code with type **DESPITE**. A package can also group the effects of one cause (“the programme raised awareness and changed behaviour”): this matters less for analysis, since effect links read separately rarely overstate the claim, but it is often the more natural reading, and it keeps the convention symmetrical: a package groups the links of one joint claim, whichever end the conjunction is on.

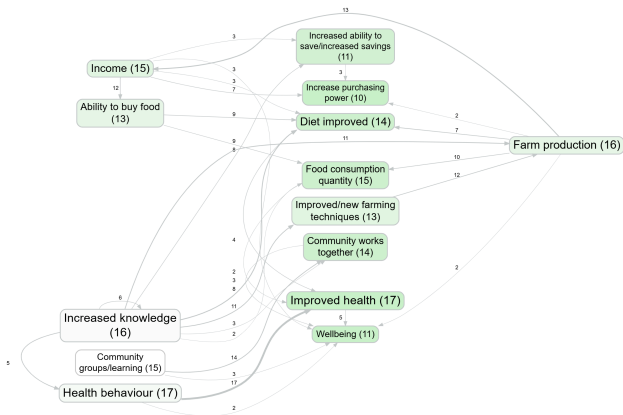
The semantics are weak by design: a package records that the speaker presented these influences as working together in the named manner, with no commitment to truth tables, necessity or sufficiency, interaction effects, or completeness. Packaged links remain ordinary links for every other purpose; the code makes the combination filterable (show all packaged links, one package, or one type) and countable without inflating conjunctive claims into independent evidence.



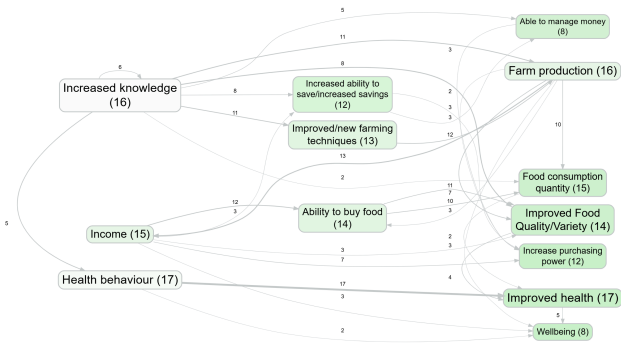
AI extensions (optional; stubs)

Soft recoding (magnets) (FIL-SOFT)

Many raw in-vivo labels can be aggregated by mapping them to a smaller vocabulary of “magnets” using semantic similarity. This changes labels (a recoding step), not the underlying minimalist meaning of a single coded link.



Bookmark #493 — soft recode using all top-level labels from a human-coded version of the same project as magnets.



Bookmark #494 — same setup but recycling weaker matches. The change in settings shifts which raw labels attach to which magnets.

Auto recoding (clustering) (FIL-AUTO)

Alternatively, labels can be clustered into emergent groups and then rewritten accordingly, again as a recoding/aggregation step rather than a new causal logic.

How other schools of causal mapping extend minimalist links

The “minimalist” stance in this paper is deliberately radical: we code *undifferentiated* causal influence claims with provenance, and then do most interpretation work via filters and views. Many other causal mapping traditions extend the representation in the opposite direction: they add more **semantic structure to factors and links**, especially **polarity** (and sometimes strength/weights), often with an implicit assumption that factors behave like **variables** and that a link expresses some kind of monotonic relationship. For a broader overview of these traditions and their differences, see (Powell et al. 2025).

What these extensions usually add

- **Polarity (+/–)**: a link can mean “more of X leads to more of Y” (positive) or “more of X leads to less of Y” (negative).
- **Strength / weights**: a link can be assigned a magnitude (sometimes elicited, sometimes assigned by analysts), which invites quantitative reasoning and simulation-like interpretations.
- **Variable semantics and counterfactual clarity**: polarity and weights usually presuppose that the endpoints are comparable as variables (or at least as ordered states), so that “more/less” (or “increase/decrease”) is meaningful.

These extensions can be extremely useful in settings where respondents are explicitly reasoning in those terms, or where the purpose is modelling/simulation. But they also raise the bar: if we write down a signed or weighted link, we are committing not just to “X influenced Y”, but to a stronger claim about *how* X and Y vary and how changes propagate.

Three traditions, briefly

In the Axelrod tradition, cognitive maps represent beliefs about causal influence among concepts and are usually coded with **positive/negative influence** as well as direction (Axelrod n.d.; Axelrod n.d.); this makes reinforcing and balancing structure visible, but moves the representation closer to variables-with-values even where the original elicitation was not precise about scales or functional form. In the Eden/Ackermann “cause maps” tradition, maps are built interactively to structure messy problems and support decisions (Eden et al. 2023; Ackermann et al. n.d.; Ackermann et al. n.d.); polarity is more natural there because the map is negotiated in a workshop, where the group can settle what “increase/decrease” means. Comparative causal mapping elicits, standardises and compares many maps across people, groups or time, which again pushes towards a more “variable-like” reading of factors so that maps can be aligned at scale (Laukkanen n.d.; Laukkanen 2024; Markicz & Goldberg n.d.; Hodgkinson et al. n.d.).

What it would take to extend our logic to signed links (and why we treat it separately)

We *can* extend our links-table logic to incorporate polarity, but doing so is not just “adding a column”; it changes the semantics.

At minimum, we would need:

- A **Polarity** field per coded link (e.g. +, -, *unknown*) that is grounded in the source text or elicitation.
- A rule for how polarity is inferred when it is implicit, ambiguous, or mixed (which is common in narrative material).
- A clear semantics for what + and - *mean* (typically: a monotonic relationship between variables), including what counts as “more/less” for a factor label.

- Aggregation rules for how to deal with disagreement and contradiction across sources, and for how to visualise those disagreements without creating spurious precision.

Because our target corpora typically do *not* make those commitments explicit (and because we are focused on modelling evidence-with-provenance rather than system dynamics), we do not treat signed links as part of the core method here. Where polarity matters in practice, we prefer to handle it with **label-level devices** (e.g. `~Employment` vs `Employment` combined with the `combine_opposites` transformation).

Context and mechanism

Evaluators reading this will recognise two questions that any causal method has to answer. How do you code context? And what do you do about mechanisms? Both come up in realist evaluation and its Context-Mechanism-Outcome formula (Pawson & Tilley 2013). Minimalist coding answers both the same way. It gives them no special machinery and treats them as ordinary parts of the causal map. This is primitive and it flattens distinctions which can be important, but it is *relatively* easy to apply, and the results are not obviously worse than any more sophisticated approach.

Coding context

Context is the standard worry. The same intervention has different effects in different places, and a good evaluation should say so. The realist instinct is to treat context as a separate kind of thing, a backdrop or a set of enabling conditions that decides whether a mechanism fires, distinct from the causes themselves.

The minimalist move is to code context as one more causal factor. If a source says the training worked because the community already trusted the facilitators, then `community already trusted the facilitators` is a factor, and it influenced the outcome, coded as an ordinary link. Nothing new is needed.

This is primitive. It drops the special status that context is often given, and it does not by itself mark the difference between a cause that triggers an effect and a condition that merely lets some other cause work. Where that difference really matters, it is the same second-order enabler problem we discuss below, and we handle it the same way, usually in the write-up.

Coding context as a factor is also not the only option. You can record it as link metadata (a column in the table of links, or in particular as a dedicated link tag like `#afterCovid` or `#onlyInSevereCases`) or as a factor tag (`Trust [afterCovid]`). We use each of these where it fits.

The idea of 'Context' also overlaps with the idea of 'Source'. *True in this context* is not so far from *True for this source*, and in our minimalist approach we have some surprisingly sophisticated ideas about dealing with what *True for this source* means when tracing causal influences upstream and downstream, see [here](#). In other words, to the extent that *Source* is for you a reasonable proxy for *Context*, then you can use the fact that all your source metadata is already available to you in your links dataset, for example you can add a filter to show only causal stories which are *true for these particular sources*.

There is no universally accepted way to encode context because there is no even half-way accepted operational agreement about how to distinguish triggers, contexts and mechanisms. Realist evaluation has argued for decades about what counts as context as opposed to mechanism or outcome. Yes, these concepts can be usefully applied in a given case or study. But there are different ways to apply them. That is a strength as well as a problem. But the problem remains that you can't just give a set of instructions on how to encode them and be sure that they will be applied in the same way by different assistant coders (human or AI). Given that, coding context as a plain factor is a reasonable default. It is simple, and it defers to the analysis phase the harder judgement about what role context is playing.

Coding mechanisms

The CMO formula raises a sharper question that the realist literature has never really settled, even after a dedicated review of how realist evaluations use the term (Lemire et al. n.d.): what is a mechanism? Definitions range over hidden generative processes, the reasoning and resources of actors, underlying social structures, and several other things, and a coder who has to decide which one applies before coding anything will not get far.

Minimalist coding avoids the definition. A mechanism is just a piece of the causal map. If context factors E and F lead to outcomes X and Y, possibly through intermediate factors M and N, then the mechanism is that part of the graph: the links from E and F through M and N to X and Y. There is nothing extra to encode and no separate object called a mechanism. You read it off the map by tracing paths, with the same caution about combining links across sources discussed under source tracing.

This too is primitive and flattening. It does not capture the generative, what-is-really-going-on-underneath sense that realists want from the word mechanism, and it will not satisfy a reader who thinks a mechanism must be more than the connections people described. But it works. It gives you a concrete, checkable object made of coded claims with quotes, where the realist account gives you a long-running argument about definitions. If a richer notion of mechanism is needed for a particular study, it can be built on top, the same way polarity or packages can. The minimalist record does not block that, and it is a usable starting point in the meantime.

Causal packages / conjunctions

This overlaps with the problem of how to code causal packages: claims in which some *combination* of factors is said to be needed for an effect. It's part of realist thinking, but it appears in other approaches too.

Our answer is the weak-by-design package extension (FIL-PACKAGE, above): the member links stay ordinary links, a shared code records the claimed combination, and the stronger model-like commitments (truth tables, interaction effects, necessity and sufficiency) are exactly what the extension refuses to make. See [Coding packages of links](#).

Where minimalist coding runs out

Minimalist coding handles maybe 90% of explicit causal claims in the kinds of texts we work with. As discussed above, it may not satisfy researchers who have a specific meta-theory they want to apply, like CMOs, or INUS packages.

The remaining 10% includes a few recognisable types: claims that are really about other claims (enablers and blockers), claims that depend on a combination of factors (causal packages, treated above), and claims that pack a sophisticated causal story into a single phrase ("went viral").

We've thought about all three. Beyond the two weak-by-design markers already described (despite links and package codes), none of them has led us to add new structure to the coding scheme. Here is why.

Causation about causation: enablers and blockers as second-order causal claims

One natural way to try to go beyond truly minimalist coding is to say that some claims are not about simple factors causing other simple factors, but about one factor influencing (enabling or blocking) a *causal connection* between two other factors.

Consider an enabler-style statement:

I went on holiday to Spain expecting to enjoy it, and indeed I did particularly enjoy it because I remembered to take my phrase book.

The phrase book enabled the causal link from X to Y, in virtue of its causal power to do so.

Blockers are closely related but show the asymmetry more starkly:

I went on holiday to Spain, but I forgot to take my phrase book so I didn't enjoy the holiday at all.

In strictly minimalist coding, we might record two undifferentiated links:

- Going on holiday to Spain → Enjoying the holiday
- Remembering to take my phrase book → Enjoying the holiday

One can code:

- Forgetting to take my phrase book → Not enjoying the holiday

But arguably the second claim is not really “the phrase book caused enjoyment” in the same way as the first. Rather, it enabled the normal causal power of the holiday to produce enjoyment. A more explicit encoding would therefore be:

1. Going on holiday to Spain → Enjoying the holiday
2. Remembering to take my phrase book → (Going on holiday to Spain → Enjoying the holiday)

However, our intuition that “going on holiday” should also be part of the story is hard to capture without moving to a second-order encoding. The blocker case is visually similar to the enabler case, but it seems to require adding the idea of a causal power to *stop* something: the blocker destroys or disables the causal power of X to bring about Y. In practice we handle the related blocked-outcome case with despite coding (FIL-DESPITE, above), which records a countervailing influence that was present and failed, without committing to a signed edge.

Visually, (2) would be drawn as an arrow pointing to the middle of the arrow in (1). Semantically, it can be read as:

We do not claim we can do much with this at scale; but it is a clear way of stating what seems to be “missing” from purely first-order, undifferentiated links in these edge cases.

Sophisticated claims as ordinary language

A more cheerful note. Some sophisticated causal constructions become part of ordinary language as a single phrase. "Her post mocking Farage went viral, so Farage was forced to respond" packs a fairly elaborate causal story into the verb "went viral", and a minimalist coder can take it as one factor without trying to unpack what "viral" really means. The same trick lets us code "she got cancelled", "the policy backfired", or "the deal fell through" as ordinary minimalist factors, deferring the unpacking to wherever it actually matters. The benefit is real: we don't need a new coding apparatus every time the world invents a new causal idiom. We just need labels that hold their meaning long enough to be useful. Or: reality is fractal.

What this section adds up to

We could in principle extend minimalist coding to handle enablers, blockers, and packages with rich new structure. We have mostly not. The cost of a richer scheme almost never pays back, because the relevant cases are rare and the analyst can usually handle them adequately in the write-up, where they belong. We would rather log a slightly imperfect minimalist record and write nuance into the prose than build it into the data structure. The two exceptions we allow ourselves are weak-by-design markers on otherwise ordinary links: the despite type (FIL-DESPITE) and package codes (FIL-PACKAGE), each described above, and each refusing the stronger commitments that make richer schemes expensive.

See also our related work: the evaluator-facing overview (Powell et al. 2025), the AI-assisted coding papers (n.d.; Powell et al. 2025), the application to outcome harvesting (Britt et al. 2025), and the handbook chapter on qualitative causal mapping in evaluations (Remnant et al. 2025).

Conclusion

Minimalist coding is proud of being unsophisticated. We code "X influenced Y, with provenance", and we let the labels do most of the work. We don't code polarity by default, we don't fit functions, and we don't worry about counterfactuals the speaker didn't supply. We give up the predictive and simulation-style affordances of more elaborate coding traditions in exchange for something more practical: an auditable evidence base that scales to thousands of sources, works well and quickly with AI assistance, and stays close to what people actually said.

We've used this stance across consultancy and research work for years now, and it keeps repaying its simplicity. The hard cases come up; we usually handle them in the write-up rather than in the data structure. The 90% rule is a working hypothesis: the coding scheme should serve the typical claim, and the analyst should not fail to remark on the unusual ones.

As a closing shot, we could say "Beyond minimalist causal coding, perhaps extended by [opposites coding](#) and [despite coding](#), there is no hidden vocabulary (packages, mechanisms, necessity/sufficiency, blockers/enablers, causal strength and polarity) which can be reliably applied to causally coding an interestingly wide range range of texts at scale, no universal language for mid-range theory". But that would perhaps be a claim too far.

References

Ackermann, Jones, Sweeney, & Eden (n.d.). *Decision Explorer: User Guide*. Vol. version, 3.

Ackermann, Eden, & Cropper (n.d.). *Getting Started with Cognitive Mapping*.

Aston (2023). *Contribution Rubrics*.

Axelrod (n.d.). *The Analysis of Cognitive Maps*. In *Structure of Decision : The Cognitive Maps of Political Elites*.

Axelrod (n.d.). *Structure of Decision: The Cognitive Maps of Political Elites*. Princeton university press.

Braun, & Clarke (2026). *Thematic Analysis : A Practical Guide*. SAGE Publications Ltd.

Britt, Powell, & Cabral (2025). *Strengthening Outcome Harvesting with AI-assisted Causal Mapping*.

Charmaz (n.d.). *Constructing Grounded Theory*. SAGE.

Copestake, Morsink, & Remnant (n.d.). *Attributing Development Impact: The Qualitative Impact Protocol Case Book*. March 21, Online.

Eden, Ackermann, & Cropper (2023). *The Analysis of Cause Maps*. Journal of Management Studies, 29, 309--324.

GL{\a (2026). *Life With and Without Coding: Two Methods for Early-Stage Data Analysis in Qualitative Research Aiming at Causal Explanations*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, Vol 14, No 2 (2013). <https://doi.org/10.17169/FQS-14.2.1886>.

Hodgkinson, Maule, & Bown (n.d.). *Causal Cognitive Mapping in the Organizational Strategy Field: A Comparison of Alternative Elicitation Procedures*. Organizational Research Methods, 7, 3--26.

Laukkanen (2024). *Comparative Causal Mapping and CMAP3 Software in Qualitative Studies*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, 13. <https://doi.org/10.17169/fqs-13.2.1846>.

Laukkanen, & Wang (n.d.). *Comparative Causal Mapping: The CMAP3 Method*. Routledge.

Laukkanen (n.d.). *Comparative Cause Mapping of Organizational Cognitions*. Organization Science, 5, 322--343.

Lemire, Kwako, Nielsen, Christie, Donaldson, & Leeuw (n.d.). *What Is This Thing Called a Mechanism? Findings from a Review of Realist Evaluations*. Causal Mechanism in Evaluation, 73--86. <https://doi.org/10.1002/ev.20428>.

Lynn (2026). *HU Seafood Retrospective*.

Markiczy, & Goldberg (n.d.). *A Method for Eliciting and Comparing Causal Maps*. Journal of management, 21, 305--333.

Maule, Hodgkinson, & Bown (2023). *Cognitive Mapping of Causal Reasoning in Strategic Decision Making*. In *Thinking: Psychological Perspectives on Reasoning, Judgment and Decision Making*. <https://doi.org/10.1002/047001332X.ch13>.

Mayring (2026). *Qualitative Content Analysis*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research, Vol 1, No 2 (2000): Qualitative Methods in Various Disciplines I: Psychology. <https://doi.org/10.17169/FQS-1.2.1089>.

Miles, Huberman, & Salda{\~n (n.d.). *Qualitative Data Analysis: A Methods Sourcebook*. SAGE Publications, Inc.

Pawson, & Tilley (2013). *Realistic Evaluation*. Sage Publications Limited.

Powell, Copestake, & Remnant (2025). *Causal Mapping for Evaluators*. Evaluation, 30, 100--119. <https://doi.org/10.1177/13563890231196601>.

Powell, Cabral, & Mishan (2025). *A Workflow for Collecting and Understanding Stories at Scale, Supported by Artificial Intelligence*. Evaluation.

Powell, & Larquemin (n.d.). *Does Our Theory Match Your Theory? Theories of Change and Causal Maps in Ghana*. In *Strategic Thinking, Design and the Theory of Change. A Framework for Designing Impactful and Transformational Social Interventions*.

Ragin (n.d.). *Measurement Versus Calibration: A Set-Theoretic Approach*. The Oxford Handbook of Political Methodology, 1--31. <https://doi.org/10.1093/oxfordhb/9780199286546.003.0008>.

Remnant, Copestake, Powell, & Channon (2025). *Qualitative Causal Mapping in Evaluations*. In *Handbook of Health Services Evaluation: Theories, Methods and Innovative Practices*. https://doi.org/10.1007/978-3-031-87869-5_12.

Salda{\~n (n.d.). *The Coding Manual for Qualitative Researchers*. Sage.

{Barbrook-Johnson (n.d.). *Participatory Systems Mapping*. In *Systems Mapping: How to Build and Use Causal Models of Systems*. https://doi.org/10.1007/978-3-031-01919-7_5.

A formalisation of causal mapping

📅 17 Dec 2025

Abstract

Draft for an IJSRM submission. This paper proposes a lightweight grammar and logic for encoding, aggregating, and querying causal claims found in qualitative text data. The specification is grounded in a "Minimalist" (or "Barefoot") approach to causal coding: it prioritises capturing the explicit causal claims made by sources, without imposing complex theoretical frameworks that may not align with how people naturally speak. **Intended audience:** methodologically minded evaluators / qualitative researchers, and AI/NLP readers who want a precise target representation for “causal claims in text” (and a clear separation between evidence storage vs downstream interpretation).

Unique contribution (what this paper adds):

- A compact **data model** for causal evidence in text (**LinksTable**, optional **SourcesTable**, derived **Factors**) with an explicit evidence constraint (quotes as substrings of source text when sources are present).
- A set of **syntax + semantics rules** for what counts as a valid coded claim and what can (and cannot) be inferred when aggregating claims across sources.
- A “pipeline” view of analysis as **operations on a links table**, which is designed to stay compatible with minimalist coding and to avoid accidental drift into system-model semantics (e.g., polarity/weights-by-default).

What does one piece of causal coding *mean*?

- We have syntactic rules to say what is a valid piece of causal coding.
- We have syntactic rules to say how to combine them into causal maps.
- We have syntactic rules for operations on causal maps.

For each of these rules we need to also give semantic rules to say what it also means, or how it combines meanings of its component parts.

The basic coding rule is something like:

Look for places where it is stated or claimed that one thing causes or influences another.

Causal mapping logic as rules about relationship between texts is particularly relevant in the age of LLMs.

Having a causal influence is a lot weaker and more easily than causally affecting. So a mitigating action can causally influence something but not make it happen.

Project context (companion papers)

- | | |
|--|--|
| <ul style="list-style-type: none">• Narrative coding stance and limits: Minimalist coding for causal mapping• QDA positioning: Causal mapping as causal QDA | <ul style="list-style-type: none">• Practical transforms over a links table: Magnetisation; Combining opposites, sentiment; Hierarchical coding• Coverage-style fit diagnostic: A simple measure of the goodness of fit of a causal theory to a text corpus |
|--|--|

1. Data Structures

The foundation of the specification is the **Project Data** package, which strictly separates the causal claims from the source material.

Definition Rule DS-PROJECTDATA: Project Data

- **Definition:** `ProjectData = LinksTable + optional SourcesTable`
- **Note:** `SourcesTable` may be omitted (or empty). If it is present, the evidence constraint in DS-LINKS-SOURCES applies.

Syntax Rule DS-LINKS: The Basic Links Table

A list of causal connections where each row represents one atomic claim.

- **Structure:** A table containing at least the following columns:
- **Cause:** Text label for the driver (influence factor).
- **Effect:** Text label for the outcome (consequence factor).
- **Link_ID:** Unique identifier for bookkeeping.

- **Context:** Optional identifier for distinguishing contexts.
- **Source_ID:** Identifier for the origin of the claim (e.g., document ID, participant ID).
- **Extensions:** The table may contain additional columns (e.g., *Sentiment*, *Time_Period*) or tags.

Syntax Rule DS-SOURCES: The Sources Table (optional)

A registry of documents, interviews, or respondents.

- **Structure:** A table containing:
- **Source_ID:** Unique key.
- **Full_Text:** The complete text of the source document.
- **Metadata:** Optional custom columns representing attributes of the source (e.g., Gender, Region, Date).

Syntax Rule DS-LINKS-SOURCES: Evidence Constraint (if Sources table is present)

- **Additional column:** The Links table also contains:

2. Coding and Interpretation

This section defines how text is translated into the data structures defined above.

Definition Rule COD-DEF: The Definition of Coding

Coding is the process of extracting links from source text into the Links table.

Interpretation Rule COD-ATOM: The Atomic Causal Claim

A single row in the Links table, `Link | Cause="A" | Effect="B" | Source_ID="S1"`, is interpreted as:

- "Source S1 claims that A causally influenced B in virtue of its causal power to do so."

Interpretation Rule COD-BARE: Bare Causation

The relationship `A -> B` implies **influence**, not determination.

- **Negative Definition:** It does NOT imply A is the only cause of B.
- **Negative Definition:** It does NOT imply A is sufficient for B.

Example:

- **Source Text:** "Because of the drought, we had to sell our livestock." (from Source 12)
- **Coded Link:**
- Cause: *Drought*
- Effect: *Selling livestock*
- Source_ID: *12*
- Quote: "Because of the drought, we had to sell our livestock"

Interpretation Rule COD-MIN: Minimalist Coding Principles

The coding schema prioritizes explicit claims over complex theoretical frameworks.

- **The 90% Rule:** Code the simple form "X causally influenced Y". Explicit coding of complex logic (enablers,

- **Quote:** The specific text segment evidencing the claim.
- **Constraint:** If a Sources table is provided, then for every link L in the Links table:
- **L.Source_ID** MUST match a **Sources.Source_ID**, and
- **L.Quote** MUST be a substring of **Sources.Full_Text** for the matching **Source_ID**.

Coding is strictly evidence-based.

Definition Rule DS-FACTORS: The Factors Derivation

There is no independent table for Factors stored in the Project. Factors are derived entities.

- **Definition:** **Factors** = unique values in `LinksTable.Cause + LinksTable.Effect`
- **Note:** A "Factors Table" is a transient structure created only during analysis.

sufficiency, non-linearity) is not provided for.

- **Propositions, Not Variables:** Factors are simple propositions (e.g., "Jo started shouting"), not variables with values. Distinct concepts should not be merged prematurely.
- **Example (why this matters):** Code *Poverty* and *Wealth* as distinct factors rather than values of one "Economic Status" variable. A source may claim *Poverty -> Stress* and also *Wealth -> Investment*; collapsing these too early can obscure distinct narratives.
- **No Absences:** Do not code absences. If a source does not mention a factor, it is unknown, not absent.
- **Realism & Partiality:** "X influenced Y" means X had the causal power to affect Y in this context. It implies a contribution, not total determination.

Surface cognition

- **Goal:** Model the speaker's expressed causal thinking (cognition), not necessarily underlying objective reality.
- **Method:** Code only what is explicitly said; avoid inferring hidden variables or unstated counterfactuals.

Interpretation Rule COD-MULTI: The meaning of multiple links

A table containing multiple links simply asserts the logical conjunction of the links:

- Source S1 claims that A causally influenced B in virtue of its causal power to do so.
- Source S1 claims that C causally influenced D in virtue of its causal power to do so.
- Source S2 claims that A causally influenced C in virtue of its causal power to do so.

So, unless specific contexts are specified, if Source S1 claims that *A -> C* and also that *B -> C*, this is neither a contradiction nor (by default) a claim that A-and-B jointly influenced C as a package. It is simply two separate claims.

So if families are talking about reasons for family disputes, and family F mentions social media use, and family G mentions homework, we do not usually interpret this as meaning that family F does not think that homework can also be a cause of family disputes.

3. The Filter Pipeline (Query Language)

Analysis is performed by passing a links table through a sequence of filters.

Syntax Rule FIL-PIPE: The Interpretation Pipeline

The interpretation of a result is defined by the cumulative restrictions or transformations of the filters applied.

- **Syntax:** `Input |> Filter1 |> Filter2 |> Output`
- **Multi-line Syntax:**

```
Input
|> Filter1
|> Filter2
|> Output
```

- **Processing:** Filters are applied sequentially. The output of `Filter N` becomes the input of `Filter N+1`.

Types of Filters

- **All filters take a Links table as input.** Most filters return a Links table (so they can be chained). **Output filters terminate the pipeline** by returning a derived view (table/summary) rather than a Links table.

In practice (as in the app), filter behaviour is often **multi-effect**. For example, a filter may rewrite labels *and* add tracking columns. So instead of forcing each filter into exactly one “type”, we treat each filter as having an **effect signature**:

- **Row selection:** changes *which links* are included (drops/retains rows).
- **Label rewrite:** changes *Cause/Effect* labels (recoding/normalisation).
- **Column enrichment:** adds or recalculates columns (metadata/metrics/flags), typically to support later filtering or display.
- **Configuration:** changes display/formatting settings without changing the links table (app-specific; not formalised here as a core links-table transform).

Below, filters are grouped by their **primary intent**, and each rule declares its **Effects**: line.

Row-selection filters

Syntax Rule FIL-CTX: Context Filters

Reduces the evidentiary base based on Source metadata.

- **Effects:** row selection
- **Operation:** `filter_sources | <criteria...>`
- **Interpretation:** Restrict the evidence base to links whose `Source_ID` is in the retained set of sources.

Syntax Rule FIL-FREQUENCY: Content Filters

Reduces the evidentiary base based on signal strength.

- **Effects:** row selection
- **Operation:** `filter_links | <criteria...>`
- **Interpretation:** Retain only links meeting an evidence threshold (e.g., `min_source_count=2`).

Syntax Rule FIL-TOPO: Topological Filters

Retains links based on their position in a causal chain.

- **Effects:** row selection
- **Operation:** `trace_paths | from="<factor>" | to="<factor>" | <options...>`
- **Interpretation:** "Retaining only mechanisms that connect *From* to *To*."

Label-rewrite filters

Interpretation Rule FIL-ZOOM: The Zoom Filter (Hierarchical Syntax)

Extends the logic to handle nested concepts via a separator syntax.

- **Effects:** label rewrite
- **Syntax:** Factors may use the `;` separator (e.g., `General Concept; Specific Concept`).
- **Interpretation:** `A; B` implies `B` is an instance or sub-component of `A`.
- **Operation:** `transform_labels | zoom_level=1`. If `zoom_level=1`, rewrite labels by truncating text after the first separator.
- **Inference:** At Zoom Level 1, `A; B` is treated logically as `A`.

Interpretation Rule FIL-OPP: The Combine Opposites Filter (Bivalence Syntax)

Extends the logic to handle polarity/negation.

- **Effects:** label rewrite; column enrichment
- **Syntax:** Factors may use the `~` prefix (e.g., `~Employment`) or tag pairs.
- **Interpretation:** `~A` is the negation of `A`.
- **Operation:** `combine_opposites`. Rewrites negative labels (e.g., `~Income`) to their positive counterparts (`Income`) and adds tracking columns such as `flipped_cause` and `flipped_effect`.
- **Inference:** Evidence for `~A -> ~B` is treated as corroborating evidence for `A -> B` (with flipped polarity).

Column-enrichment filters

Syntax Rule FIL-BUNDLE: The Bundling Filter

This filter aggregates co-terminal links (links with the same cause and effect) to calculate evidence metrics without reducing the row count. We normally think of it as being automatically applied after any other filter.

- **Effects:** column enrichment
- **Operation:** `bundle_links`
- **Definition (bundle object):** `Bundle(A, B) = all links L where L.Cause <mark> A AND L.Effect </mark> B`
- **Logic:** For every link `L`, identify the set of all links `S` where `S.Cause <mark> L.Cause AND S.Effect </mark> L.Effect`.
- **Transformation:** Appends new columns to the Links table:
- **Bundle:** For convenience, a text representation of the connection (e.g., "`A -> B`").
- **Citation_Count:** The total count of rows in set `S`. Represents volume of coding.
- **Source_Count:** The number of unique `Source_IDs` in set `S`. Represents breadth of evidence (consensus).
- **Lemma:** `Source_Count <= Citation_Count`.

We measure importance using two distinct metrics:

- **Citation Count:** The total number of times a link or factor was mentioned across the entire project. This counts every single row in the data.
- *Technical:* `citation_count`
- **Source Count (or Number of People):** The number of *unique* sources (people or documents) that mentioned a link or factor. This avoids double-counting if one person repeats the same point multiple times.
- *Technical:* `source_count`

Output filters

Output Rule OUT-FACTORS: Factors table view

Returns a Factors table (one row per factor) derived from a Links table (typically after `FIL-BUNDLE`).

- **Operation:** `factors_view`
- **Interpretation:** Aggregate over the set of factor labels appearing anywhere in `Cause` or `Effect`, and compute per-factor summaries (e.g., role metrics).

Output Rule OUT-MAP: Graphical map view

Returns a graphical network view of the current Links table.

- **Operation:** `map_view`
- **Interpretation (data):**

- **Nodes:** Factors (labels appearing in `Cause` or `Effect`).
- **Edges: Bundles** (one edge per `Cause -> Effect` pair), built from the **current filtered/transformed labels** (so the map reflects Zoom/Combine-Opposites/etc.).
- **Bundling:** If bundle-level columns are not already present, the map view implicitly applies `FIL-BUNDLE` to compute bundle metrics (e.g., `Citation_Count`, `Source_Count`) used for labels and styling.
- **Interpretation (presentation):** The view includes a formatting configuration that maps derived metrics to visual encodings (e.g., edge width by citation/source count; edge colour/arrowheads by mean sentiment; node colour by outcomeness; node/label sizes by frequency), plus a legend summarising the current view and applied filters.

Definition Rule MET-NODE: Factor Role Metrics

These metrics describe the topological role of a factor.

- **In-Degree (incoming citations):** Count of incoming links (times this factor appears as an Effect).
- *Technical:* `citation_count_in`
- **Out-Degree (outgoing citations):** Count of outgoing links (times this factor appears as a Cause).
- *Technical:* `citation_count_out`
- **Outcomeness:** $\text{In-Degree} / (\text{In-Degree} + \text{Out-Degree})$.
- *Interpretation:* A score nearing 1 indicates an Outcome; a score nearing 0 indicates a Driver.

4. Example Queries

Example A: The "Drivers" Query *Question: What do female participants say are the main drivers of Income?*

```
Result = ProjectData
|> filter_sources | Gender="Female"           // Rule
FIL-CTX
|> trace_paths | to="Income" | steps=1         // Rule
FIL-TOPO
|> filter_links | min_citations=2             // Rule
FIL-FREQUENCY
```

Example B: The "Mechanism" Query *Question: Is there valid narrative evidence that Training leads to Better Yields?*

```
Result = ProjectData
|> transform_labels | zoom_level=1           //
Rule FIL-ZOOM
|> trace_paths | from="Training" | to="Yield" |
thread_tracing=TRUE // Rule FIL-TOPO + Rule INF-THREAD
```

5 Causal Inference?

Inference Rule INF-EVID: Evidence is not effect size

We quantify **evidence strength**, not **causal effect strength**.

- **Observation:** `Link | Cause="A" | Effect="B"` appears 10 times.
- **Inference:** There are 10 pieces of evidence (10 coded mentions) for the claim `A -> B`.
- **Invalid inference:** The influence of A on B is 10 times stronger than a link appearing once.

Inference Rule INF-FACT: Factual Implication

If we observe `Link | Cause="A" | Effect="B" | Source_ID="S1"`:

- **Deduction:** Source S1 claims A happened/exists.
- **Deduction:** Source S1 claims B happened/exists.

Inference Rule INF-THREAD: Thread Tracing (Valid Transitivity)

We can infer a long causal chain (indirect influence) only if one source provides every step.

- **Logic:** `Link | Cause="A" | Effect="B" | Source_ID="S1"`
AND `Link | Cause="B" | Effect="C" | Source_ID="S1" =>`
Valid path `A -> B -> C`.

Inference Rule INF-CTX: The Context Rule (The Transitivity Trap)

We cannot infer causal chains by stitching together different sources without checking context.

- **Logic:** `Link | Cause="A" | Effect="B" | Source_ID="S1"`
AND `Link | Cause="B" | Effect="C" | Source_ID="S2" =>`
INVALID path `A -> C`, unless S1 and S2 share the same Context.

Context

These questions may depend on a somewhat hidden assumption: that all the causal claims (the links) come from a single context. Such as, in most cases, when all the claims are all agreed on by a group as in participatory systems mapping (PSM). For example, when we wrote "which factors are reported as being causally central?", can we really answer that by simply checking the network? From:

Factor X is central within these claims

Can we deduce:

Factor X is claimed to be central

Or, can we go from:

There are two relatively separate groups of causal claims?

Can we deduce:

It is claimed that there are two relatively separate groups of causal claims?

In general, no. It is easy to think of counter-examples. If we ask a parents and children about the causal network surrounding family disputes, we might get two relatively separate causal networks with only a little overlap. From this we cannot conclude that these respondents taken together claim that there are two relatively separate systems. It might be that the parents and children indeed are giving information about relatively separate systems about which they each have the best information, or it might be that the two groups are telling conflicting and perhaps incompatible stories.

We could express this as, say, the first axiom of causal mapping:

If a network of causal evidence from context C has property P, we can conclude that there is evidence that the corresponding causal network has property P, but again only in context C.

$P(E(N)) \rightarrow E(P(N))$

We often assume that contexts are sources and sources are contexts. But this is not always the case. For example one respondent might give two sets of information, one from before losing their job and one set from afterwards, without trying to encode the job loss as a causal factor within the network of claims.

In a PSM workshop, there may be multiple respondents but, as long as they construct a consensus map, these are all treated as one source. Part of the job of the moderator is also to ensure that the claims (evidence) all come from one context, which is the same as saying: we can validly make inferences like those above. I don't know whether PSM moderators actually do this.

You might say "this is all pointless because it depends what you mean by context", and that is exactly true. All we have done is

Appendix A: AI Extensions

These filters extend the core logic using probabilistic AI models (Embeddings or Clustering).

Interpretation Rule FIL-SOFT: The Soft Recode Filter

Extends interpretation logic using semantic similarity (vector embeddings).

- **Operation:** `soft_recode | magnets="<...>" | similarity_threshold="<...>"`

- **Inference:** If `Label A` is similar to `Magnet M` (> threshold), treat `A` as `M`.

Interpretation Rule FIL-AUTO: The Auto Recode Filter

Extends logic using unsupervised clustering.

- **Operation:** `auto_recode | target_clusters=K`
- **Inference:** Factors group together based on inherent semantic proximity into `K` emergent themes.

Plain coding

Combining opposites, sentiment

📅 23 Dec 2025

Abstract

This guide addresses a cluster of **tricky but practical problems** in causal coding: how to represent **oppositeness, sentiment/valence**, and **“despite” constructions** in a way that stays close to ordinary language and remains auditable.

Many causal mapping and systems traditions address these issues by quickly “variablising”: treating factor labels as variables, and links as signed (and sometimes weighted) relationships. That can be powerful, but it also introduces strong extra commitments (about polarity, scales, functional form, and what counts as “more/less”) that are often not warranted by ordinary narrative text.

Instead we take a **piece-by-piece approach**:

1. We start with **combining opposites** as a conservative label convention plus an explicit transform over a links table.
2. We then “turn 45 degrees” to the different-but-overlapping problem of **sentiment**, which becomes especially useful/necessary when doing AI-assisted coding and embedding-based aggregation. Sentiment and opposites are hard to combine cleanly.

3. We then introduce **“despite” coding** for countervailing conditions (“E happened despite Z”) without pretending that Z “caused” E in the ordinary way.

We end by noting some hard cases where these systems collide.

See also: [Minimalist coding for causal mapping](#); [A formalisation of causal mapping](#); [Magnetisation](#); [A simple measure of the goodness of fit of a causal theory to a text corpus](#).

Intended audience: practitioners doing causal coding from narrative text (especially at scale / with AI assistance) who keep running into polarity/valence edge cases.

Unique contribution (what this guide adds):

- A conservative **opposites convention + transform** (combine opposites while retaining flip status).
- A clear separation between **oppositeness** and **sentiment** (why they overlap in practice but don’t collapse cleanly).
- A scalable encoding for **“despite”** clauses as a link type/tag, rather than mis-coding them as ordinary causes.

Introduction

In the first part of this guide we dealt only with undifferentiated causal links which simply say “C influenced E”, or more precisely: “Source S claims/believes that C influenced E.” This is the **minimalist** representation: a links table whose rows are individual causal claims with provenance (source id + quote) and whose columns include at minimum **Cause** and **Effect** labels.

Minimalist-style causal links are commonly used in (at least) two ways:

- **Event-claim reading (QuIP-style):** interpret a link as a claim about a past episode (“C happened; E happened; C made a difference to E”), with an open question of how far it generalises.
- **Factor-relation reading:** treat links as claims about influence relations among factors, without committing to what happened in any specific case.

In Part 1 we also introduced **hierarchical factor labels** using the ; separator, where **C**; **D** can be read as “D, an example of / instance of C”, and can later be **rewritten** (zoomed) to a higher level by truncating the label.

This guide (Part 2) adds three extensions that remain compatible with minimalist link coding:

- **Opposites conventions** in factor labels (a label-level device).
- **Sentiment/valence** as an additional annotation layer (useful especially with AI coding).
- **Despite coding** to capture countervailing conditions without misrepresenting them as ordinary causes.

We will describe the conventions in app-independent terms. (The Causal Map app happens to implement these ideas as part of a standard “filter pipeline”, but the logic is not app-specific.)

Combining opposites

The problem

In everyday coding, we often end up with “opposite” factors like:

- **Employment** vs **Unemployment**
- **Good health** vs **Poor health**
- **Fit** vs **Not fit**

If we keep these as unrelated labels, we make downstream analysis harder. For example:

- When we query for “health”, we may miss evidence coded as “illness”.
- We cannot easily compare (or combine) evidence for **Fit -> Happy** with evidence for **Not fit -> Not happy** without manually re-aligning them.

Many causal mapping traditions solve this by treating factors as variables with signed links. Here we describe a simpler alternative that stays close to ordinary language and avoids variable semantics “by default”.

The convention: mark opposites in labels

To signal that two factor labels are intended as opposites, use the ~ prefix:

- Y and ~Y

We talk about **opposites** rather than plus/minus because this avoids implying valence or sentiment. For example:

- Smoking is the opposite of ~Smoking (not smoking), but which one is “good” depends on context.

Non-hierarchical opposites are straightforward:

- Eating vegetables
- ~Eating vegetables
- Smoking
- ~Smoking

The useful part: apply a transform/filter to a links table (and/or a map view)

The convention above is only a convention until we do something with it. The next step is to apply an explicit **transform** to a links table (and then to whatever views are derived from it).

The transform is:

1. Detect opposite pairs that are present (both Y and ~Y appear in the current dataset).
2. Rewrite any occurrence of ~Y to Y.
3. Record, for each link endpoint, whether it was flipped (e.g. `flipped_cause`, `flipped_effect`).

After this transform:

- We can aggregate evidence for Y and ~Y under a single canonical factor label Y **without losing the original meaning**, because we still know which endpoints were flipped.
- A link has two local “polarities”: whether the cause label was flipped, and whether the effect label was flipped.
- If **exactly one** end is flipped, the overall relationship direction is “reversed” in the intuitive sense (compared to the unflipped link).
- If **both** ends are flipped, the overall relationship direction is not reversed, but the evidence is still distinct (it came from the opposite-on-both-ends claim).

Crucially: **no information is lost** by the transform, because flip-status remains attached to the evidence. You can always reconstruct the original statement-level content from the transformed table.

This general “apply transforms to a links table; then render a map/table view of the transformed data” is the same idea as the filter pipeline described in the user-guide material: filters are operations over a links table, and maps are derived views of the filtered/transformed links.

Opposites coding within a hierarchy

When using hierarchical labels (with ;), the ~ sign may appear:

- at the very start of the whole label, and/or
- at the start of any component within the label.

The same transform idea applies: to “combine opposites” we flip components so that opposites align at each hierarchical level.

Opposites within components of a hierarchy

Sometimes we need ~ within components, e.g.:

- Healthy habits; eating vegetables
- ~Healthy habits; ~eating vegetables

and:

- Healthy habits; ~smoking (not smoking is a healthy habit)
- ~Healthy habits; smoking (smoking is an unhealthy habit)

After combining opposites, these pairs can be aligned under shared canonical labels while retaining flip status per component (so we do not collapse “healthy” into “unhealthy” or vice versa by accident).

Bivalent variables?

Opposites coding is **not** the same as assuming that every factor is a bivalent variable (present vs absent). We are not claiming exhaustiveness: it is not the case that everything must be either **Wealthy** or **Poor**, and it is often wrong to treat “absence” as having causal powers.

Opposites coding is a practical device used only where:

- both poles occur naturally in the text and therefore in the coding, and
- it would usually be incoherent to apply both poles simultaneously in the same sense.

Which pairs of factors should we consider for opposites coding?

Use opposites coding for a pair of factors X and Y (i.e. recode Y as ~X or recode X as ~Y) when both occur in the data and are broadly opposites.

If in doubt about which member to treat as the canonical label, we usually pick X as the “primary” member if it is:

- usually considered as positive / beneficial / valuable, and/or
- usually associated with “more” of something rather than “less” of something.

Alternative convention (explicit opposite pairing)

Sometimes you may have pairs that are conceptually opposite but do **not** share a clean string form like Y vs $\sim Y$ (e.g. *Wealthy* vs *Poor*).

In that case, use an explicit pairing tag so that a deterministic transform can combine them later. For example:

- *Wealthy* [1]
- *Poor* [~1]

This makes the pairing unambiguous: *Poor* [~1] is declared to be the opposite of *Wealthy* [1]. A “combine opposites” transform can then rewrite the opposite-labelled item to the canonical label while recording flip status (exactly as described above). This is the same general idea as the “transform filters temporarily relabel factors” pattern in the filter pipeline documentation (see [content/999 Causal Map App/080 Analysis Filters Filter links tab \(\(filter-link-tab\)\).md](#)), but the logic is independent of any particular software.

What can we *do* with opposites once we have them?

Once opposites are marked (and optionally combined via an explicit transform), we can apply ordinary operations to a links table and then render useful views:

- **Querying:** searching for Y can intentionally retrieve both Y and $\sim Y$ evidence (depending on whether you search pre- or post-transform).
- **Aggregation without collapse:** you can summarise evidence under a canonical label Y while still distinguishing which claims involved the opposite sense via flip flags.

- **Visualisation:** you can render a map from the transformed links table and style links differently depending on whether the cause and/or effect endpoint was flipped, so viewers can see “this includes opposite-evidence” rather than mistaking it for ordinary evidence.

This “links table → transforms → map/table view” pattern is the same general idea as a filter pipeline (implemented in many tools; the Causal Map app is one).

Adding sentiment

Polarity of factor labels

There are challenges with coding and validating concepts which on the one hand could be seen to have polarity from a quantitative point of view and on the other hand may have positive or negative sentiment associated with them. Quantitative polarity and subjective sentiment often overlap in confusing ways. When coding, distinguishing between opposites like employment and unemployment is usually important: they can be viewed as opposites, but each pole has a distinct meaning which is more than just the absence of the other. We can call these “bipolar” concepts after (Goertz 2020).

However, though employment and unemployment can be seen as opposites from a “close level” view, at a more general or abstract level, they could both fall under a category like economic issues. Their NLP embeddings may have surprisingly high cosine similarity.

For other pairs like not having enough to eat, and having too much to eat, there are multiple opposites (which may appear frequently as a causal factor) with an intervening zero (which may not be mentioned very often).

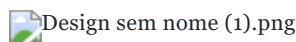
Coding these different kinds of concept pairs can be difficult and depends on use and context. For these and other reasons, our naive approach codes employment and unemployment as well as not having enough to eat, and having too much to eat separately.**

How to add sentiment?

You can now auto-code the sentiment of the consequence factor in each link.

You only have to do this once, and it takes a little while, so wait until you’ve finished coding all your links.

When you are ready, click on the File tab, and under the ‘About this file’, there’s the ‘ADD SENTIMENT’ button. You just have to click on it and wait for the magic to happen



So each claim (actually, the consequence of each claim) now has a sentiment, either -1, 0 or 1.

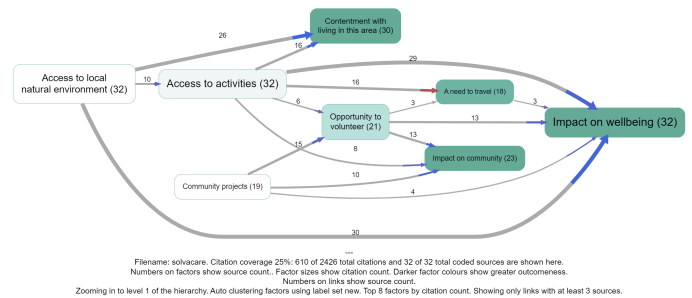
Many links are actually a bundle of different claims. We can calculate the sentiment of any bundle, as simply the average sentiment. So an average sentiment of -1 means that all the claims had negative sentiment. An average of zero means there were many neutral sentiments and/or the positive and negative sentiments cancel each other out.

Only the last part is coloured, because the colour only expresses the sentiment of the effect, not the cause.

Once you have autocoded sentiment for your file, you can switch it on or off using  [Formatters: Colour links](#).

Tip

When displaying sentiment like this, reduce possible confusing by making sure that you either use only neutral factor labels like Health rather than Good health or Improved Health: an exception is if you have *pairs* of non-neutral labels like both Poor health alongside Good health. You can do this either in your raw coding or using  [Transforms Filters](#):  [Magnetic labels](#) or  [Canonical workflow](#)



Adding some colour: a discussion of the problem of visualising contrary meanings

The problem

We’ve already described our approach to making sense of texts at scale by almost-automatically coding the causal claims within them, encoding each claim (like “climate change means our crops are failing”) as a pair of factor labels (“climate change” and “our crops are failing”): this information is visualised as one link in a causal map. We use our “coding AI” to code most of the causal claims within a set of documents in this way. We have had good success doing this quickly and at scale without providing any kind of codebook: the AI is free to create whatever factor labels it wants.

There is one key remaining problem with this approach. Here is the background to the problem: if the coding AI is provided with very long texts, it tends to skip many of the causal claims in fact contained in the documents. Much shorter chunks of text work best. As we work without a codebook, this means that the AI produces hundreds of different factor labels which may overlap substantially in meaning. In turn this means that we have to cluster the labels in sets of similar meaning (using phrase embeddings and our “Clustering AI”) and find labels for the sets. This all works nicely.

But the problem is that, when we use phrase embeddings to make clusters of similar labels, seeming opposites often have high cosine similarity. Unemployment and employment vectors are similar – they would for example often appear on the same pages of a book – and both are quite different from a phrase like, say, "climate change". But this is unsatisfactory because if in the raw text we had a link describing how someone lost their job, coded with an arrow leading to a factor unemployment alongside another piece of text describing how someone gained work, represented by an arrow pointing to employment if these two labels are combined, say into employment or employment issues the items look very similar and we seem to have lost some essential piece of information.

Can't we use opposites coding?

In ordinary manual coding (see [+ - Opposites](#)) we solve this problem by marking some factors as contrary to others using our ~ notation (in which ~Employment can stand for Unemployment, Bad employment, etc) and this works well. However while it is possible to get the coding AI to code using this kind of notation, it is not part of ordinary language and is therefore not understood by the embeddings API: the ~ is simply ignored even more often than the difference between Employment and Unemployment. In order to stop factors like employment and unemployment ending up in the same cluster it is possible to exaggerate the difference between them by somehow rewriting employment as, say, “really really crass absence of employment” but this is also unsatisfactory (partly because all the factors like really really crass X tend to end up in the same cluster).

New solution

So our new solution is simply to accept the way the coding-AI uses ordinary language labels like employment and unemployment and to accept the way the embedding-AI clusters them together. Instead, we recapture the lost “negative” meaning with a third AI we call the “labelling AI”. This automatically codes the sentiment of each individual causal claim so that each link is given a sentiment of either +1, 0 or -1. For this third step we use a chat API. The instruction to this third AI is:

"I am going to show you a numbered list of causal claims, where different respondents said that one thing ('the cause') causally influenced another ('the effect') together with a verbatim quote from the respondent explaining how the cause led to the effect.

The claims are listed in this format: 'quote ((cause —> effect))'.

The claims and respondents are not related to one another so treat each separately.

For each claim, report its *sentiment*: does the respondent think that the effect produced by the cause is at least a bit good (+1), at least a little bad (-1) or neutral (o).

Consider only what the respondent thinks is good or bad, not what you think.

If you are not sure, use the neutral option (o).

NEVER skip any claims. Provide a sentiment for every claim."

The previous solution coloured the whole link which was fine in most cases but led to some really confusing and incorrect coding where the influence factor was involved in the opposite sense, as in Access to activities below. One might assume that the red links actually involve some kind of negative (or opposite?) access to activities, but we don't actually know that because it wasn't coded as such. Other alternatives would be to also automatically separately code the sentiment of the first part of the arrow, but this doesn't work because sometimes the sentiment is not in fact negative. We would have to somehow automatically code whether the influence factor is meant in an opposite or contrary sense but this is hard to do.

**

Worked example: short interview

See this [example](#).

The short interview excerpt below is designed to yield the causal claims used in the examples above: enough to illustrate opposites coding inside a hierarchy of healthy habits and good health.

Interview: Priya

I used to smoke about ten a day all through my twenties and I never really thought about vegetables. Honestly I was unfit then. I would get out of breath walking up the hill to my flat, and I was low quite a lot of the time, although I did not really join the dots back then.

Things changed when I turned thirty-five. I gave up smoking and started cooking properly with lots of greens. Within about six months I was a different person. The fitness came back quite quickly once the cigarettes were out of the way, and I was in a much better mood for it. Being fit just lifts you, you feel capable.

The claims, coded hierarchically

Two hierarchies are used: **Healthy habits** groups the diet and smoking claims, and **Good health** groups fitness and mood. The six claims become:

- ~Healthy habits; smoking causes ~Good health; physical fitness
- ~Healthy habits; ~eating vegetables causes ~Good health; physical fitness
- Healthy habits; ~smoking causes Good health; physical fitness
- Healthy habits; eating vegetables causes Good health; physical fitness

- ~Good health; physical fitness causes ~Good health; positive mood
- Good health; physical fitness causes Good health; positive mood

Combine opposites then aligns labels at each level of the hierarchy. ~Healthy habits; ~eating vegetables pairs with Healthy habits; eating vegetables, and ~Good health; physical fitness pairs with Good health; physical fitness. The transform preserves the flip status of each link end, so the original wording of every claim stays visible. Zoomed out to the top level, the diet and smoking claims collapse onto a single canonical bundle from **Healthy habits** to **Good health**.

References

Goertz (2020). *Social Science Concepts and Measurement*. Princeton University Press.

Despite-claims

24 Jan 2026

"Despite" coding

Start with minimalist causal mapping

Causal mapping is a way of analysing what people say about causes. We read a text, an interview, a report, an open survey answer, and wherever someone claims that one thing influences another, we code that claim as a link. If a respondent says that the heat made them thirsty, we write down a link from **Heat** to **Feeling thirsty**, together with a note of where it came from. That is the whole basic move, and it is easy: find a causal claim, record the cause, the effect, and the source. (See and .)

A single coded link looks like this: a box for the cause, a box for the effect, and an arrow from one to the other.



Gather many such links from many sources and they join into a causal map.

This short paper is about the formal coding part. The question is not why "despite" matters, but how to record these near-misses and failures so they mean something inside the coding system, rather than as a handwavy line with a funny arrow on it.

Our approach codes only bare cause-to-effect links and leaves most detail off the link. "Despite" is an example of an hue of causation that we can't capture in our basic system. But it is an idea we don't want to do without: it's frequently asserted and it seems to have a strong causal meaning and relevance. So, as reluctant as we always are when adding extra bits to a very minimal system, we add a "despite extension" to a small set of extensions to the base (alongside opposites and sentiment, see). We declare this type of link as a special type that the app or any other causal link database can store, filter and count, as just another row in the table of links, but with some special marker.

The puzzle: an influence that could have worked, but did not

Narratives do not only tell us what worked. Sometimes they tell us about an influence that could have worked but did not. Someone says:

"The river still rose, despite the flood-prevention work."

The flood-prevention work is the kind of thing that pushes against rising water. It had some causal power to hold the river down. In realist terms (see) it was a real influence with a real tendency to work against the outcome. But on this occasion it was not enough: maybe something pushed harder the other way, and the river rose anyway.

This sentence names only the prevention work, not whatever drove the river up. The dashed pink arrow records the influence that was present but failed.



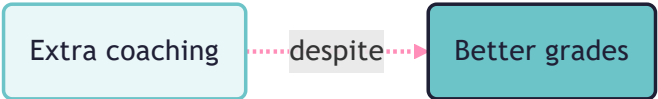
"Despite" claims come in two flavours, depending on which way the failed influence was meant to push. Basically it's the same thing, but we tend to express them a bit differently in words.

A blocking influence that failed. Z was the sort of thing expected to prevent or reduce E, but E happened anyway. The flood-prevention work above is this kind: it should have held the river down, and the river rose all the same.

A helping influence that failed. Z was the sort of thing expected to bring about or increase E, but E did not happen. For example:

"Despite the extra coaching, her grades did not improve."

The coaching had the causal power to lift her grades. It was a real, relevant influence. But the improvement did not come.



Either way, the despite clause carries two pieces of information at once:

- 1. A relevant influence was present. Z had the causal power to push the outcome one way, whether to block it or to help it.
- 2. On this occasion it failed. Z was insufficient, absent or overridden, so the outcome went the other way.

Why the obvious options both fail

What should a coder do with the "despite" clause?

- If we code an ordinary link **Flood-prevention work** -> **River rose**, we assert the opposite of what the speaker meant, namely that the prevention work made the river rise.
- If we ignore the clause and code nothing, we throw away both pieces of information above: that a countervailing influence was present, and that on this occasion it failed, and the other thing happened (or failed to happen).

Neither is satisfactory. We want to keep the speaker's meaning and keep the information.

But couldn't you just use opposing arrows like in systems modelling?

A systems modeller will say there is no problem here. Code two arrows into the outcome: a positive one from the rains, which raise the river, and a negative one from the prevention work, which lowers it. Whichever is stronger wins, and the river's level is the net result. For systems modelling that is a reasonable move, and it is roughly how signed, weighted influence diagrams already work.

But look at what it takes to make those arrows say what the speaker said. To record that the prevention work lost and the river rose anyway, the modeller has to put magnitudes on the two arrows and compute a resultant, or add further codes for "this one prevailed". The opposing-arrows picture on its own does not tell you which side won, or whether the outcome actually happened.

The real difference is that we are not building a model of the river system. We are encoding what people say and think: claims, cognitions, something much closer to language than to a quantitative model of forces (see and). When someone says "in spite of", "although" or "even though", they are expressing an idea with a particular flavour (see Talmy n.d.): they name an influence that was real and relevant, then set it aside as having failed. It is open to question whether two opposing arrows, one a little larger than the other, captures that idea at all.

So despite-coding is an extension we add to capture, in the language of claims, something a systems modeller can arguably get out of the box from signed and weighted arrows. The trade is straightforward: we stay close to what people say and pay for it with one extra link type, where the systems modeller pays for the net-result calculation with numbers and machinery which as humble causal mappers we do not have or want.

The convention: a "despite" link type (or tag)

We therefore treat "despite" as a special **link type**.

A minimal encoding is:

- **Main link** (ordinary): Heavy rain -> River rose
- **Despite link** (typed or tagged): Flood-prevention work - despite-> River rose

Equivalently, in a plain links table, we keep the same Cause and Effect columns but add either:

- a **link_type** column with values like normal and despite, or
- a **link_tags** column holding a tag like #despite.

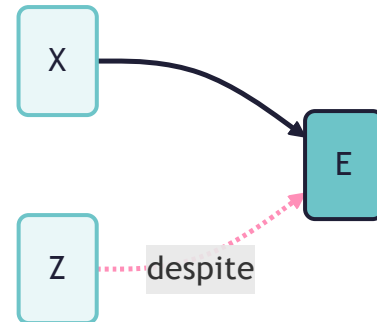
This is the same idea as attaching other metadata to a link (see). The point is that -despite-> does **not** mean "Z caused E". It means one of the two flavours above:

"Z was presented as a countervailing influence against E, in virtue of its causal power to work against E, but E occurred anyway." (the blocking case)

or

"Z was presented as an influence in favour of E, in virtue of its causal power to do so, but E failed to occur anyway." (the helping case)

In a map, the ordinary link and the despite link both point at the same effect, but they look different: the solid arrow is the ordinary influence, the dashed pink arrow is the despite link.



What can we do with "despite" links?

Because the distinction is explicit in the links table, we can decide, per analysis, how to treat these links:

- **Visualise** them distinctly, with a different colour or line style, so the map shows the tension rather than dropping it or misreading it.
- **Filter** to show only despite links, to see what people present as failed protections, resistances or mitigations.
- **Compare:** for similar outcomes, do some sources describe Z as effective, an ordinary preventive claim written $Z \rightarrow E$ (the tilde ~ marks the opposite of a factor, so ~E means E did not happen, or the opposite of E; see), while others describe a "despite" failure ($Z \text{ -despite-} \rightarrow E$)? That contrast can matter analytically.
- **Count separately**, under their own counter (for example `Despite_Citation_Count`), so you do not accidentally inflate the evidence for "Z causes E" (on counting evidence, see and).
- **Count together**, so we can at least see how often this causal power was present regardless of whether it was effective or not.

Worked examples

Text:

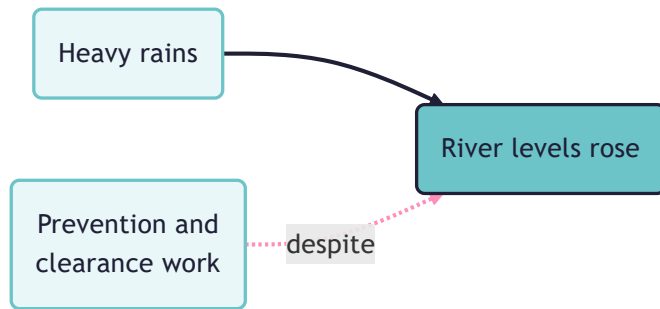
"The heavy rains made the river levels rise, despite the prevention and clearance work."

Coding:

- Heavy rains -> River levels rose
- Prevention and clearance work -despite-> River levels rose

This keeps the main mechanism while also recording that a mitigation was present but insufficient.

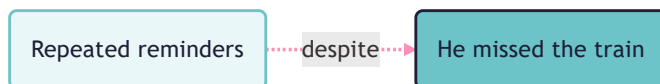
As a map, the two links sit side by side on the same outcome. The solid arrow is the ordinary influence; the dashed pink arrow is the despite link.



Sometimes the speaker gives no positive cause at all, only the countervailing one that failed:

"Despite me reminding him several times, he still missed the train."

Here the only thing to code is the despite link. The reminders had the power to prevent the missed train, but did not.



Optional background: force dynamics

You do not need any theory to use `-despite->` coding. But it helps to know why "despite" claims feel different from ordinary causal claims.

Leonard Talmy's "force dynamics" (Talmy n.d.) treats much of our causal talk as patterned talk about forces: an **Agonist** with a tendency, towards motion or rest, and an **Antagonist** that counters it. Talmy analyses "causing" into finer ideas that include "letting", "hindering" and "helping", and notes that the same pattern runs from the physical through to the psychological and social. "Despite" is a canonical force-dynamics marker: it signals a countervailing force that failed to stop the outcome. "The ball kept rolling despite the long grass" has the ball as Agonist and the grass as the Antagonist that did not prevail.

This is why "despite" is worth capturing explicitly: it shows that a causal power was present and was overcome, which is information about how the world works, not just noise to discard.

Despite links inside packages

Often the narrative names both the influence that failed and the influence that overcame it: "the fan failed to cool him down because the sun was too strong". The despite link (`Fan -despite-> Feeling cooler`) and the ordinary link (`Strong sun -> Feeling hot`) tell one story, and we can record that by giving both the same package code with type `DESPITE`. See the sister paper [Coding packages of links](#) for the package convention.

Hard cases (brief notes)

- **"Despite" plus opposites:** should `Z -despite-> E` count as evidence for `Z -> ~E`? Usually no. Treat it as its own link

type, or at most as weaker evidence, depending on the analysis goal. See for how opposites and sentiment interact with this.

- **Bipolar concepts:** some pairs (employment and unemployment) are mentioned in very similar contexts but are opposite in ordinary talk. This is why also coding sentiment often becomes very useful when doing AI-assisted clustering and aggregation.

A more formal picture

This puts despite-coding into the grammar of [A formalisation of causal mapping](#). There, an ordinary link is read with rule COD-ATOM as "Source S claims that X causally influenced Y in virtue of its causal power to do so", and rule INF-FACT lets us deduce from any link that S claims X happened and Y happened. A despite link adds another link type to that grammar.

Semantics (the blocking case). A row `X -despite-> Y` from source S is read as:

Source S claims that X happened, and Y happened, and X had some causal power to make `~Y` (to work against Y), but X did not prevail.

Three deductions, three lists. From that one row we can deduce three things, each feeding a different query (compare INF-FACT):

- **X occurred (with caveats).** S claims X happened. So when listing claims that the Xs happened, include despite links, with the caveat that "despite" sometimes frames X as only partly realised or as what should have been done.
- **Y occurred.** S claims Y happened. So when listing cases where Y happened, include despite links on the same footing as ordinary links.
- **X had countervailing power (with caveats).** S claims X had some causal power to make `~Y`. So when listing cases where X is claimed to have had the power to prevent or reduce Y, include despite links, with the caveat that this is the speaker's claim about a power that did not, in the event, carry the outcome.

The helping case. When the despite clause is of the helping kind ("despite the coaching, the grades did not improve"), the same row is read with Y negated:

Source S claims that X happened, and Y did not happen, and X had some causal power to make Y, but X did not prevail.

The three deductions mirror the blocking case: read "Y did not happen" for the second, and "power to make Y" for the third.

What a despite link is not. On its own it is never evidence for an ordinary `X -> Y`. It is closer to evidence about `X -> ~Y`, but we keep it as its own type rather than folding it into the opposites machinery (rule FIL-OPP in the formalisation; see also the hard case above and).

References

Talmy (n.d.). *Force Dynamics in Language and Cognition*. Cognitive Science, 12, 49--100. https://doi.org/10.1207/s15516709cog1201_2.

Coding packages of links

📅 13 Jul 2026

Coding packages

Start with minimalist causal mapping

Causal mapping is a way of analysing what people say about causes. Wherever someone claims that one thing influences another, we code that claim as a link: a cause, an effect, a quote and a source ID. That is the whole basic move (see [minimalist coding](#)).

Our [minimalist](#) approach codes only bare cause-to-effect links and leaves most detail off the link. Like [despite coding](#), what follows is a small extension we add reluctantly, because it captures something people say often enough, and clearly enough, that throwing it away feels wasteful. As with despite links, the extension is just another marker on ordinary rows in the links table: nothing about the core coding move changes.

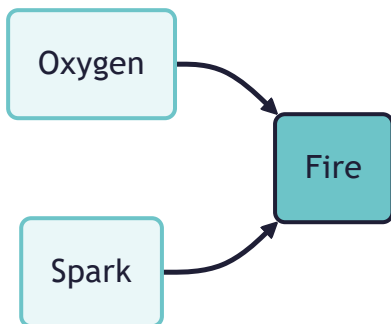
The puzzle: causes that are claimed to work together

Sometimes a source does not present two influences as independent. They present them as a team:

"You need both oxygen and a spark to make fire."

Minimalist coding gives us two links:

- Oxygen -> Fire
- Spark -> Fire



Each link on its own is fine. But together they have quietly dropped the speaker's central point: that neither influence works alone. Oxygen without a spark gives you nothing; a spark without oxygen gives you nothing. The claim was about a *combination*, and two separate rows in a links table cannot say so.

This is the problem of causal packages or conjunctions, familiar from realist evaluation, from INUS conditions in the philosophy of causation (n.d.), and from everyday talk. It is one of the recognisable ways that pure minimalist coding runs out (see the discussion in [the minimalist paper](#)).

Why the obvious options both fail

- **Code the two links separately** and the conjunctive structure is lost without trace. Worse, each link now reads as

an independent claim of influence, which is subtly more than the speaker said.

- **Code one combined factor** such as *Oxygen and a spark -> Fire*, and the label is no longer parsable: it cannot be related to other claims about oxygen, or sparks, on their own. Clustering, recoding and label search all degrade.

We want to keep the individual factors as individual factors, and still record that on this occasion they were claimed to work as a team.

The convention: a shared package code

We give the links that belong together a shared, otherwise meaningless code, plus a name for *how* they belong together. Concretely, each link in a links table can carry two extra pieces of metadata:

- **package**: an arbitrary unique code (say *pk_x7f2*) shared by all the links in one package and by nothing else in the dataset.
- **package_type**: the named connective, most commonly **AND** or **OR**, sometimes **DESPITE** (below), and **?** where the speaker grouped the causes without saying which. The vocabulary is open: these four cover most of what we meet, but a coder can write anything.

So the fire claim is coded:

- Oxygen -> Fire (package: *pk_x7f2*, type: **AND**)
- Spark -> Fire (package: *pk_x7f2*, type: **AND**)

The two links are still ordinary links: they appear in maps, they carry quotes, they can be filtered and counted like any others. The package code is the only thing tying them together, and the type says what the tie means.

Both fields sit on the link, not on some separate package object. For **package** that is just bookkeeping, since the code is the same on every member. For **package_type** it is a real choice, and we say more about it under [asymmetric packages](#) below: usually every member of a package carries the same type, but they need not, and the cases where they differ are interesting.

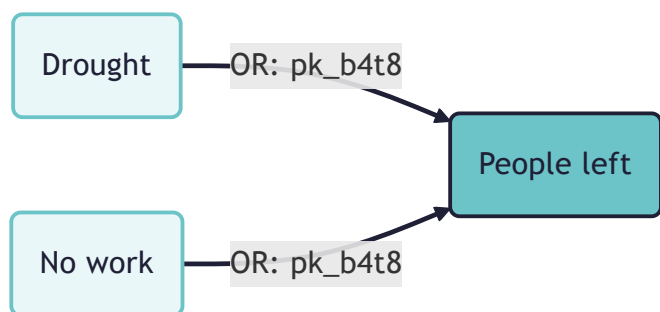


AND packages

The commonest case: a claimed conjunction. Each factor is presented as needed; the effect follows from the combination. "The training only worked because the community already trusted the facilitators" can be coded as two links into the outcome sharing an **AND** package.

OR packages

The source presents alternatives: "people left because of the drought, or because there was no work"; "either the pension or the remittances would have kept them afloat".



OR is less well defined than **AND**, and it is fair to ask whether it needs a package at all. In several coding traditions, alternative sufficient causes are exactly the case where no extra structure is wanted: if either influence would have done the job on its own, the two links really are independent claims, and coding them separately loses nothing. On that view an **OR** package is redundant by definition.

We keep it anyway, for a weaker reason: the package records that the speaker *mentioned the influences together as alternatives*. That is itself information about the claim. "Either would have done it", "sometimes one, sometimes the other", "one or other of these was always the reason": these are not the same as two separate unqualified claims of influence, and a coder reading one link later may want to know that the speaker was hedging between alternatives rather than asserting each outright. The **OR** package carries that, and nothing more. As with the rest of the convention, we do not pin down which flavour of "or" was meant (exclusive or inclusive, alternation over occasions, uncertainty between explanations); if that matters for an analysis, the quote is on each link.

In practice we treat **OR** packaging as really optional. People do not often speak or write in true either-would-have-done alternatives, so the case arises rarely, and when it does, coding two plain links is usually an adequate record. Use an **OR** package only when the togetherness of the alternatives seems worth keeping.

? packages

Often a speaker gives two or more causes in the same breath without saying whether both were needed or either would have done: "we got there because of the funding and because the new manager pushed it through". Forcing a choice between **AND** and **OR** here puts words in the speaker's mouth. The honest connective is **?**: the package records that the causes were presented together as one claim, and stays silent on how they combine. In coding practice **?** is the default for same-breath groupings, with **AND** reserved for causes presented as needed together and **OR** for explicit alternatives.

DESPITE packages

Despite links record an influence that was present and relevant but failed. Often the narrative names both the failed influence and the influence that overcame it:

"The fan failed to cool him down because the sun was too strong."

Coding:

- Fan -despite-> Feeling cooler (the failed helping influence)
- Strong sun -> Feeling hot (the influence that prevailed)

These two links tell one story, and a package records that: both get the same package code with type **DESPITE**. The package says "these belong to a single episode of one influence overcoming another", which neither link says alone.



Packages of effects

The same move works in the other direction. A speaker can fan one cause into several effects in a single claim: "the programme raised awareness and changed behaviour". Coding two links and giving them a shared package records that this was one claim with two outcomes rather than two separate claims, using the same connectives (**AND** where both outcomes are asserted, **?** where the grouping is looser).

A package of effects matters less for analysis than a package of causes. Two effect links read separately rarely overstate what the speaker said, whereas a single link quoted out of an **AND** package of causes does. But it is arguably the more natural reading of such sentences, which present one claim fanned out, and allowing it keeps the convention symmetrical: a package groups the links of one joint claim, whichever end the conjunction is on. (Until July 2026 the convention grouped causes only; effect packages were admitted as a simplification.)

Asymmetric packages: the type belongs to the link

Because **package_type** is a column on the link, the members of one package can in principle carry *different* types. Most of the time they do not: an **AND** package is **AND** all the way through, and that is what the app fills in for you. But the possibility is not an accident, and it is worth seeing what it buys.

Take the despite pair again ("the fan failed to cool him down because the sun was too strong"). You could describe it as an asymmetric package: one member is the influence that failed, the other is the influence that prevailed, and those are plainly not the same kind of membership. We do not in fact code it that way. The failure is a property of the link itself, so we mark the failed link as a failure (see **Despite-claims**) and let the shared package code do nothing more than tie the episode together. Recording the asymmetry twice, once in the link's own failure marker and again in a pair of contrasting package types, would be redundant and would invite the two records to disagree.

The case that asymmetric types are genuinely *for* is different, and more interesting: **claims about how one causal link affects another**. Suppose a source says that factor A, acting through link A, changes whether factor B can work through link B at all. This is the enabler and blocker problem, the second-order causal claim discussed in [Minimalist coding for causal mapping](#): an arrow pointing at an arrow, rather than at a factor. A package can hold the two links together, and asymmetric types can then say which is which: one link is the mechanism, the other is the thing acting on that mechanism. Something like a meta-mechanism, coded without inventing a new object in the data structure.

We flag this as a possibility rather than a settled convention. We have not fixed a vocabulary for these types, and until we have coded enough real examples to know what recurs, we would rather leave the field open than standardise something premature. The minimalist instinct applies here as everywhere: the extension exists because the data structure permits it cheaply, and it should stay unused until the texts demand it.

The semantics: weak by design

A package is a *claim of joint working*, nothing more. Reading a package of links {L1, L2, ...} with type T from source S:

Source S presents the influences coded in L1, L2, ... as working together in the manner named by T to affect the outcome(s) they point to.

We go no further, on purpose. In particular we do not commit to:

- **Truth tables or INUS structure.** An **AND** package does not assert that the factors are individually necessary and jointly sufficient. It asserts that the speaker presented them as a combination.
- **Interaction effects or functional form.** No claim about how the combined influence scales.
- **Completeness.** The package contains the co-causes the speaker mentioned, not all the co-causes there are.

This mirrors the minimalist stance for ordinary links: we record the surface shape of the claim, with provenance, and defer stronger modelling to the analyst. The deductions that ordinary links license still hold for each link in a package (the factors happened, the source claimed influence), with the caveat for **AND** packages that the influence claimed for each factor is influence *as part of the combination*, so a single link from an **AND** package quoted on its own slightly overstates what the speaker said. Keeping the package code on the row is what lets a careful analyst notice this.

What can we do with packages?

Because the package structure is explicit in the links table, we can:

- **Filter** to show only links that are in any package (where **package** is not empty), to review all the conjunctive coding in a project at once.
- **Show a single package** (where **package** equals a given code) to see one claimed combination in isolation, with its quotes.
- **Filter by type**, for example all **DESPITE** packages, to see every episode where one influence was claimed to overcome

another.

- **Count** packages separately from ordinary links, so conjunctive claims are not silently inflated into independent evidence.
- **Visualise** packaged links distinctly if wanted, though the default map view simply shows the ordinary links.

How this is coded in the Causal Map app

The [Causal Map app](#) stores **package** and **package_type** as standard columns on every link.

- **While coding:** when a statement gives several causes for one effect, enter the FROM factors as usual, switch on the package toggle, and pick or type the package type (**AND**, **OR** and **DESPITE** are offered). The app mints the shared package code itself; you never type or track codes by hand.
- **After coding:** select several rows in the links table and package them (a fresh shared code) or unpackage them (clear their codes) in one action. The packaging dialog lists each selected link and gives it its own type field, so the usual single-type package is one click while an [asymmetric package](#) remains possible.
- **For analysis:** the Everything Filter can select on **package** and **package_type** like any other link field: show only packaged links, only one package, or only one type.

The filter engine behind these selections is also published as a standalone open-source package, ([n.d.](#), [/filter-engine](#)) ([https://www.npmjs.com/package/\(n.d.\)/filter-engine](https://www.npmjs.com/package/(n.d.)/filter-engine)), so the same **package/package_type** filtering runs on a links table outside the app.

Hard cases (brief notes)

- **Don't package the merely co-mentioned.** Two causes in one sentence are not automatically a package. Package only when the speaker presents the combination itself as doing the work ("both... and...", "only because... together with...", "either... or...").
- **Packages versus bundles.** A *bundle* is our name for co-terminal links (same cause, same effect) collapsed for display; it is a display-time aggregation. A package is a coded claim tying *different* links together. The two are unrelated mechanisms.
- **One link, one package.** In this convention a link carries at most one package code. Truly nested combinations ("A and either B or C") are rare in ordinary talk; when they matter, handle them in the write-up.
- **Counting.** Whether an **AND** package of three links counts as one piece of evidence or three depends on the question being asked; the package code is what makes either count possible.

A more formal picture

In the grammar of [A formalisation of causal mapping](#), a package adds one rule on top of the atomic coding rule COD-ATOM: a set of links sharing a package code is read with the joint-working semantics above (call it COD-PACK), parameterised by the type each member carries (usually one type throughout; see [asymmetric packages](#)). The factual deductions of INF-FACT still apply per link. No new inference rules are licensed by default: in particular, we do not license inferring the effect from the presence of all **AND** members, because packages record claims about combinations, not verified mechanisms.

Causal mapping as causal QDA

📅 23 Dec 2025

Abstract

Causal mapping is a well-established family of approaches in social science for representing “what influences what”, according to sources, as a network of claims. This paper presents causal mapping as an interesting variant of Qualitative Data Analysis (QDA) in which the primary act of coding is not “apply a theme”, but **code a causal link** (an ordered pair of cause/effect labels) grounded in a quote and source. The resulting list of causal links can then be queried (filtering, tracing paths, etc) to answer research questions. Qualitative judgement (what are the main cause/effect labels and how are they organised?) remains central while many of the other tasks become more reproducible, checkable, and scalable. We will demonstrate causal mapping using Causal Map (app.causalmap.app) which is free to use for public projects.

This paper is written for QDA/CAQDAS users who want:

- a clear definition of causal coding as a qualitative method,
- an account of why it can be simpler to apply than broader forms of thematic coding,
- an introduction to how causal analysis can provide answers to useful questions,
- a comparison of causal mapping to neighbouring approaches (thematic analysis, qualitative content analysis, systems modelling).

Unique contribution (what this paper adds):

- It defines causal mapping as **link-coding with provenance** (a links table as the core qualitative product), and it is explicit about the semantics: **claims ≠ facts**.
- It frames analysis as an explicit **pipeline of operations** over the links table (filters/transforms → derived views), rather than as a single narrative synthesis step.
- It proposes a bounded role for **LLMs** as a **checkable extraction assistant** (“clerk”) and locates the interpretive burden in human choices about transforms and synthesis (“architect”).

See also:

- [Minimalist coding for causal mapping](#) (coding stance, extensions, limits)
- [A formalisation of causal mapping](#) (companion spec)
- [Combining opposites, sentiment](#) (opposites/sentiment/despite as extensions)
- [Magnetisation](#) (soft recoding / magnets)
- [A simple measure of the goodness of fit of a causal theory to a text corpus](#) (coverage-style fit diagnostics)

1. What is “causal QDA”?

In ordinary qualitative coding, a code typically denotes a **concept/theme**. In causal QDA, a coded unit denotes a **causal claim** expressed in the text:

- a **cause (influence factor)** label,
- an **effect (consequence factor)** label,
- a **verbatim quote** that provides evidence for the claim,
- and a **source identifier** (so we can maintain provenance).

One coding act yields an ordered pair: **Cause -> Effect**. A dataset of such acts yields a **links table**. The set of factor labels is derived from the endpoints of links; factors “exist” here primarily as participants in claims.

This paper treats that links table as the core qualitative product: a structured repository of **evidence-with-provenance** that can be inspected and re-analysed.

In evaluation practice, closely related “causal-claims-first” ways of working with narrative evidence already appear in the wild (e.g. causal mapping as described on BetterEvaluation (Evaluation 2024), and QuIP-style narrative causal-map visualisations (Evaluation 2021)).

1.1 A worked example (what one coding act looks like)

If an interviewee says:

“The floods destroyed our crops.”

then a causal-coding act records a link such as:

- **Floods -> Crops destroyed**

along with the quote and its source id. In this framing, the link is the code. A set of such links can be immediately visualised as a network (or analysed as a table) with off-the-shelf tools.

2. Why causal coding is often easier (and more checkable) than “find the themes”

The instruction “find the main themes” is (legitimately) open-ended: it depends on theoretical stance, positionality, research question, and the analyst’s preferred granularity. That openness is often a feature of “Big-Q” qualitative work; but it makes systematic comparison and scale difficult (e.g. thematic analysis (Braun & Clarke 2021)).

This difference is also visible when people use generative AI. “List the main themes in this document” can be a useful time-saver, but it is massively sensitive to what one means by *theme* (and to the analyst’s implicit theory of what “matters”). You can narrow the prompt (“Identify the main kinds of relationship issues mentioned”), but at that point you are already moving from open generation towards a more constrained extraction task.

The causal coding task is narrower:

Identify each passage where the text says that one thing influenced another, and record what influenced what.

This does not remove judgement (labels still matter; causal language can be ambiguous), but it reduces degrees of freedom at the point of coding. In practice, that usually improves:

- **traceability** (every link can be checked against a quote),
- **comparability** (multiple coders/teams can aim at the same target representation),
- **automation** (an AI can be constrained to do the low-level extraction task).

This is a “small-Q” move: it is not a claim that causal QDA replaces interpretive qualitative work, but that it is a useful, rigorous option when the research questions are themselves causal (which they often are in evaluation and applied social research).

2.1 Example: “themes” vs “links” on the same excerpt

Consider:

“After the clinic started opening on Saturdays, I didn’t have to miss work, so I could actually attend.”

A “theme finding” pass might code: *Access*, *Clinic opening hours*, *Employment constraints*, *Attendance*.

A causal-coding pass would typically try to capture the explicit influence structure:

- *Saturday opening* -> *Not missing work*
- *Not missing work* -> *Attendance*

Both can be valuable, but the causal representation is immediately queryable as a mechanism (and can be checked line-by-line against quotes).

3. The output is not “just codes”: it is a queryable qualitative model

Ordinary QDA typically culminates in a narrative account plus some supporting tables. Causal QDA yields a different primary object: a **network of causal claims**.

Once you have a links table, you automatically have a graph:

- nodes = factor labels
- directed edges = coded claims (often bundled by identical endpoints)

That graph is a qualitative model in a specific sense:

- it is a model of **what sources claim** influences what,
- not (by itself) a model of causal reality.

The payoff is that you can answer many questions *by querying this model* — without needing to ask an AI to produce a global synthesis, and without hiding methodological steps inside the analyst’s head.

3.1 Example questions that become natural once you have a links table

- “What are the most frequently mentioned upstream influences on *Attendance*?”
- “How do the pathways into *Wellbeing* differ for younger vs older respondents?”
- “Which links are contested (both *X* -> *Y* and *X* -> *~Y* appear), and by which subgroups?”

4. A transparent “library of operations” (filters + views)

Causal mapping analysis is often best described as a **pipeline**: pass the links table through a sequence of operations, then render a view (map/table).

At a high level these operations fall into:

- **Row selection**: restrict sources and/or links (contexts, evidence thresholds).
- **Topological selection**: retain only links on paths relevant to a question (e.g. mechanisms connecting *X* to *Y*).
- **Label transforms**: rewrite labels for summarisation (e.g. zoom/hierarchy; opposites).
- **Bundling + metrics**: compute *Citation_Count* / *Source_Count* etc.
- **Views**: map view, factors view, tables, exported summaries.

The crucial methodological point is not which software you use, but that the meaning of a derived map is always:

“This view of the evidence, after these explicit transformations.”

This is what makes the method checkable and extensible: other researchers can replicate the same pipeline on the same links table, or change one step and see what changes.

4.1 Example: a concrete analysis pipeline (from a broad corpus to a specific view)

Suppose you want: “Mechanisms connecting **Training** to **Adoption**, but only for the younger respondents, and only where more than one source supports each link.”

One explicit pipeline could be:

- **Context selection:** keep only sources where **Age_Group** = **Younger**.
- **Evidence threshold:** keep only link bundles with **Source_Count** ≥ 2 .
- **Path tracing:** retain links on paths from **Training** to **Adoption** (with an explicit path-length limit).
- **(Optional) zoom:** rewrite hierarchical labels to level 1 to produce a high-level view.
- **Render view:** show the resulting subgraph as a map, but keep click-through to the underlying quotes for each remaining link.

The point is not that this exact pipeline is “right”, but that it is explicit: the reader can see what was done, rerun it, and inspect what evidence is inside the view.

4.2 Causal QDA as “qualitative split-apply-combine” (and why this is a good place for AI)

A useful way to describe this workflow is as a qualitative variant of the **split-apply-combine** strategy: break a hard analytic problem into manageable pieces, operate on each piece consistently, then recombine into a coherent answer (Wickham 2011).

In causal QDA, the mapping is unusually clean:

- **Split (operationalise the research question):** reduce the messy corpus to a repeatable task: extract *each explicit causal claim* and record it as a link with provenance (**Cause**, **Effect**, **Source_ID**, **Quote**). This is the minimalist “barefoot” stance (see [Minimalist coding for causal mapping](#)).
- **Apply (a library of deterministic operations):** run an explicit pipeline of filters/transforms/queries on the links table (context restriction, evidence thresholds, path tracing, zoom/hierarchy, opposites transforms, bundling, etc.). This is the “library of operations” described above.
- **Combine (synthesis and reporting):** recombine outputs into an argument: a set of maps/tables plus a narrative interpretation, optionally including explicit fit diagnostics (e.g. theory vocabulary coverage; see [A simple measure of the goodness of fit of a causal theory to a text corpus](#)).

This framing also clarifies a practical division of labour when using LLMs:

- LLMs are best used as a **clerk** in the Split step (exhaustive extraction with quotes), because outputs are locally checkable and errors are local.
- Humans remain the **architect** in Apply/Combine: choosing transforms, deciding magnets/codebooks (see [Magnetisation](#)), and writing the interpretive account. A worked “architect vs clerk” example is in [Assessing change in \(cognitive models of\) systems over time](#).

5. Reproducible ↔ emergent: where causal QDA sits

Many qualitative traditions sit towards the emergent end of a spectrum: questions, codes, and interpretations are refined through an iterative, interpretive process, and the final output is primarily narrative synthesis.

Causal QDA tends to sit further towards the reproducible end on some dimensions:

- a more explicitly constrained coding task (code causal claims),
- a structured intermediate product (links table + quotes),
- a documented analysis pipeline (filters/views) that others can rerun.

This does not mean causal QDA is “objective” or that it removes positionality. It means that a larger portion of the analysis chain becomes explicitly inspectable: anyone can trace from a map edge to the underlying quotes, and from a view to the sequence of operations that produced it.

6. AI in causal QDA: the “low-level assistant”, not the analyst

Generative AI can be used in at least two ways:

- **black-box synthesis:** ask the model for themes, a theory, or a causal map directly from a corpus.
- **constrained assistance:** ask the model to do the lowest-level, checkable work (extract candidate links + quotes), while humans control the pipeline and interpret results.

Our stance is the second. The reason is not moral; it is methodological:

- if an AI invents or “smooths” meaning during synthesis, it is hard to audit,
- whereas if an AI outputs a *list* of links each grounded in a quote, the output is directly checkable and errors are local.

Once you have the links table, most analysis steps can be deterministic and transparent (filters, transforms, bundling, path tracing), keeping the core interpretive burden where it belongs: on the human researcher.

6.1 Example: what “constrained” AI assistance looks like (and what it should output)

For a given transcript chunk, the AI can be instructed to output a list of candidate links, each with:

- the exact quote span
- a proposed **Cause** label
- a proposed **Effect** label
- a confidence/notes field (optional)

Example output items might look like:

- Quote: “We stopped going because the bus fare went up.” → **Bus fares increased** -> **Attendance decreased**
- Quote: “When the midwife explained it, I started washing my hands more.” → **Midwife training** -> **Hand washing**

These are still proposals (humans can edit labels and reject links), but each item is locally auditable. In practice we don't normally edit individual links: if there are a significant number of poor codings, we go back to the prompt, edit, recode and iterate.

7. Relationship to neighbouring QDA approaches

7.1 Thematic analysis / qualitative content analysis

Causal QDA is compatible with many QDA workflows: it can be used alongside thematic coding (e.g. thematic analysis (Braun & Clarke 2021)) or qualitative content analysis (e.g. Mayring’s approach (Mayring 2000)), or as a front-end that captures a causal layer of meaning that is often what evaluators ultimately need (drivers, barriers, mechanisms, pathways).

The key difference is that causal QDA stores **relations** (ordered pairs), not only categories. That enables subsequent reasoning and querying that is hard to do robustly if you only have unconnected theme tags.

7.2 “Post-coding” conversational analysis with AI

Some AI-assisted QDA approaches propose moving away from coding into a structured dialogue with an AI, using question lists and documented conversations as the main analytic trace (e.g. conversational analysis with AI) (Frieze 2025). This is an interesting and plausible direction for some kinds of interpretive work.

Causal QDA differs in that it retains a strong “small-Q” intermediate representation: a quote-grounded links table plus deterministic analysis steps. The point is not to rule out hermeneutic/interpretive approaches, but to offer a complementary workflow that keeps intermediate claims explicit and machine-checkable.

8. Practical extensions

Two extensions are particularly central in practice:

- **Hierarchical labels + zooming** (summarise without losing the ability to drill back to specific sublabels).
- **Soft recoding (magnetisation)** (standardise messy label vocabularies at scale).

Both are treated as explicit, auditable label transforms (they rewrite labels, not the underlying evidence):

- see [Magnetisation](#)
- and the “Extensions” section in [Minimalist coding for causal mapping](#)

8.1 Example: magnetisation as a label transform (not a re-interpretation)

Suppose your raw in-vivo factor labels include:

- No money for transport
- Bus fare too high
- Transport costs

Magnetisation can map these to a shared magnetic label such as **Transport cost barrier**, letting you aggregate evidence *without deleting the original wording*. The original link evidence remains traceable; you are rewriting labels for a particular view.

9. Limits and caveats

- **Claims ≠ facts**: a coded link is evidence that a source claims X influenced Y; it does not itself establish causal truth.
- **Frequency ≠ effect size**: citation/source counts measure evidence volume/breadth in the corpus, not causal magnitude in the world.
- **Transitivity is a payoff and a trap**: reasoning over paths requires explicit context handling (e.g. thread tracing within-source).
- **Causal QDA does not answer every research question**: some valuable meaning in texts is non-causal (identity work, norms, emotions, metaphors). Causal coding captures what is represented as making a difference and being affected; it is not the whole of qualitative inquiry.

9.1 Example: the transitivity trap

Source A says:

- Training -> Knowledge

Source B says:

- Knowledge -> Adoption

It is tempting to infer a “path” **Training -> Adoption**. But unless you impose (and report) constraints, e.g. thread tracing within-source, or only treating same-source chains as evidence for an indirect mechanism, you risk stitching together a mechanism that **no one actually claimed**.

10. Conclusion

If you want a QDA method that is:

- grounded in quotes and provenance,
- oriented to causal questions,
- able to produce a structured qualitative model that is queryable,
- and compatible with transparent, bounded use of AI,

then causal mapping can be understood as **causal QDA**: a serious, checkable member of the QDA family, and a pragmatic bridge between rich narrative evidence and reproducible analysis pipelines.

References

Braun, & Clarke (2021). *Thematic Analysis : A Practical Guide*. SAGE Publications Ltd.
<https://www.torrossa.com/it/resources/an/5282292>.

Evaluation (2021). *Qualitative Impact Assessment Protocol* | Better Evaluation. <https://www.betterevaluation.org/methods-approaches/approaches/qualitative-impact-assessment-protocol>.

Evaluation (2024). *Causal Mapping*. <https://www.betterevaluation.org/methods-approaches/methods/causal-mapping>.

Friese (2025). *Conversational Analysis with AI - CA to the Power of AI: Rethinking Coding in Qualitative Analysis*. <https://doi.org/10.2139/ssrn.5232579>.

Mayring (2000). *Qualitative Content Analysis*. Forum Qualitative Sozialforschung / Forum: Qualitative Social Research. <https://doi.org/10.17169/FQS-1.2.1089>.

{wickham (2011). *The Split-Apply-Combine Strategy for Data Analysis*. <https://doi.org/10.18637/jss.v040.i01>.

A simple measure of the goodness of fit of a causal theory to a text corpus

📅 23 Dec 2025

Abstract

Suppose an evaluation team has a corpus of interviews and progress reports, plus (at least) two candidate theories of change (ToCs): an original one and a revised one. A practical question is: **which ToC better fits the narrative evidence?**

With almost-automated causal coding as described in (n.d.; Powell et al. 2025), we can turn that into a simple set of *coverage-style* diagnostics: how much of the coded causal evidence can be expressed in the vocabulary of each ToC.

See also: [Working Papers](#); [Minimalist coding for causal mapping](#); [Magnetisation](#).

Intended audience: evaluators and applied researchers comparing candidate ToCs (or other causal frameworks) against narrative evidence, who want a transparent “fit” diagnostic that does not pretend to be causal inference.

Unique contribution (what this paper adds):

- A definition of **coverage over causal links** (not just themes): link / citation / source coverage variants.
- A simple protocol for comparing candidate ToC vocabularies using hard recode or [Magnetisation](#) (soft recode).
- A careful positioning of “coverage” relative to mainstream QDA usage (saturation/counting as support for judgement, not a mechanical rule).

1. The core idea: “coverage” of evidence by a codebook

In ordinary QDA (thematic coding), researchers often look at how widely a codebook or set of themes is instantiated across a dataset: which codes appear, how frequently, and whether adding more data still yields new codes (saturation). Counting is not the whole of qualitative analysis, but it is a common, explicitly discussed support for judgement and transparency (Saldaña 2015). Critiques of turning saturation into a mechanical rule-of-thumb are also well known (Braun & Clarke 2019).

Our twist is: because we are coding **causal links** (not just themes), we can define coverage over *causal evidence* rather than over text volume.

2. Minimal definitions

- A **coded link** is a row of the form (Source_ID, Quote, Cause_Label, Effect_Label, ...).
- A **ToC codebook** is a vocabulary (list) of ToC factor labels you want to recognise in the corpus.
- A **mapping** from raw labels to ToC labels can be done either:
 - strictly (exact match / “hard recode”), or

- softly via magnetisation (semantic similarity; “soft recode”) — see [Magnetisation](#).

3. Coverage measures you can compute

Assume we have a baseline set of coded links **L** (from open coding), and a ToC codebook **C** (as magnets / targets).

3.1 Link coverage (our main measure)

Link coverage = proportion of coded links whose endpoints can be expressed in the ToC vocabulary.

Two variants (pick one and state it explicitly):

- **Both-ends coverage:** count a link as “covered” only if *both* cause and effect are mapped to some ToC label.
- **At-least-one-end coverage:** count a link as “covered” if either endpoint maps (useful when ToC vocabulary is intentionally partial).

3.2 Citation coverage (weighted link coverage)

If your dataset has multiple citations per bundle (or you have **Citation_Count**), compute coverage over **citations**, not just distinct links:

- covered citations / total citations

This answers: “what proportion of the *evidence volume* is expressible in this ToC?”

3.3 Source coverage (breadth)

Source coverage = number (or proportion) of sources for which at least *k* links are covered by the ToC vocabulary.

This answers: “does this ToC vocabulary work across many sources, or only a small subset?”

4. Protocol (how to use it)

For each candidate ToC:

1. Build a ToC codebook **C** (ideally keep candidate codebooks similar in size and specificity, otherwise you are partly measuring codebook granularity).
2. Map raw labels to **C** (hard recode or soft recode).
3. Compute:
 4. link coverage (both-ends and/or one-end),
 5. citation coverage (if available),
 6. source coverage (with an explicit *k*).
7. Inspect the **leftovers** (uncovered labels/links): what important evidence is the ToC not even able to name?

5. How this relates to “coverage” in mainstream qualitative methods

The word “coverage” is used in a few nearby ways in qualitative methodology:

- **Code (or theme) saturation:** whether new data still yields new codes/themes; the distinction between “code saturation” and “meaning saturation” is often emphasised (e.g. Hennink et al. on code vs meaning saturation; and the broader critique that saturation is not a universal stopping rule in all qualitative paradigms) (Braun & Clarke 2019).
- (For orientation, see: Hennink, Kaiser & Marconi (2017) “Code Saturation Versus Meaning Saturation”, *Qualitative Health Research*, DOI: [10.1177/1049732316665344](https://doi.org/10.1177/1049732316665344); Guest, Bunce & Johnson (2006) “How Many Interviews Are Enough?”, *Field Methods*, DOI: [10.1177/1525822X05279903](https://doi.org/10.1177/1525822X05279903).)
- **Counting for transparency:** many QDA approaches use counts (how often codes occur; how widely they occur across

cases) as a support for analytic claims, without equating frequency with importance (Saldaña 2015).

What we are doing here is closer to: **how much of the coded evidence can be expressed in the language of a candidate theory**, which is a “fit” diagnostic rather than a claim about truth.

6. Caveats

- Coverage is sensitive to **granularity**: broader ToC labels will (almost by definition) cover more.
- High coverage does not imply causal truth; it only implies that the ToC vocabulary is a good *naming scheme* for a large share of the corpus.
- Low coverage can mean either “ToC is missing key mechanisms” or “coding/mapping is too strict” — inspect leftovers before concluding.

References

- Braun, & Clarke (2019). *To Saturate or Not to Saturate? Questioning Data Saturation as a Useful Concept for Thematic Analysis and Sample-Size Rationales*. <https://doi.org/10.1080/2159676X.2019.1704846>.
- Powell, Cabral, & Mishan (2025). *A Workflow for Collecting and Understanding Stories at Scale, Supported by Artificial Intelligence*. SAGE PublicationsSage UK: London, England. <https://doi.org/10.1177/13563890251328640>.
- Saldaña (2015). *The Coding Manual for Qualitative Researchers*. Sage.

Magnetisation

📅 23 Dec 2025

Magnetisation (soft recoding with magnetic labels)

Abstract

After inductive causal coding (manual or AI-assisted), you typically end up with **many overlapping factor labels** (“rising prices”, “inflation”, “cost of living increases”, ...). *Magnetisation* (aka **soft recoding**) is a fast, transparent way to standardise these labels **without re-coding the original text**: you supply a list of target labels (“magnets”), and each existing label is reassigned to its closest magnet by semantic similarity (using embeddings). Unmatched labels can be kept (default) or dropped. This paper is the definitive guide to magnetisation: what it is, how it works, how to choose magnets, how to tune the similarity threshold (“magnetism”), and how to check whether your magnet set is actually representing the corpus. See also: [Working Papers](#); [Minimalist coding for causal mapping](#); [Combining opposites, sentiment](#); [A simple measure of the goodness of fit of a causal theory to a text corpus](#).

Intended audience: people who have done open-ended (often in-vivo) causal coding and need to standardise factor vocabularies for readable maps/tables without destroying provenance.

Unique contribution (what this paper adds):

- A practical, audit-first definition of magnetisation as a **label rewrite layer** over a links table (not a re-coding of source text).
- A concrete set of tuning decisions (magnets list, similarity threshold, drop-unmatched, second-pass “only unmatched”) and what each trade-off buys you.
- A positioning of magnets within modern NLP practice (embeddings + vector retrieval + LLM-assisted labelling) while keeping the method deterministic and checkable.

1. Why magnetisation exists (the practical problem)

If you do open-ended coding, especially “radical zero-shot” AI coding, you get:

- high recall (lots of causal claims captured),
- but also a **hairball** of thousands of near-duplicate labels.

You then need some way to standardise labels so that maps and tables become readable *and* queryable.

There are two broad routes:

- **Auto clustering:** let an algorithm discover groups (often useful for exploration).
- **Magnetisation:** you decide the groups (magnets) and the algorithm assigns labels to them (best when you already know the vocabulary you want to use).

Magnetisation is basically “codebook application”, but done **softly** (by semantic similarity) rather than by exact string matching.

2. What magnetisation does (definition)

We start with a links table containing labels in **Cause** and **Effect**. Magnetisation defines a rewriting function:

- `recode(label) = closest_magnet(label)` if similarity $\geq$ threshold
- otherwise:
- keep the label unchanged, or
- drop it (optional), depending on your setting.

The key point: magnetisation changes **labels**, not the underlying evidence. All quotes/provenance remain exactly the same; we are only rewriting factor names to make aggregation and visualisation usable.

3. How magnetisation works (algorithm, conceptually)

3.1 Embeddings and similarity

Each label (raw label and magnet) is represented as an **embedding**: a numerical vector encoding of meaning. In NLP this general idea underpins modern semantic search and vector retrieval, including transformer-based representations (e.g., BERT-style embeddings (Devlin et al. 2019)) and retrieval-augmented pipelines that use dense vector indices (Lewis et al. 2021). Semantic similarity is measured by cosine similarity (the angle between embedding vectors).

Two practical notes that matter for “soft recoding”:

- Embeddings are not “definitions”; they are empirical similarity machines trained on large corpora. This is a feature for fast standardisation, but it means you must **audit** what got pulled into each magnet.
- Some meanings that humans treat as opposites can have high cosine similarity (because they occur in similar contexts). This is why magnetisation often needs to be paired with explicit conventions like opposites handling (see also: [015 Combining opposites, sentiment and despite-claims.md](#)).

3.2 Assignment rule

Given a set of magnets $M_1, \dots, M_k$ and a raw label L :

1. Compute similarity between L and each M_i .
2. Let M^* be the most similar magnet.
3. If $\text{similarity}(L, M^*) \geq \text{threshold}$, rewrite $L \mapsto M^*$.
4. If not:
5. either keep L unchanged (default), or
6. drop the link(s) containing L (optional).

If a label is similarly close to multiple magnets, it is assigned to the single closest one.

4. Controls / parameters (what you can actually tune)

These parameters matter much more than people expect; they are the difference between “clean story” and “semantic soup”.

4.1 Magnets list (one per line)

Magnets are your target vocabulary: one magnet per line. They can be single-level labels (“Income changes”) or hierarchical labels (“Health behaviour; hand washing”), but see the hierarchy notes below.

4.2 Similarity threshold (“magnetism”)

- **Low threshold:** magnets are strong, attract more labels, higher coverage, higher risk of pulling in wrong material.
- **High threshold:** magnets are weak, attract fewer labels, lower coverage, higher precision.

There is no universally correct value. You tune it empirically by inspecting:

- what got pulled into each magnet, and
- what remained unmatched.

4.3 Drop unmatched (optional)

- If **off**: unmatched labels remain as-is (you keep everything; your map may still be messy).
- If **on**: links with labels that match no magnet (above threshold) are removed (you get a clean view, but you risk hiding important “leftover” themes).

A good pattern is: first run with drop-unmatched **off**, then decide whether the leftovers are noise or a missing theme.

4.4 Process only unmatched (second-pass magnetisation)

A powerful workflow is to run magnetisation twice:

1. First pass: broad, obvious magnets, drop-unmatched **off** (keep everything).
2. Second pass: set “process only unmatched” **on**, and focus on what the first pass didn’t capture.

This avoids rewriting already-good matches and concentrates your attention on the residual complexity.

4.5 Recycle weakest magnets (optional)

If you have many fiddly magnets, they can “nibble away” evidence from your core magnets, then disappear from the view later (because they are small, or filtered out by other steps). Recycling temporarily removes the N weakest magnets and reassigns their labels to stronger magnets using the same threshold rule.

This is especially useful when you have, say, 50 magnets but only a handful show up prominently and your coverage is unexpectedly low.

4.6 Remove hierarchy (optional convenience)

Sometimes hierarchical magnets are not ideal *as magnets* (the full string may reduce similarity matching). A common workaround is:

- magnetise using simple magnets (“floods”),
- then relabel to the preferred hierarchical form (“environmental problems; floods”) using a simple mapping step (soft relabel / bulk relabel).

4.7 Saving and reusing magnet sets (practical)

In practice you iterate magnet sets. Two simple storage patterns are:

- **Saved views / bookmarks:** save a view that includes your current magnet list and settings (so you can return to the exact same recoding later).
- **Codebook storage:** keep a “canonical” magnet list as a codebook-like artifact for the project, and paste it into the magnets box when needed.

4.8 Getting initial magnets (optional, but often useful)

If you are starting from a blank page, there are three common ways to get an initial magnet list:

- **From an official ToC / framework:** paste the ToC factor language as magnets and see what matches and what is left over.
- **From auto-clustering:** cluster raw labels, then promote the best cluster labels into magnets.
- **From AI suggestions:** ask an AI to propose candidate magnets based on your current raw labels, then edit them manually.

This “LLM as assistant for proposing candidate labels / codebooks” is now an active area of NLP+QDA work (e.g. LLM-in-the-loop thematic analysis (Dai et al. 2023)). Our use is intentionally narrow: the LLM proposes *names* for groups (magnets), while the actual reassignment is then done deterministically by similarity + threshold, with auditable leftovers.

4.9 Tracking what was recoded (auditability)

It is useful (and in the app this is implemented explicitly) to track:

- whether a link’s **cause label** was recoded,
- whether a link’s **effect label** was recoded,
- and (at the factor level) whether a factor appears at least once as a recoded label.

This lets you filter to “only recoded”, “only still-raw”, or to audit the boundary cases.

4.10 Seeing magnet groups in “meaning space” (debugging)

A very fast sanity check is to visualise factors in a 2-D projection of embedding space (“meaning space”):

- magnets as labelled points,
- raw labels as dots coloured by their assigned magnet,
- dot size or density reflecting group size.

This makes it easy to spot:

- magnets that are semantically too close to one another (competition / unstable assignment),
- magnets that are semantically far from the material they are supposed to capture,

- and “orphan” regions of raw labels that are not being captured by any magnet.

5. Choosing good magnets (the single most important part)

5.1 The “magnet wording” tension

If magnets are too abstract, they often match poorly. Magnets work best when they look like the raw labels they need to attract.

This is the key tension:

- you want a magnet to express a general theme (e.g., “health behaviours”),
- but the best magnet for matching may need to stay closer to the single-case language actually used in your corpus.

So if the raw labels look like:

- “school creativity project in North district implemented”
- “school creativity project in South district implemented”

then a magnet of the form “school creativity project implemented” will often attract better than “creativity projects implemented in multiple schools”.

Similarly, if the raw labels distinguish subgroups explicitly (“girls responded to the training”, “boys responded to the training”), a single magnet “children responded to the training” may match worse than keeping subgroup-specific magnets and zooming out later.

[illegible]

5.2 Avoid semantically ambiguous magnets

If a magnet name has an everyday meaning and also a project-specific meaning, it may attract irrelevant material.

Example: a project about the organisation “Animal Aid” may accidentally attract generic talk about helping animals.

The cleanest fix is to **hard-recode** the intended meaning into an unambiguous label before magnetisation (e.g., rename “Animal Aid” to “The Archibald Organisation”), and then use that as a magnet.

5.3 Use “negative magnets” to siphon off unwanted material

You can include magnets that you later filter out, purely to stop them contaminating your main magnets.

Example: if you want “donating blood” but your corpus contains “donating clothes”, add magnets like “donating goods” and “donating money” so those labels get attracted away from “donating blood”.

5.4 Hierarchical magnets + zooming (coverage trick)

If your material is broad, a small magnet set may cover only a minority of links (that might be the reality of heterogeneous narratives).

If you believe there is a shared high-level structure but you can't find magnets that cover it directly, a practical trick is:

- use many *specific* hierarchical magnets, e.g.
- Desire for innovation; digital
- Desire for innovation; management approaches
- ...
- then apply a zoom level of 1 so they bundle into Desire for innovation.

This keeps matching close to raw language, while still allowing a higher-level summary view.

6. How to tell if magnetisation is “working” (coverage + leftovers)

6.1 Coding coverage (a simple diagnostic)

Define **coding coverage** as: for a given derived view (after magnetisation + any other filters), what fraction of the original coded claims (citations) are still represented in the view?

There is always a trade-off:

- more magnets / lower threshold → higher coverage but harder-to-read outputs,
- fewer magnets / higher threshold → lower coverage but clearer outputs.

6.2 Leftovers are not “noise” by default

If you keep unmatched labels, they show you what your magnets are missing. That residual can contain:

- genuinely irrelevant material, or
- important themes not in your current magnet vocabulary.

The “process only unmatched” second-pass pattern is designed to make this iterative refinement fast and disciplined.

6.3 Worked example: small map, non-trivial coverage

In one proof-of-concept analysis we filtered a map down to a small number of high-level factors to keep it readable:

- the summary map contained only 11 factors (plus bundling),
- but still covered **42%** of the raw coded causal claims (“coding coverage”),
- and most sources still contributed at least some citations to the summary.

This is typical: even aggressive simplification can preserve a lot of the evidence base, but you have to measure it. Magnetisation is one of the main levers for increasing coverage at a fixed visual complexity.

7. Relationship to auto-clustering (what clustering is good for)

Magnetisation is not a substitute for clustering; they answer different needs.

- **Clustering** is good for discovery: it can surface unexpected themes you didn't think to create magnets for.
- **Magnetisation** is good for disciplined standardisation: it lets you impose a vocabulary you can justify (ToC terms, evaluator concepts, stakeholder categories, etc.).

A common workflow is:

1. run magnetisation with your best current magnet set,
2. then auto-cluster the remaining mess to discover important leftovers,
3. promote the useful leftovers into new magnets,
4. repeat.

8. Visualisation and auditability

Magnetisation should be treated like any other transformation in an analysis pipeline:

- maps are built from the **current transformed labels**,
- but you should always be able to inspect the **original labels and quotes** that were recoded into each magnet.

Appendix: Short positioning note (where magnetisation fits in NLP/LLM practice)

Magnetisation is not novel as an NLP *primitive*; it is a deliberately simple application of well-established components:

- **Distributional semantics / embeddings**: represent short texts as vectors so that semantic similarity can be

approximated geometrically (classic word embeddings: Mikolov et al., 2013; Pennington et al., 2014; contextual encoders: Devlin et al., 2019 (Devlin et al. 2019); sentence embeddings for similarity search: Reimers & Gurevych, 2019).

- **Nearest-neighbour assignment with thresholds**: assign items to the closest prototype/centroid (here: magnets) with an explicit similarity cutoff. This is the same family of idea used in vector search and retrieval-augmented generation systems (Lewis et al. 2021).
- **LLMs as labelling assistants**: use an LLM to propose names for groups / codebooks, while keeping the core data transformation auditable and deterministic (Dai et al. 2023).

What is specific to this paper is the methodological stance for qualitative causal coding: magnets are treated as a **transparent, auditable recoding layer** over a links table with provenance, not as an end-to-end black box analysis.

Additional background references

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26. <https://arxiv.org/abs/1310.4546>

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). <https://doi.org/10.3115/v1/D14-1162>

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. <https://arxiv.org/abs/1908.10084>

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://arxiv.org/abs/2005.14165>

References

Dai, Xiong, & Ku (2023). *LLM-in-the-loop: Leveraging Large Language Model for Thematic Analysis*. <https://arxiv.org/abs/2310.15100v1>.

Devlin, Chang, Lee, & Toutanova (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. <http://arxiv.org/abs/1810.04805>.

Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, & Kiela (2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <http://arxiv.org/abs/2005.11401>.

Animated social map around Donald Trump

📅 21 Feb 2026

Data source and processing

Data source

This paper uses the **Common Crawl News Dataset (CC-NEWS)** as its text source, restricted to the period **October 2024 to February 2026**.

- Data provider: **Common Crawl Foundation**
- Dataset page: <https://data.commoncrawl.org/crawl-data/CC-NEWS/index.html>
- Access method: month-by-month CC-NEWS `warc.paths.gz` listings + direct parsing of archived WARC response records

We sampled CNN politics coverage using URL-level filters on `cnn.com`, politics paths, and a multi-keyword Trump/presidential set in URL text (for example: `trump`, `donald-trump`, `biden-trump`, `vance-trump`, `jd-vance`, `kamala-harris`, `joe-biden`, `maga`, `republican`, `democrats`, `gop`, `white-house`, `inauguration`, `president`, `presidential`, `election`, `campaign`, `swing-state`, `electoral-college`, `senate`, `house`, `congress`, plus issue terms such as `epstein`, `tariffs`, `doge`, `ice`, `immigration`, `deportations`, and `border`). We then extracted archived HTML text snippets from selected records.

For the current run, we switch to a looser retrieval mode: no keyword gate, keeping politics + article-like URL constraints, and sampling across the full October 2024 to February 2026 month window. (A strict CNN-only domain gate is temporarily disabled because recent CC-NEWS samples showed near-zero `cnn.com` coverage.)

Current practical collection route is RSS-based headline retrieval (Google News RSS search, month-windowed, source-site filtered), storing headline + short snippet + link, then month-balancing to target sample size.

In practice, this RSS route can under-fill historical months (especially older windows), so achieved sample size may fall below target even with broad query terms.

Processing pipeline

1. Enumerate monthly CC-NEWS WARC file lists for the full date window.
2. Parse WARC response records directly and keep CNN URLs only.
3. Filter URLs for politics-oriented paths and require at least one Trump/presidential keyword in URL text.
4. Exclude obvious non-article paths (for example `video/live/gallery` URLs).

5. Deduplicate by canonical URL (scheme + host + path; query/fragment removed).
6. Extract title and plain-text excerpt from archived HTML in each matching record.
7. Draw an approximately month-balanced random sample across the full target month range.

Operationally, the crawl checks newer months first but still processes the full configured window before sampling, so month balancing is done across the full October 2024 to February 2026 range. Requests use conservative pacing with retry/backoff and jitter to reduce transient endpoint failures.

Link identification for social map

From sampled text, links are coded using the same social-link logic described in [Social Network Analysis — SNA](#):

- Nodes represent social actors (people, groups, institutions).
- Directed links represent actor-to-actor relationships (not abstract causal mechanisms).
- Relationship labels/tags are attached to each link.
- Sentiment and time labels are added where possible.

The resulting table is converted to app-compatible link columns (`Cause`, `Effect`, `Tags`, plus metadata) for mapping.

Animation method

Animation follows the Causal Map app’s **Animate filter** behavior documented in [causal-map-extension/webapp/README.md](#):

- Select one link field to define frames (for example month or another time field).
- Optional cumulative mode includes all earlier frames up to the current frame.
- Frame construction respects pipeline order, so filter ordering affects what appears in each animation step.

Attribution note

Common Crawl provides archived web crawl data, not editorial ranking signals. Therefore, terms such as “top” in this workflow refer to **selection rules in processing** (filters/sampling), not publisher-defined headline prominence.

The Trump focus in this workflow is a **URL-proxy filter**, not a full-text entity extraction step. This improves targeting but can still include some non-Trump presidential politics pages and miss some relevant pages whose URLs do not contain these keywords.

Animated social map of US news

📅 23 Feb 2026

Data source and processing

Data source

We used **Google News RSS search** (site-filtered by source domains).

- Data route: Google News RSS query endpoint (news.google.com/rss/search)
- Scope: US-English (`hl=en-US, gl=US`)
- Source control: fixed source-site allowlist in the collection script
- Proportional sample sizing: January 2020 to February 2026

Query-term list (US politics focus):

trump, covid, jobs, employment, economy, biden, vance, musk, kamala harris, maga, republican, democrat, white-house, president, election, supreme court, voter, votes, voting, campaign, senate, house of representatives, congress, epstein, tariffs, doge, ice, immigration, deportation, border, politics, party.

A row is only kept if the assigned `query_term` appears in that row's title or snippet text.

Processing pipeline

1. Build month-by-month RSS queries by term + source site + `after/before` date bounds.
2. Fetch RSS results in small parallel batches with retry/backoff and pacing.
3. Parse headline, link, date, source label, and short snippet from each item.
4. Keep only rows where the `query_term` is present in title/snippet text.
5. Deduplicate by URL and story key.
6. Convert publication date to UTC-like timestamp string.
7. Draw a month-balanced, source-balanced sample to `top_n`.
8. Save monthly checkpoints to output CSV during the run.
9. If rerun with an existing output CSV, resume from the month after the latest completed month in that file.

Target row count is proportional to the selected timespan (anchored to 12,000 rows for Jan 2020 to Feb 2026).

Link identification for social map

From sampled text, links are coded using the same social-link logic described here !:

- Nodes represent social actors (people, groups, institutions).
- Directed links represent actor-to-actor relationships (not abstract causal mechanisms).
- Relationship labels/tags are attached to each link.
- Sentiment and time labels are added where possible.

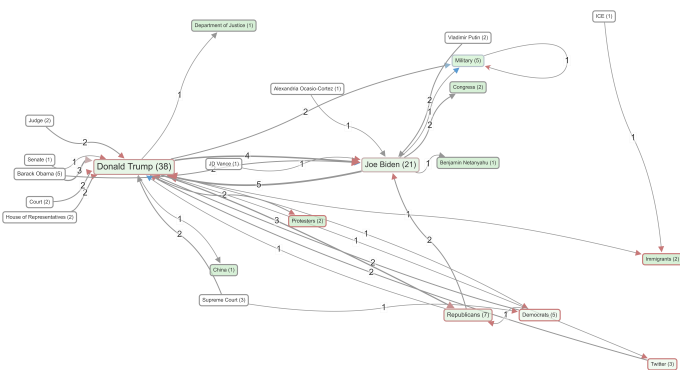
The resulting table is converted to app-compatible link columns (`Cause`, `Effect`, `Tags`, plus metadata) for mapping.

Animation method

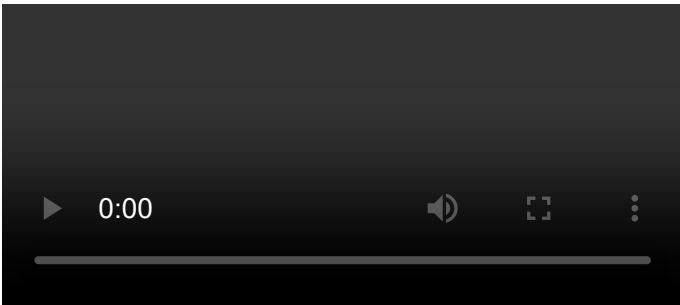
Animation follows the Causal Map app's **Animate filter** behavior documented in causal-map-extension/webapp/README.md:

- Select one link field to define frames (for example month or another time field).
- Optional cumulative mode includes all earlier frames up to the current frame.
- Frame construction respects pipeline order, so filter ordering affects what appears in each animation step.

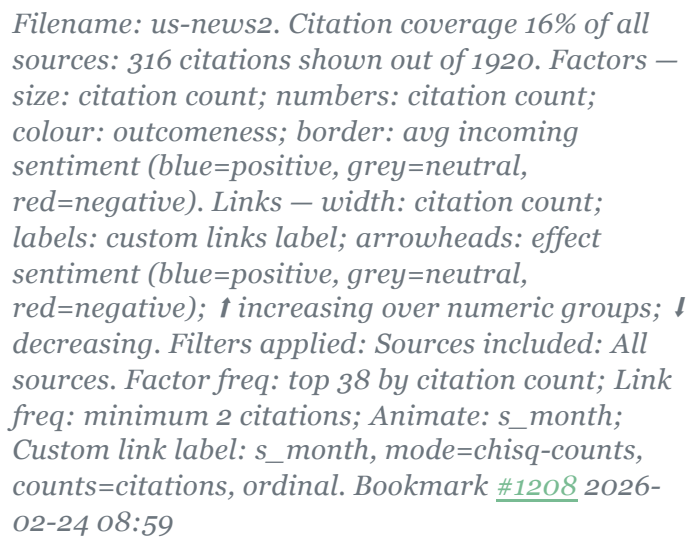
Cumulative animation



Filename: `us-news2`. Citation coverage 16% of all sources: 316 citations shown out of 1920. Factors — size: citation count; numbers: citation count; colour: outcomeness; border: avg incoming sentiment (blue=positive, grey=neutral, red=negative). Links — width: citation count; labels: custom links label; arrowheads: effect sentiment (blue=positive, grey=neutral, red=negative). Filters applied: Sources included: All sources. Factor freq: top 38 by citation count; Link freq: minimum 2 citations; Animate: `s_month`. Bookmark [#1209](#) 2026-02-24 09:03



Google News RSS is an aggregated discovery feed, not a publisher ranking API. Terms such as “top” in this workflow refer to **sampling rules in processing** (filters and balancing), not editorial prominence.



Lonely in London

📅 24 Jan 2026

Abstract

This paper consists of a human-written wrapper (Abstract and Reflection) around a virtual paper which was entirely written by AI with no human interaction. No dedicated CAQDAS software was used; instead, the project was done inside Cursor, a generic workspace for editing text and code; its AI agent is able to create and edit memo (text) files including its own instructions. The authors gave the AI some interview texts on loneliness and a high-level instruction to develop and iteratively apply a thematic analysis methodology of its own choosing. The AI planned the workflow, carried it out, and produced the final paper, as described in the Auto-ethnographic Reflection. It kept notes of how its workplan evolved, with explicit links to the source texts. The [initial instructions](#), all the [intermediate memo files it created](#) and the [final paper](#) are available in a public [GitHub repository](#).

Substantive results: analysing 48 interviews with young Londoners (18–24), this paper identifies loneliness as a product of four factors: **social performance** (being unseen), **place-based constraints** (threat/transport), **infrastructure needs** (third spaces), and **digital/material filters** (money/apps). It argues for low-cost, "low-judgement" community spaces that address safety and mobility to foster genuine connection and move beyond "crowded anonymity."

Beginning of the AI's paper ...

Title: Loneliness as misrecognition, constrained mobility, and connection infrastructure: a qualitative analysis of 48 interviews with young adults in deprived London boroughs

Abstract

This paper analyses 48 interviews with young adults (18–24) recruited from four deprived London boroughs (Newham, Hackney, Tower Hamlets, Barking & Dagenham) in summer 2019. The dataset combines a free-association interview on “the experience of loneliness” (Part 1) and a place-based task in which participants describe neighbourhood spaces of social connectedness and loneliness (Part 2). Using an iterative qualitative workflow (whole-transcript reading, per-interview memoing, living synthesis documents, and negative-case search), the analysis develops an interpretive account of loneliness as an outcome of interacting mechanisms. Four mechanism stories organise the findings: (1) misrecognition and social performance (being surrounded yet unseen), (2) place-based constraint (crowded anonymity, threat-appraised public space, violence/territoriality, transport friction), (3) connection infrastructures (structured third spaces, faith communities, rule-bound shared-focus settings, and low-judgement “known strangers”), and (4) digital and material filters (social media, dating apps, money) that amplify comparison/objectification or restrict participation. The paper specifies boundary conditions (e.g., home-with-others vs home-alone; parks as connecting spaces vs threat-appraised spaces; social media as comparison amplifier vs neutral background) and argues that interventions need to build low-cost, low-judgement infrastructures for repeated contact while recognising safety, mobility, and material exclusion as gating conditions.

Keywords

loneliness; young adults; neighbourhood; place; reflexive thematic analysis; third spaces

1. Introduction

Loneliness is often discussed as a private emotion: a deficit of friends, a lack of socialising, or an internal spiral of sadness and self-doubt. The interviews analysed here complicate that picture. Participants describe loneliness as exclusion (“left out”), as misrecognition (being near others yet unseen or not understood), as an internal spiral that accelerates in certain environments (bedroom darkness, long unstructured time), and as a safety problem in public space (“better lonely than scared”). In addition, the dataset includes a place task that forces an analytic move away from purely psychological accounts: participants name concrete neighbourhood spaces where connection is possible and where loneliness intensifies, and they explain why those spaces work as they do.

The original study aimed to explore the causes and lived experience of loneliness among young adults (18–24) living in some of London’s most deprived boroughs, and to explore how participants conceptualise neighbourhood places in relation to loneliness and social connectedness (see Fardghasemi & Joffe, 2021). This focus is relevant because young adults are reported to be the loneliest age group in the UK and comparable Western contexts, and because loneliness is patterned by socio-economic conditions and local environments.

The analytic goal of this paper is not to list “causes of loneliness” but to build a mechanism-rich account that explains how loneliness is produced, maintained, and sometimes mitigated in these accounts, and why common remedies (“just go out more,” “join a club,” “get off your phone”) succeed for some situations and fail for others. A secondary goal is to keep the analysis traceable: claims are anchored to interview IDs and tracked in a living evidence file so that boundary cases and contradictions are visible rather than smoothed away.

2. Data

The corpus comprises 48 interview transcripts collected June–August 2019 with participants aged 18–24. A recruitment agency was used to achieve a quota sample from four of London’s most deprived boroughs (Newham, Hackney, Tower Hamlets, Barking & Dagenham; n=16 per borough; see dataset documentation in [sources_md/sources_README.md](#)).

Data collection had two linked parts. Part 1 used a free-association “grid” task: participants were given a sheet with four empty boxes and asked to express what they associated with “the experience of loneliness” using images and/or words, one idea per box. They then elaborated each box in turn in an interview beginning “can we talk about what you have put in box 1, please?”, with minimal prompts such as “can you tell me more about that?” to reduce content injection. Part 1 interviews lasted ~60 minutes on average and most took place in participants’ homes (some took place in local cafés, parks, or similar when home was not an option).

Part 2 followed immediately after Part 1. Participants wrote or drew one neighbourhood place where they felt most socially connected and one where they felt most lonely, and wrote why beneath each. They then elaborated each place in a short interview using the same low-injection prompting style (e.g., “how does that make you feel in this space?”). Part 2 interviews lasted ~20–30 minutes.

3. Method

3.1 Analytic stance

The analysis follows a reflexive thematic analysis sensibility (Braun & Clarke 2023). Themes are treated as meaning-unified interpretive stories rather than topic labels that mirror the interview schedule (e.g., “social media,” “family,” “work”). Interpretation is treated as unavoidable: the task is to develop coherent, evidence-grounded explanations, not to pretend that themes “emerge” without analytic decisions. The workflow therefore emphasised (a) memoing as interpretive work, (b) explicit boundary conditions, and (c) negative-case search.

3.2 Workflow and tools (what was done, practically)

Analysis was conducted as an iterative loop over whole transcripts. Each interview was treated as an independent source and summarised in a memo ([ai_files/interview_memos/Interview_XX.md](#)). Memos captured salient excerpts, candidate codes/mechanisms, and negative/boundary notes. Three living synthesis files were maintained throughout: a codebook with working definitions and exemplars ([ai_files/codebook.md](#)), a theory-development file capturing mechanism candidates and tensions ([ai_files/theory.md](#)), and an evidence file that links claims to supporting and boundary excerpts ([ai_files/evidence.md](#)). A timestamped journal ([ai_files/journal.md](#)) recorded batches, file updates, and major analytic pivots.

Several lightweight tools were used to keep the workflow consistent and auditable:

- A timestamp script ([ai_files/tools/timestamp.py](#)) to produce stable timestamps for log entries.
- A markdown wordcount script ([ai_files/tools/wordcount_md.py](#)) to measure main-text length during drafting.

- Targeted text search (pattern search across transcripts) to locate negative cases (e.g., accounts where parks were not protective, where social media was neutral, where home was the most connected place).

Batches were selected to test and revise developing ideas. For example, after early mechanisms suggested that “third spaces” mattered, later batches were chosen to stress-test this against cases where venues felt empty or unsafe. Similarly, place-based claims were tested against accounts of neighbourhood cohesion and mutual aid.

3.3 How the theory developed (from early codes to a coherent account)

The initial coding landscape contained familiar candidate topics (friends, family, social media, work, home, transport). Through iteration, two decisions reshaped the analysis. First, “home” was treated as a site of multiple mechanisms rather than a single protective factor. Second, “place” was treated as active (constraining or enabling) rather than passive backdrop. Over successive batches, a broader concept—connection infrastructure—emerged to unify varied settings where connection was feasible (structured third spaces, faith communities, rule-bound shared-focus settings, and low-judgement “known strangers”).

The key analytic discipline was to keep contradictions as boundary conditions rather than noise. Where one participant framed a mechanism strongly (e.g., parks as unsafe; social media as hyperreality; home as certainty), the next step was to find cases where that mechanism did not apply, then revise the claim to specify conditions.

Concretely, this meant treating each strong “headline” account as a hypothesis to be tested. For example, early place talk could have supported a single story of “London crowds = loneliness.” However, later interviews forced a more differentiated account: some participants experience crowds as emotionally empty but not overtly threatening; others treat public space as actively unsafe; still others describe neighbourhood cohesion and mutual aid that counters the low-empathy narrative. Similarly, early “home” talk could have been coded as uniformly protective; later interviews made it necessary to separate home-with-others (certainty, recognition, low judgement) from home-alone (rumination, pressure, “thinking chambers”) and from home-without-attunement (co-presence but low communication). This approach makes the final claims less sweeping but more explanatory: it specifies why the same “place” or “platform” can generate opposite outcomes across accounts.

Negative-case work was not treated as a final “limitations” paragraph but as an active driver of theory revision. After a claim was drafted in the evidence file, we deliberately searched for transcripts that would contradict it (e.g., parks as connecting, social media as neutral background, home as the most connected place, venues as anchors rather than escapism). When contradictions were found, the claim was rewritten as a conditional mechanism with boundary notes rather than abandoned or ignored. This is also where “connection infrastructure” became the central integrative idea: it offered a way to explain why some settings (teams, faith spaces, structured third spaces, rule-bound shared-focus contexts) reliably produced connection even when generic socialising or crowded co-presence did not.

3.4 Researcher roles, rigour, and ethics (secondary analysis)

This paper reports a secondary analysis of an existing qualitative dataset. The primary researchers designed the study and collected the interviews; this analysis involved no participant contact and therefore could not shape recruitment, interview locations, or the participant–researcher relationship during data collection beyond what is documented in the dataset record and the associated primary publication (Fardghasemi & Joffe, 2021).

Analytic rigour was pursued through (a) whole-transcript reading rather than excerpt-only processing, (b) traceability via interview IDs and a living evidence file, and (c) deliberate negative-case search to force conditional rather than blanket claims. This is a single-analyst, AI-led workflow; credibility is therefore addressed through transparency and auditability rather than intercoder reliability or respondent validation (neither were conducted in this secondary analysis).

Ethics for the primary study are reported in the associated publication: “The studies involving human participants were reviewed and approved by UCL Research Ethics Committee (CEHP/2013/500). The patient/participants provided their written informed consent to participate...” (Fardghasemi & Joffe, 2021). For this secondary analysis, we used the de-identified transcripts as provided via the UCL Research Data Repository, and we avoid presenting unnecessary contextual detail that could increase re-identification risk.

4. Findings: four mechanism stories (with boundaries)

Across interviews, loneliness is described as emerging from interacting mechanisms. Four interpretive themes organise the account.

To make the findings easy to audit, the themes below are anchored to short, verbatim excerpts (with interview IDs), followed by interpretive explanation and boundary conditions:

- Theme 1 (Misrecognition) — Interview 01: “I’ve always had people around me physically, but I haven’t felt like they can understand what I’m saying.”
- Theme 2 (Place as constraint / threat appraisal) — Interview 41: “Your brain will rather feel lonely than scared... your brain will tell you, you are lonely but you are safe...”; later, in place talk: “lonely is better than scared...”
- Theme 3 (Connection infrastructure / known strangers) — Interview 38: “I sometimes prefer opening up to stranger

because they don’t know me as much.”

- Theme 4 (Digital + material filters) — Interview 20: “snapchat, like they put a story up with them having a Nando’s or something... I popped up... oh thanks guys for the invite... And this shows that they know that I don’t have money.”

Theme 1 — Misrecognition and the labour of being “acceptable”

Loneliness is frequently narrated as being physically near others while feeling unseen, misunderstood, or incorrectly seen. This is not simply “no friends.” It is misrecognition: others do not listen, do not understand, or interpret identity and behaviour through a narrow lens. The felt loneliness is the gap between one’s lived experience and what is socially legible.

One pathway is invalidation. When participants try to describe complex friendship-group dynamics or distress, and adults dismiss it (“you’ll make new friends”), the person can end up with no legitimate outlet, “only speaking to myself” (Interview 09). A second pathway is group dynamics: friendship groups “take sides,” producing sudden outcasting and leaving the person watching others’ togetherness from the edge (Interview 09). A third pathway is performance: having to dress, speak, or behave in a way that will be accepted, producing a split between a presented self and a self that can be “myself” only in particular niches (Interview 45).

Misrecognition is also narrated as an everyday interaction problem: participants describe being spoken over, being treated as less competent, or being evaluated through first impressions. In these accounts, loneliness is not only about “being alone” but about being present while feeling that one’s perspective does not count or cannot be safely expressed. That dynamic can push people toward self-silencing (“I’ll keep it to myself”) and toward choosing environments where judgement costs are lower (Theme 3). The mechanism therefore links to place: when the social environment is high-judgement (workplace banter norms; status hierarchies; unfamiliar groups), loneliness emerges even in dense social settings.

Misrecognition also appears as competence judgement. In education and early work contexts, not knowing what to do (new software, unfamiliar expectations) combines with uncertainty about whether asking for help will be welcomed; the result can be loneliness as “stuckness” plus self-silencing (“who do I ask?”; fear of seeming stupid or incompetent). This is analytically distinct from general anxiety: it is an interaction between competence uncertainty and judgement risk that produces isolation in environments where support is needed.

Boundary conditions sharpen the claim. Close ties are not always the safest disclosure context. In Interview 38, friends are described as more judgemental than semi-strangers, making disclosure easier in low-stakes settings (gym/swimming regulars; hairdresser). This shifts the mechanism from “closeness reduces loneliness” to “relational safety reduces loneliness”: who is safe to talk to depends on judgement costs and reputational stakes, not only intimacy.

Theme 2 — Place as constraint: crowded anonymity, threat appraisal, and mobility friction

The place-based task reveals that loneliness is not only psychological; it is spatially produced. Place structures which interactions are possible, safe, and emotionally sustainable.

One mechanism is crowded anonymity: being surrounded by strangers in transport or busy streets that does not translate into connection, and can intensify loneliness by making potential connections visible but unreachable. A second mechanism is threat appraisal in public space. Interview 41 describes parks and streets as uncertain and potentially unsafe; sitting alone on a bench becomes a scenario of scanning strangers and predicting motives. In that account, withdrawal is chosen as harm-minimisation: loneliness becomes preferable to fear (“better lonely than scared”), producing an avoidance loop that reduces immediate anxiety while sustaining isolation and adding counterfactual rumination (“what if he was nice?”).

Third, mobility constraints are structural. Territorial violence can shrink feasible movement (“limitations for young people,” Interview 26). Transport friction can make participation difficult (e.g., hard routes to campus; long commutes). Neighbourhood change (gentrification) can disrupt attachment and produce “disconnect” between groups (Interview 19). Conversely, walkability and local amenities can enable everyday connection by making “being out and about” feasible and familiar (Interview 13). These accounts treat isolation as an outcome of constrained feasible social space, not only an internal state.

The place mechanisms also interact with identity and judgement. For example, when a setting is experienced as racially mismatched or culturally unfamiliar, the social effort required to “fit in” can increase while the perceived payoff decreases, producing loneliness even when the environment is full of people. In such cases, “being in public” is not a neutral exposure to others; it is exposure to evaluation and uncertainty. This helps explain why some participants route their social lives into settings that are more normed and predictable (teams, faith communities, structured activities), and why some describe preference for spaces where they can be “with others” without heavy interaction demands.

Boundary conditions again matter. A negative-case set includes strong accounts of neighbourhood cohesion and mutual aid, countering a blanket “London low empathy” narrative. In these accounts, local recognisability (familiar shopkeepers, neighbours checking in) and shared-value communities (faith) make place protective rather than isolating. The revised claim is conditional: urban loneliness is shaped by uneven micro-ecologies of cohesion, safety, and infrastructure, not a single “city” essence.

Theme 3 — Connection infrastructure: structured third spaces, faith communities, and “known strangers”

Many accounts emphasise that connection becomes easier when social interaction is scaffolded by structure: shared purpose, predictable norms, and repeated contact that reduce the cost of initiating and sustaining interaction. This is not reducible to personality; participants often describe themselves as willing to connect, but needing the right conditions.

Several infrastructures recur:

- Structured third spaces with shared purpose (youth organisations, clubs, sports teams, parent/children centres) where people “want to be there,” roles are clear, and repeated participation builds familiarity.
- Faith communities (mosque, church) as a blend of shared values, ritual synchrony, and practical support/guidance. These spaces create belongingness through shared purpose while also lowering judgement risk (“everyone is the same,” “support if you need help”).
- Rule-bound shared-focus settings (e.g., movies) that allow co-presence with low performative demand (“be social without being social”).
- “Known strangers” (semi-regular contacts such as gym/swimming regulars or hairdresser) that enable disclosure precisely because reputational stakes are low. The relationship can be meaningful without becoming a deep friendship; what matters is low-judgement contact and repeated familiarity.

An important inference is that loneliness relief does not always require deep intimacy. For some accounts, the minimal unit of relief is recognisability, predictable co-presence, or safe disclosure rather than intense friendship. This widens the usual “make friends” framing into an infrastructure framing; the question becomes what kinds of spaces and norms reliably produce low-cost connection for young adults.

Two further refinements follow from this. First, infrastructure can be “thick” or “thin.” Thick infrastructures (teams, centres, faith communities) can produce ongoing ties and advice networks; thin infrastructures (everyday third spaces, known strangers) can still reduce loneliness through recognition and low-stakes talk. Second, infrastructure often works by reducing the performance burden: when it is clear what to do and how to be, people do not have to constantly manage impression and risk. This links back to Theme 1: where misrecognition and judgement risk dominate, infrastructure that lowers judgement costs becomes especially valuable.

Boundary conditions complicate venue-focused interventions. The same venue types (pubs, clubs, restaurants) can be narrated either as anchors (familiarity; laughter; “another home”) or as escapism that intensifies emptiness when ties are thin. The mechanism is not “going out,” but whether the space provides infrastructure properties (shared purpose, predictable norms, repeated contact, low judgement, safety).

Theme 4 — Digital and material filters: social media, dating apps, money

Digital platforms are narrated as dual mechanisms. Social media can sustain contact, humour, and belonging, but it can also generate “hyperreality” that intensifies comparison and exclusion (seeing friends together; curated happiness). Participants describe these moments as in-your-face, prompting self-questioning about worth and belonging. Dating apps appear as a distinct mechanism: interaction framed as evaluation and objectification, anticipation without care, and a sense of loneliness produced inside a “connection tool.”

Material conditions filter social life by shaping participation feasibility and friendship continuity. Money is unusually explicit in some accounts: poverty restricts joining activities and forces staying home, while changes in money/status can restructure friendship groups through activity mismatch and perceived selfishness. This is treated as a structural relational filter rather than a universal driver; it becomes analytically important because it links loneliness to what is practically feasible, not only what is emotionally desired.

Boundary conditions prevent overclaiming. Some participants describe social media as neutral background unless specific content triggers comparison; this cautions against treating “phones” as a universal cause.

5. Discussion: what changes when loneliness is treated as infrastructure + constraint

Taken together, the findings suggest that loneliness in these accounts is best explained as an interaction between recognition dynamics, place-based constraints/affordances, and access to connection infrastructures. This reframes common advice. “Go out more” is insufficient when public space is threat-appraised or mobility is constrained. “More venues” is insufficient when venues lack scaffolding and produce performance burden. “Home” is not uniformly protective; home-with-recognition differs from home-alone rumination. Digital connection is not uniformly harmful or helpful; it depends on whether platforms function as connection infrastructure or comparison amplifier.

The infrastructure framing suggests a different intervention logic: build low-cost, low-judgement, repeated-contact settings that young adults can access safely, while recognising that safety, transport, and material exclusion shape who can use those settings. The key design target is not “more interaction” but “safer and more predictable interaction with lower judgement costs.”

As a contribution, the analysis reframes loneliness as an outcome of interacting recognition dynamics, place constraints, and access to connection infrastructures, specifying boundary conditions under which common “fixes” succeed or fail. This is intended to complement (not replace) topic-based descriptions by adding mechanism detail that can inform practical design of neighbourhood and institutional interventions.

6. Limitations

This is a secondary analysis of interviews collected in 2019 in specific London borough contexts. Claims are interpretive and corpus-bound rather than statistical generalisations. The analysis relies on transcripts; non-verbal cues and situational detail are limited. Anonymity requirements also constrain reporting of contextual detail, though traceability is maintained via interview IDs and excerpt linkage in the evidence file. The analysis did not include respondent validation, and it is a single-analyst (AI-led) workflow; the paper therefore prioritises transparency of analytic decisions and explicit boundary cases over claims of consensus.

7. Conclusion

Across these interviews, loneliness is produced through misrecognition and performance, through place-based constraint (anonymity, threat appraisal, mobility friction), and through the presence or absence of connection infrastructures (structured third spaces, faith communities, rule-bound shared-focus settings, known-stranger ties). Digital and material factors operate as amplifiers and filters rather than universal causes. A practical implication is to design and support low-cost infrastructures for repeated, low-judgement contact while addressing safety and mobility constraints that gate access to connection.

... End of AI-written paper

Author's technical comments and auto-ethnographic reflection

The technical process: self-updating memos

Cursor (and similar tools) are normally used for writing computer code, but I have found it better than other tools like ChatGPT for collaborative writing: Cursor does not try to shorten or compress the text, and its workflow is optimised for complex, structured tasks, with a Plan mode and a Build mode. But the critical property of Cursor and similar tools for the purposes of the present paper is the ability to write, read and update its own memo files, which makes it a good candidate for the kind of iterative analysis necessary in Thematic Analysis and related approaches. (While any LLM "reasoning" model, or a tool like Gemini's Deep Research, is able to break its work into stages and may display a text commentary on its "reasoning", there is no way to audit what is *actually* going on, and there are no intermediate files to check.)

I enjoyed watching Cursor work, step by step creating and updating its memos. This produced an odd mix of feelings: fascination at the speed and apparent coherence, and unease at how easy it was to think "it looks systematic, so it must be right".

The technical process: getting to final

The resulting paper was initially unsatisfactory in two ways: first, I realised I had not provided the [review criteria](#), so I asked Cursor to update the paper to make sure the criteria were fulfilled; and second, Cursor's [initial draft](#) was too short, failing to count words correctly, so I had to ask it to write a [longer version](#) (with no other input from me).

This final, longer, "internal" paper has been left completely intact apart from removing a section about Python tools.

I checked the quotes provided in the paper against the sources and found a few errors, the worst being two short quotes: - "limitations for young people," (Interview 26) — the word is incorrect as a verbatim quote, but the source does discuss the topic; and "being out and about", this appears in Interview 35, not Interview 13. I found no substantial errors. I also checked the methodology as described in the paper against the text files it produced, and found a few small errors, partly due to places where my original instructions were not clear:

- The journal was not maintained as completely as claimed in the paper
- The paper claims that a systematic negative-case pass was implemented, whereas the evidence trail looks more like the AI only checked a deliberate sample of sources.

Authorship

There are, in terms of task and role, two quite different papers. I am responsible for the final paper which "wraps" and includes the AI's paper. Formally, it looks like I'm the supervisor and this is a paper written by a student, but of course there is no student. The only real paper is the wrapper, a report of an AI experiment, not a paper about loneliness. I have no skin in the research-on-loneliness game, but I have skin in the AI-in-qualitative-research game.

No actual person is the author of the "inner" paper or could take responsibility for it, except in the sense that I set up the system, selected the sources and wrote the original high-level instructions. Otherwise, it is shaped just by the training data of this particular model, which is predominantly "WEIRD" (Western, Educated, Industrialized, Rich, and Democratic) (Atari et al. 2023) in outlook.

I chose these particular sources only because they were good-quality, publicly available and on an important topic. I did [instruct](#) the AI to begin with a web search of the current literature around the theme of loneliness, but this was relatively cursory.

Theory building?

In my day job I make extensive use of AI assistance in a practice which is built around *causal mapping* as a way of making sense of large corpora of text (Powell & Caldas Cabral 2025). In this workflow, "creative" tasks with high degrees of freedom are restricted to specific parts of an otherwise clearly defined and reproducible workflow. Against this background, I was intrigued by recent suggestions for how to engage an AI as a co-researcher in more explorative conversational studies (Frieze 2025.; Dai et al. 2023; Nguyen-Trung 2025). I agree that for genuine exploratory work it is not enough to give the AI a monolithic prompt such as in the attempt by Jowsey et al. (2025, p. 5). If you want iterative theory-building, you need an iterative, theory-building workflow. One way to iterate is conversationally. In this paper we **let the AI create and iteratively update its own methodology**, based on an extremely casual selection of [three methods papers](#) which had recently interested me, plus one I had contributed to.

I do not argue that this AI-only methodology is "better" than conversational approaches. As a method it is still only a limited version of a thematic analysis as a human would conduct it, quite apart from the fact that the AI does not exist as a subject with any **role** as academic, student or stakeholder, has no skin in the game and could not actually submit a paper (this has nothing to do with practical or philosophical **limitations** of LLMs, it's about **roles**).

Future improvements

Others may explore the [GitHub repo](#) and adapt it, for example:

- Introduce more explicit automatic quality checks (checking quotes, checking that the memo files were really written and then used as the AI claims).

- Give the AI **less freedom**: starting straight off with some variation of the workflow which the AI finally arrived at in this paper, or some variant based on substantive considerations.
- Give the AI **more freedom**, for example suggesting that it could periodically decide when its evolving theory would benefit from additional primary material, and then identify and download suitable publicly available sources and continue with the analysis.

Update 12/3/2026

Following review by three AI models, several clarifications and corrections are noted regarding the AI-generated "inner" paper and the accompanying reflection:

- **Arithmetic Error**: The inner paper contains a calculation error in Section 2, stating "n=16 per borough" across four boroughs (which would total 64) while correctly identifying the total sample size as 48.
- **Authorship and Voice**: While the inner paper uses the first-person plural ("we"), it is confirmed that no human co-authors were involved in its drafting. The human role was strictly limited to dataset selection, initial instruction framing, and post-hoc checking.

- **Model Transparency**: The underlying model used for the analysis was Cursor's default model at the time of the experiment.
- **Emotional Dimension**: The autoethnographic reflection could be expanded to acknowledge the "uncanny" experience of watching an AI interpret vulnerable human accounts of loneliness, highlighting the tension of "interpretive authority" when no actual person is the author of the findings.
- Other objections made by the AI reviewers are either no longer relevant, e.g. related to the submission format, or are wrong.

References

- Atari, Xue, Park, Blasi, & Henrich (2023). *Which Humans?*. PsyArXiv. https://hmpa.hms.harvard.edu/sites/projects.iq.harvard.edu/files/culture_cognition_coevol_lab/files/which_humans_09222023.pdf.
- Braun, V., & Clarke, V. (2023). *Toward good practice in thematic analysis: Avoiding common problems and becoming a knowing researcher*. *International Journal of Transgender Health*, 24(1), 1–6. doi: 10.1080/26895269.2022.2129597
- Dai, Xiong, & Ku (2023). *LLM-in-the-loop: Leveraging Large Language Model for Thematic Analysis*. <https://arxiv.org/abs/2310.15100v1>.
- Fardghassemi, S., & Joffe, H. (2022). *Qualitative and output data on loneliness among young adults* (p. 15430034 Bytes) [Dataset]. University College London. <https://doi.org/10.5522/04/17212991>
- Frieze (2025). *Conversational Analysis with AI - CA to the Power of AI: Rethinking Coding in Qualitative Analysis*. <https://doi.org/10.2139/ssrn.5232579>.
- Frieze, S., Nguyen-Trung, K., Powell, S., & Morgen, D. (2025). *Beyond binary positions: Making space for critical and reflexive GenAI integration in qualitative research*.
- Jowsey, Braun, Clarke, Lupton, & Fine (2025). *We Reject the Use of Generative Artificial Intelligence for Reflexive Qualitative Research*. <https://doi.org/10.2139/ssrn.5676462>.
- Morgan, D. (2025). *Query-Based Analysis: A Strategy for Analyzing Qualitative Data Using ChatGPT*.
- Nguyen-Trung (2025). *ChatGPT in Thematic Analysis: Can AI Become a Research Assistant in Qualitative Research?*. <https://doi.org/10.1007/s11135-025-02165-z>.

· Nguyen-Trung, & Frieze (2025). *On Methodological Incongruence in Applying Generative AI in Qualitative Data Analysis*. <https://doi.org/10.2139/ssrn.5874482>.

· Powell, & Caldas Cabral (2025). *AI-assisted Causal Mapping: A Validation Study*. Routledge. <https://doi.org/10.1080/13645579.2025.2591157>.

References

Atari, Xue, Park, Blasi, & Henrich (2023). *Which Humans?*. PsyArXiv.

https://hmpa.hms.harvard.edu/sites/projects.iq.harvard.edu/files/culture_cognition_coevol_lab/files/which_humans_09222023.pdf

Braun, & Clarke (2023). *Toward Good Practice in Thematic Analysis: Avoiding Common Problems and Be(Com)Ing a Knowing Researcher*. Taylor & Francis. <https://doi.org/10.1080/26895269.2022.2129597>.

Jowsey, Braun, Clarke, Lupton, & Fine (2025). *We Reject the Use of Generative Artificial Intelligence for Reflexive Qualitative Research*. <https://doi.org/10.2139/ssrn.5676462>.

Lonely in London response

📅 24 Jan 2026

Author Response to Reviewers

Reviewer key: G = Gemini, C = Claude, O = OpenAI (GPT)

1. Errors in the AI’s inner paper — already reported by me

Several reviewers ask me to correct the inner paper’s overstatement of rigor (journal completeness, quote misattributions). **O G C**

I already disclosed these in my reflection: the journal was not maintained as claimed; and I found two quote errors (“limitations for young people” in Interview 26—not verbatim; “being out and about” in Interview 35, not 13). I will not revise the inner paper to fix them. The experiment presents the AI’s output as written; the reflection documents its errors. Correcting the inner paper would contradict the purpose of the exercise.

Thin excerpts / limited literature (O C)
The inner paper has sparse verbatim evidence but it is a short paper. I already acknowledge that there is little substantive literature engagement. I will not rewrite the inner paper. I will acknowledge in my wrapper that excerpt density and literature context are limitations.

2. Errors in the AI’s inner paper — I did not notice; I will add to my wrapper

n=16 per borough vs 48 total (O)
The inner paper states “n=16 per borough” across four boroughs (16×4=64) but the total is 48. I did not catch this arithmetic error. I will add a note in my wrapper section listing it among the AI’s uncorrected errors.

3. Errors and omissions in my wrapper: won't fix

These are more formal and don't really matter at this point, as the paper was accepted for the conference.

Anonymity (O C)
GitHub link and identifying self-citations (Powell & Caldas Cabral; causal mapping) will be removed or anonymised for review.

AI Involvement Checklist (O)
Writing score 3.5 is invalid (scale 1–4). I will correct to a valid score.

Formatting (O C)
References, headings, and template alignment will be brought into line with the CFP.

3. Errors and omissions in my wrapper: will fix

Authorship and voice (O C)
The inner paper uses “we” despite there being no human co-author. I will add an explicit note in my wrapper: who is speaking in the inner paper; what the human did (dataset choice, instruction framing, post-hoc checking, revision prompts); and where interpretive authority lies.

Autoethnographic reflection too thin (O C G)
I will expand the reflection to address the emotional/experiential dimension: what it felt like to watch the AI interpret vulnerable people’s accounts; moments of wanting to intervene; how “no actual person is the author” sat with me.

Model transparency (O C G)
I will specify the underlying model (Cursor’s default at the time) for reproducibility.

Ethics of AI processing (O C)
The data is already anonymised and public, but I will add a paragraph on data handling.

4. Where both I and the reviewer were wrong

Negative-case search claimed as “systematic” (O G C)

The inner paper says “deliberate negative-case search” (3.4) and “we deliberately searched for transcripts” (3.3). It never claims a “systematic pass.” My reflection incorrectly stated that the paper claims a systematic negative-case pass; the reviewers echoed that. I will correct my reflection.

Causal mapping of loneliness interviews

📅 24 Jan 2026

How to make sense of 48 interviews about loneliness¹?

- On the one hand we'd like to go beyond mere listing of counts of themes or tags to identify and present *the stories themselves*.
- On the other hand, we'd be pleased if the results were more than merely one researcher's response to the material and could perhaps even be to an extent reproduced, such that we'd get somewhat similar results when applying similar methods.

Those two bullet points are already, of course, dancing around a **whole firestorm** of argument around research methods, positionality, validity, having skin in the game, and so on.

But **bear with me**: I want to cut to the chase and suggest *causal mapping* is a great way to square this circle.

We applied causal mapping in a standardised way to these 48 interviews. The coding procedure is automated with AI but we primarily use it as a low-level assistant to do reproducible and transparent standardised coding, rather than as a "black box".

The result of this process, which took about 20 minutes, is a fully coded knowledge graph of 3392 identified causal claims, which can be interrogated in many different ways. Here is a top-level map. Having read some of the interviews (I know, right?) I'd say it's a pretty good summary of what these young Londoners were saying. Fatter links and boxes were mentioned more often; blue arrows and factors are judged to be associated with more positive sentiment.

It's only "human in the loop" in the sense that humans set up the procedure and kept an eye on the results and made a few tweaks here and there. But neither do we succumb to giving the AI much freedom to simply "identify the themes" or "identify the themes". So if neither human nor AI was doing any very deep sensemaking, where does the story which the map tells actually come from?

Could it be that there is something like, in overview, a "truth" about what the interviewees were saying?

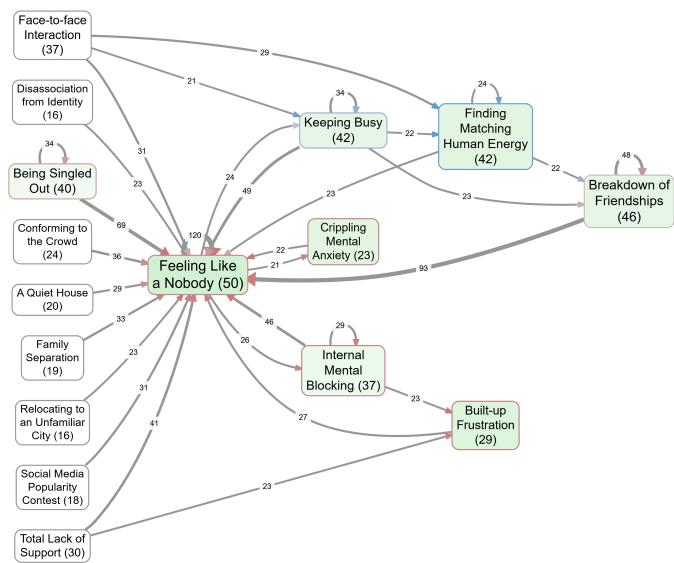
The (non-AI) algorithms behind the analyses are relatively elaborate and very deliberate, but they are transparent and reproducible. Their claim to be a valid way to synthesise and present causal links is checkable and can be challenged.

So, what do you think? Can we identify and present the stories in narrative data without having humans or LLMs in the loop?

This analysis is related to one I recently presented at the [Agents4qual](#) conference: [Lonely in London](#). It presented a "qualitative" paper which was entirely written by AI, mimicking traditional, iterative methods of identifying themes.

You can view and interrogate the causal map above, [here](#).

1. Fardghassemi, S., & Joffe, H. (2022). *Qualitative and output data on loneliness among young adults* (p. 15430034 Bytes) [Dataset]. University College London. <https://doi.org/10.5522/04/17212991> ↩



Assessing change in (cognitive models of) systems over time

📅 19 Jan 2026

The Architect and the Clerk: Analysing Central Bank speeches with GenAI-supported causal mapping

Abstract

Using Generative AI (GenAI) as a "mere assistant" to code text according to a pre-defined thematic codebook is possible but relatively uninteresting, because the AI is not involved in theory building. Many qualitative researchers are using GenAI for collaborative meaning-making, with and without a codebook. This paper presents another way, based on causal mapping, a well-established approach in social research which codes *only* causal claims within text. We use a "zero-shot" approach with no codebook, using only "In vivo" labels for the identified causes and effects. This results in heaps of causal claims containing large numbers of cause and effect labels. Making sense of these heaps is then done in two phases: firstly applying Large Language Models (LLMs) for semantic clustering and secondly using non-GenAI causal mapping techniques to visualise overall and divergent causal narratives within the text(s). This procedure is quite highly standardised and yet still depends on creative and iterative human input at key points: Qualitative judgement (what are the main cause/effect labels and how are they best organised in order to build an interesting and useful theory?) remains central while many of the other tasks become more reproducible, checkable, and scalable.

We demonstrate this approach with a corpus of speeches from leaders of Central Banks, asking: what drives what in the national and global economy, in the opinion of these experts? And how do these opinions change over time?

Introduction: Beyond the Conversation

GenAI can be used to automatically code themes according to a pre-defined codebook (Xiao et al. 2023). But this is a relatively uninteresting use of AI as it is used purely as a "clerk" and is not directly involved in theory building.

Recent debates about AI-assisted qualitative research invite us to give AI a bigger role. One way to do this is the "conversational" paradigm, where AI acts as a co-researcher in a dialogic, hermeneutic process (Friese 2025; n.d., Dai et al. (2023), (n.d.), Nguyen-Trung (2025)). That approach leverages the AI's capacity to identify meanings within larger passages of text and thus in various ways to take part in a conversation around theory building.

We propose a complementary, yet distinct, path. It is based on causal mapping, a social research approach technique that identifies and visualizes beliefs about "what causes what" (Nadkarni & Shenoy 2004; Scavarda et al. 2006; Ackermann et al. 2004; Eden et al. 1992; Axelrod 1976). In causal mapping, we code *only* the causal claims within a text. This can be done with a codebook, but more interesting is with a zero codebook, using only "In vivo" labels for causes and effects.

This preliminary task can be done by a "clerk" and does not need an "architect".

Causal mapping is a very good fit for the initial step of almost-automated coding because a) causal coding is quite surprisingly easy to automate on a page-by-page basis (Studdiford & Lupyan 2025; n.d.; Veldhuis et al. 2024; Powell et al. 2025), being a much more highly determined task than, say, the task of "identify themes within this text"; and b) extracted *causal* narratives are usually more interesting and closer to answering actual research questions than lists of themes (Britt et al. 2025; n.d.).

Making sense of — and building a theory around — the resulting heaps of causal claims (containing large numbers of cause and effect labels) is then done in two phases: first, applying LLM-supported semantic clustering and then using established (non-GenAI) causal mapping techniques to visualise the overall and divergent causal narratives within the text(s). Qualitative judgement (what are the main cause/effect labels and how are they best organised?) remains a central theory-building step. Most of the other tasks are reproducible, checkable, and easy to scale.

Case study dataset: Central Bank speeches (1996–2023)

We used a corpus of central bank speeches (1996–2023) Campiglio et al. (2025): 1,354 speeches spanning 1996–2023, sampling up to 20 speeches per year, resulting in a sample of 522 speeches, equivalent to 3934 pages at 500 words/page.

We use this corpus purely as an illustrative worked example; our contribution is methodological rather than domain-substantive.

Causal coding

We applied causal coding to this dataset. The coding prompts were quite restrictive. We did not, for example, ask the AI to "summarize" or "discuss themes." We instructed it simply to identify every specific instance where the text claimed X causally influences Y , and record only cause, effect and a supporting verbatim quote, without using a pre-existing codebook. We call this minimalist or "barefoot" causal coding: capturing explicit causal claims using the source's own vocabulary.

In practice, we iterated the extraction prompt and ran basic quality checks (spot-checking pages and links) until we were satisfied that the outputs were consistent with these instructions.

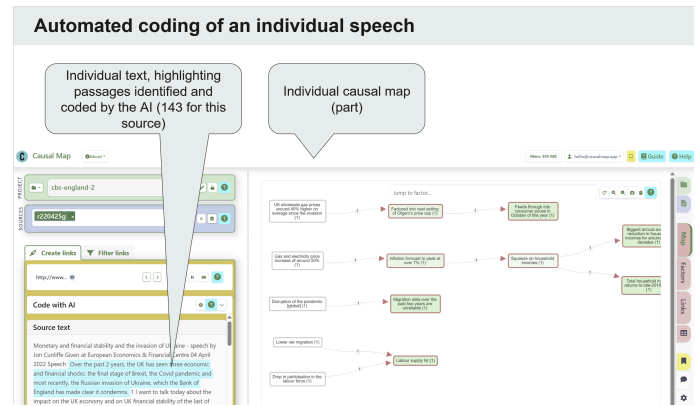


Figure 1: A screenshot of the Causal Map app after automatically coding one source.

The Creative Challenge: Magnetic Clustering as (part of) theory building

The output of this extraction is a "hairball" of thousands of links between thousands of unique factor labels often with overlapping meaning (e.g., "rising prices," "inflation," "cost of living increase"). It is here that the AI-supported process confronts the core challenge of standard qualitative inquiry: Theory building.

In most forms of qualitative coding, the researcher must decide which codes belong together to form clusters. This is a highly under-determined task. It is not completely arbitrary, because some sets of clusters will better cover the raw codes than others. But it is not well specified, because many alternative "theories" may cover the raw codes equally well. The assignment of codes to clusters as well as assessing the goodness of fit of this assignment is carried out through Magnetic Clustering, as follows.

1. **Identifying initial clusters:** Sorting thousands or, as in this case, tens of thousands of labels into clusters of similar meaning is a task which in principle humans can carry out but in practice the scale is overwhelming. So, each label (raw label and magnet) is represented as an **embedding**: a numerical vector encoding of meaning. In NLP this general idea underpins modern semantic search and vector retrieval, including transformer-based representations (e.g., BERT-style embeddings (Devlin et al. 2019)) and retrieval-augmented (RAG) pipelines that use vector indices (Lewis et al. 2021). Semantic similarity is measured by cosine similarity (the angle between embedding vectors). We then use a clustering algorithm (k-means) to create clusters of raw labels with similar meaning (Grootendorst 2022).
2. **Defining the Labels:** The next task is to generate labels for each cluster. This is a more feasible task for humans to do (usually there are 5-50 clusters with a handful of exemplars for each cluster) but in practice even this is tiresome over many iterations, so we employ GenAI to help but usually inspect and adjust the labels as our theoretical ideas of the subject matter evolve.
3. **The Attraction:** these first two steps are relatively standard procedure. But our next step differs: An algorithm "attracts" the thousands of raw, granular labels to these suggested cluster labels, which we call "Magnets", based on semantic similarity: each raw code is assigned to that Magnet to which it is semantically most similar. An assignment fails if the closeness of code to theme is below some similarity threshold, in which case the raw code is not assigned to any Magnet. The percentage of raw codes which succeed in being reassigned or "magnetised", for any given closeness threshold, can be called "coverage". Note **we do not**

actually use the original cluster solution other than to provide exemplars in order to create the magnetic labels, which then create their own clusters: the clusters of raw labels attracted to each magnet will be similar to the original solution but not identical to it. This is what makes our approach different from most clustering procedures: we **re-cluster the raw labels all over again** on the basis of their similarity to these Magnets, a process which can also be called "**soft recoding**". This means that we can tweak the Magnet labels on the fly, individually or severally, and watch as the raw labels quickly recluster themselves. We follow the principle of Braun and Clarke's (Braun & Clarke 2021) Reflexive Thematic Analysis that themes must show internal homogeneity (by using k-means clustering) and external heterogeneity (our labelling procedure, whether conducted by AI or ourselves, is explicitly designed to find labels which are distinct from one another in meaning). The original AI-generated labels provide a good start, but they are usually only the beginning of a process. For example, we might want to substitute one label with a pair of labels which are relatively similar in meaning — and so would not have been produced in the initial phase — but which represent for us an important theoretical distinction.

4. The Refinement: We iteratively tweak the "magnetism" and experiment with different lists of magnets.

Qualitative scholars usually distinguish between mere clusters of codes and *themes* which may form part of a more meaningful theory and which have a closer relationship to the research questions. In our case, it would be perfectly *possible* to use more theory-driven labels, but these would in general not function as well as magnets as they would usually be further away semantically from the original raw labels. Ideally we might discover labels which are both more interesting theoretically and nevertheless also fit the original language of the respondents. Failing this, we can use an additional filter to provide convenience labels for the original Magnets which describe them in terms of our evolving theory, for example we might relabel "Interest rate cuts" as part of a hierarchical system: "Monetary policy; interest rate cuts", in such a way that only "interest rate cuts" is given to the clustering algorithm but the full label is shown in user-facing outputs such as tables⁴.

This process of Magnet selection is related to the codebook development phase in, say, Thematic Analysis, but it is a soft (temporary) rather than a hard coding procedure. It is also possible to go back to the start and "hard-recode" the source texts again from scratch, using the Magnets list as a codebook. In practice, "soft-recoding" with magnetic labels is our preferred approach because we can test and compare the "fit" of different theoretical models to the data more or less in real time.

Results: A Query-able Qualitative Model

The result of this process is not a static narrative, but a dynamic causal map ,i.e. a query-able database of causal evidence) : a large number of links between a set of common labels (the "Magnets"). It is possible to simply list the "Magnets" and the frequency with which they were mentioned, or show a map with only the most frequently mentioned Magnets. While we can create standard frequency tables (see figure 2), the most interesting part of this process is being able to interrogate this qualitative model to answer more interesting questions, by applying a library of pre-set filters individually or in chains, as discussed in the next section.

Factor	Citation Count	Source Count	Citation Count: In	Citation Count: Out
Major global events [global]	2384	295	1005	1379
Inflation	1230	221	747	483
Financial market conditions	821	258	403	418
Lack of economic growth	818	242	514	304
Financial instability	783	267	406	377
Underlying economic factors	515	228	293	222
The macroeconomic policy environment	471	179	153	318
Long-term economic trends	386	167	224	162
Financial stability	366	161	191	175
Financial crisis management	293	134	129	164

Figure 2: The most frequently mentioned causal factors, in descending order of citation count. Also showing "Source Count" (number of sources mentioning the factor at least once) and "Citation Count: In" aka "Indegree" (number of mentions of the factor as an effect) and "Outdegree" (number of mentions of the factor as a cause).²(<https://app.causalmap.app/?bookmark=966>) 2026-01-12 16:44_] As an aside: Researchers used to most forms of systems modelling will find this kind of presentation confusing because some factors appear with both positive and negative variants (Financial stability and Financial instability) and some do not. But this simply reflects the sources' causal narratives. Causal mapping does not usually attempt to model a logical world of facts: it attempts to model the (often not very logical) mental models implicit within texts. So, if two opposed concepts appear separately within the text, by default we will simply use them as-is. Other techniques are available to combine opposed pairs of ideas, but they will not be discussed here. Combining opposites, sentiment.

Deciding which Magnets to use is a substantial qualitative decision that shapes the entire model and which falls under the researcher's theory-building responsibility even when using AI-assisted causal mapping.

However the results of the "magnetisation" process still does not reach the level of declarative conclusions which one would expect from a theory-building analysis. That is because the immediate result of the magnetic is not a single text but a query-able qualitative model, as discussed in the next section.

What are the main consequences of Brexit according to the sources?

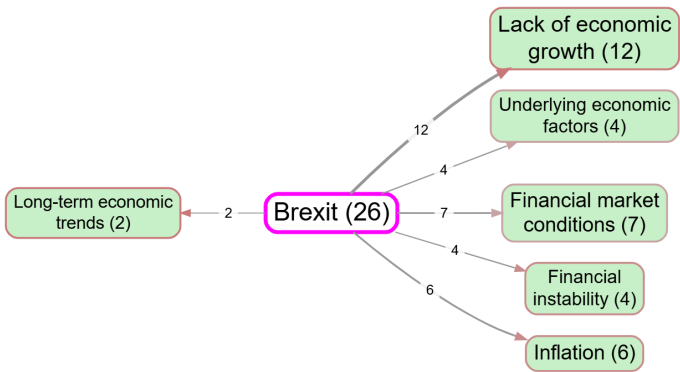


Figure 3: A screenshot of the Causal Map app after automatically coding one source. ³ In this case we apply a simple filter to explore the consequences of "Brexit": any Magnets listed as its immediate effects. We can explore the resulting map further, e.g. by inspecting the verbatim quotes associated with each link.

What are the top-level causal narratives in each decade (specifically, as explanations of growth) and how do they differ from one another?

Here, we use the same set of magnets for each decade, filter to show only paths leading to Economic growth or Lack of economic growth and then again show the most frequently mentioned links in each case.

Decade 1 1996-2005

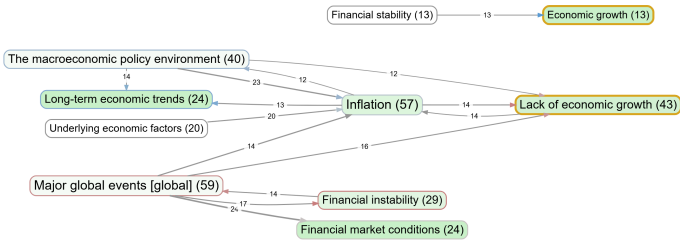
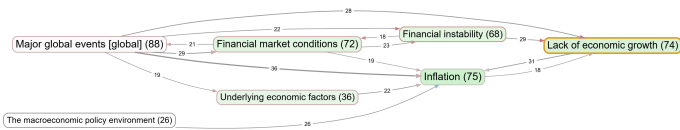


Figure x Filters applied: Soft recode: 18 magnets, similarity>=0.62; Source Groups: Decade=1; Path: to "Growth", 2 steps; Link frequency: top 19 by source count.

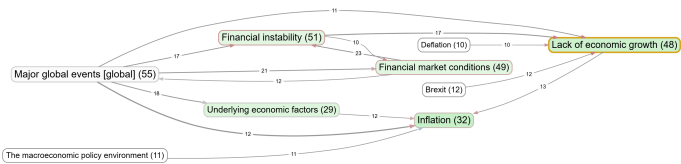
Decade 2 2006-2015



Bookmark #962 Filters applied: Soft recode: 18 magnets, similarity>=0.62; Source Groups: Decade=2; Path: to "Growth", 2 steps; Link frequency: top 19 by source count.

With these filters, retaining only the most frequent links, there is no more mention of: Long-term economic trends or of Financial stability leading to economic growth.

Decade 3 2016-2023



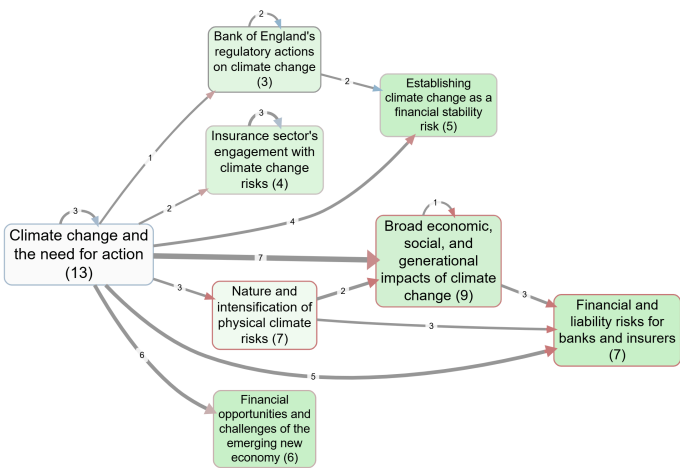
Bookmark #963 2026-01-13 09:12_

Retaining only the most frequent factors and links, now we see the first mentions of Brexit, Deflation and Climate change.

This technique allows us to systematically compare maps across groups or time-points. This is something which is much harder to do reliably using human coders. Concepts such as inter-rater reliability are often treated as a distraction by qualitative researchers and there are good reasons for that: but using the techniques described here we can have our cake and eat it: get involved in in-depth theory construction and have reliable and reproducible coding too, at the scale of hundreds of sources.

What are the narratives around climate change?

In this case, climate change and its specific causes and effects did not appear frequently enough to appear much amongst the most important Magnets. So we simply filtered for all mentions of climate change (as causes and effects) amongst the *raw* labels and created new Magnets to cover just this material.



Filename: cbs-england-2-filtered-2. Citation coverage 0% of all sources: 59 citations shown out of 37648. Factors — size: citation count; numbers: source count; colour: outcomeness; border: avg incoming sentiment (blue=positive, grey=neutral, red=negative). Links — width: citation count; labels: source count; arrowheads: effect sentiment (blue=positive, grey=neutral, red=negative). Filters applied: Sources included: All sources. Labels: climate chang, climate crisis, global warm; Soft recode+: 10 magnets, sim=0.62; Link freq: minimum 2 citations. Bookmark #967 2026-01-13 09:24

This is also a good place to showcase and additional way to involve GenAI in this kind of research: the entirely automated production of narrative vignettes which describe any given map and the data behind it, using a standardised prompt. The results are as follows.

Central Bankers Warn: Climate Crisis Poses Significant Threat to UK Financial Stability

This analysis, derived from a highly filtered view of UK central bankers' speeches, outlines their perspectives on climate change, focusing on its implications for the financial system and broader economy.

The discourse among UK central bankers, as captured in this data, predominantly frames **Climate change and the need for action** (40 citations) as a foundational premise. This central theme is consistently reported to lead to a range of significant negative outcomes across the economy and financial sector.

A primary pathway identified is the link between **Climate change and the need for action** and **Broad economic, social, and generational impacts of climate change** (11 citations, average sentiment -0.727). The sources frequently highlighted the severe negative implications for society and future generations. Furthermore, it was observed that climate change directly contributes to **Financial and liability risks for banks and insurers** (6 citations, average sentiment -1), indicating a clear concern for the financial sector's direct exposure.

The **Nature and intensification of physical climate risks** (11 citations) is another critical factor, with reports indicating its direct connection to **Climate change and the need for action** (3 citations, average sentiment -1). These physical risks, in turn, were mentioned as exacerbating both **Financial and liability risks for banks and insurers** (3 citations, average sentiment -1) and the **Broad economic, social, and generational impacts of climate change** (5 citations, average sentiment -1). This suggests a cascading effect where physical climate events translate into tangible financial and societal burdens.

The role of regulatory bodies, specifically the **Bank of England's regulatory actions on climate change** (8 citations), is also a notable theme. These actions were reported to contribute to **Establishing climate change as a financial stability risk** (2 citations, average sentiment 0.5), underscoring the central bank's proactive stance in integrating climate considerations into financial oversight. The sources also mentioned that **Climate change and the need for action** itself prompts these regulatory responses (2 citations, average sentiment -0.5).

While the focus is largely on risks, the speeches also touched upon **Financial opportunities and challenges of the emerging new economy** (6 citations, average sentiment -0.333), indicating an awareness of the dual nature of the transition to a greener economy.

The analysis reveals a strong emphasis on the *effects* of climate change rather than its scientific *causes*. The factor **Climate change and the need for action** serves as the primary causal node, acting as a given and urgent reality. From this starting point, the discourse largely explores the subsequent impacts on financial stability, economic performance, and societal well-being. The central bankers' speeches, therefore, function more as an assessment of the consequences and necessary responses to climate change, rather than an exploration of its origins. The most cited links consistently describe negative effects, such as the "broad economic, social, and generational impacts" and "financial and liability risks for banks and insurers," highlighting a clear concern for the downstream implications of a changing climate.

Discussion: Complementarity and the Architect

The “dialogue” described here is not between human and GenAI directly, but between human and the causal mapping process, mediated by GenAI which does the original coding, suggests magnetic labels and provides summary vignettes (and by the LLMs which provide the label embeddings and underly the clustering process). Although the procedure presented here is quite highly standardised in outline, each step may be iterated several times either alone, or going back one or more steps to revise preceding steps as well. What causal mapping adds is an almost universally applicable way to extract and visualise *what led to what*: a key, theory-adjacent aspect of almost any set of narratives.

The “conversational paradigm” makes liberal use of GenAI as a human-like research assistant who is asked questions like “identify the themes” or “summarise this document”, even at the very beginning of the research, following an argument that coding (Nguyen-Trung & Frieze 2025) is a perhaps redundant, “skeuomorphic” remnant of the age before AI. But giving such immense degrees of freedom to any interpretative task exposes it to large, arbitrary and poorly defined influences from the analyst, whether human or machine. At least we can hope that the human analyst may be at least partially aware of their own influence on such tasks, due to mood, tiredness or positionality or whatever. Humans’ ability to actually understand the influence of these factors on their analysis is of course limited and error-prone. Machines are less likely to be influenced by situational factors but their performance is of course massively influenced by their architecture and training data in a way that they are unlikely to be actually aware of, regardless of what they say if we ask them.

Coding is one way to reduce this exposure to arbitrary influences.

This paper complements the conversational paradigm and shows that standardising and automating many parts of the workflow does not mean dumbing it down. By delegating the massive cognitive load of extraction and clustering mostly to the AI, we free the researcher to focus on the architecture of meaning. The “Architect” is asked for creative input in a well-defined way at well-defined, specific points in the workflow. But it remains a demanding task. Getting the clusters “right” and applying the right filters to create relevant maps requires deep, iterative engagement with the research question. The AI provides the bricks; the human must still design the house.

AI Contribution Disclosure Checklist

- Research Design: Human led (definition of Causal Mapping logic).
 - Data Collection: Human/Existing Data.
 - Data Analysis (Prompt engineering): human.
 - Data Analysis (Coding): AI (Radical Zero-Shot extraction).
 - Data Analysis (Clustering): Collaborative (AI performed clustering; Human defined “Magnets” and iteratively refined structure).
 - Data Analysis (Answering questions): Mostly human, leveraging existing non-AI algorithms/filters.
 - Initial drafting of Paper: AI (Gemini), based on human-provided structural constraints and source files.
 - Refining and Editing: Human.
1. These labels, like every label in causal mapping, can also form a hierarchical structure — “Improved economic conditions; improved investor confidence” as well as “Improved economic conditions”, but we do not use this extensively in the current study). ↩
 2. _Filename: cbs-england-2-filtered-2. Citation coverage 8% of all sources: 3065 citations shown out of 37648. Filters applied: Sources included: All sources. Soft recode+: 18 magnets, sim=0.62; Link freq: top 40 by source count. Bookmark [#966 ↩
 3. Legend: Factors — size: citation count; numbers: source count; colour: outcomeness (darker=more incoming links); border colour: average incoming sentiment (blue=positive, grey=neutral, red=negative). Links — width: citation count; labels: source count; arrowheads: sentiment of effect (blue=positive, grey=neutral, red=negative). Filters applied: Soft recode: 18 magnets, similarity>=0.62; Path: from “Brexit”, 1 steps; Link freq: minimum 2 sources. ↩

References

Ackermann, Eden, & Cropper (2004). *Getting Started with Cognitive Mapping*.

Axelrod (1976). *The Analysis of Cognitive Maps*. In *Structure of Decision : The Cognitive Maps of Political Elites*.

Braun, & Clarke (2021). *Thematic Analysis : A Practical Guide*. SAGE Publications Ltd.

<https://www.torrossa.com/it/resources/an/5282292>.

- Britt, Powell, & Cabral (2025). *Strengthening Outcome Harvesting with AI-assisted Causal Mapping*. https://5a867cea-2d96-4383-acf1-7bc3d406cdeb.usrfiles.com/ugd/5a867c_ad000813c80747baa85c7bd5ffafo442.pdf.
- Campiglio, Deyris, Romelli, & Scalisi (2025). *Warning Words in a Warming World: Central Bank Communication and Climate Change*. <https://doi.org/10.1016/j.euroecorev.2025.105101>.
- Dai, Xiong, & Ku (2023). *LLM-in-the-loop: Leveraging Large Language Model for Thematic Analysis*. <https://arxiv.org/abs/2310.15100v1>.
- Devlin, Chang, Lee, & Toutanova (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. <http://arxiv.org/abs/1810.04805>.
- Eden, Ackermann, & Cropper (1992). *The Analysis of Cause Maps*. <https://onlinelibrary.wiley.com/doi/10.1111/j.1467-6486.1992.tb00667.x>.
- Friese (2025). *Conversational Analysis with AI - CA to the Power of AI: Rethinking Coding in Qualitative Analysis*. <https://doi.org/10.2139/ssrn.5232579>.
- Grootendorst (2022). *BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure*. <https://doi.org/10.48550/arXiv.2203.05794>.
- Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, & Kiela (2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. <http://arxiv.org/abs/2005.11401>.
- Nadkarni, & Shenoy (2004). *Nadkarni and Shenoy 2004 -A Causal Mapping Approach*.Pdf.
- Nguyen-Trung (2025). *ChatGPT in Thematic Analysis: Can AI Become a Research Assistant in Qualitative Research?*. <https://doi.org/10.1007/s11135-025-02165-z>.
- Nguyen-Trung, & Friese (2025). *On Methodological Incongruence in Applying Generative AI in Qualitative Data Analysis*. <https://doi.org/10.2139/ssrn.5874482>.
- Powell, Cabral, & Mishan (2025). *A Workflow for Collecting and Understanding Stories at Scale, Supported by Artificial Intelligence*. SAGE PublicationsSage UK: London, England. <https://doi.org/10.1177/13563890251328640>.
- Scavarda, Bouzdine-Chameeva, Goldstein, Hays, & Hill (2006). *A Methodology for Constructing Collective Causal Maps**. <https://doi.org/10.1111/j.1540-5915.2006.00124.x>.
- Studdiford, & Lupyan (2025). *Contextual Effects in LLM and Human Causal Reasoning*. <https://openreview.net/forum?id=BMHkg3BL6e>.
- Veldhuis, Blok, family=Boer, Kalkman, Bakker, & family=Waas (2024). *From Text to Model: Leveraging Natural Language Processing for System Dynamics Model Development*. <https://doi.org/10.1002/sdr.1780>.
- Xiao, Yuan, Liao, Abdelghani, & Oudeyer (2023). *Supporting Qualitative Analysis with Large Language Models: Combining Codebook with GPT-3 for Deductive Coding*. In *28th International Conference on Intelligent User Interfaces*. <https://doi.org/10.1145/3581754.3584136>.